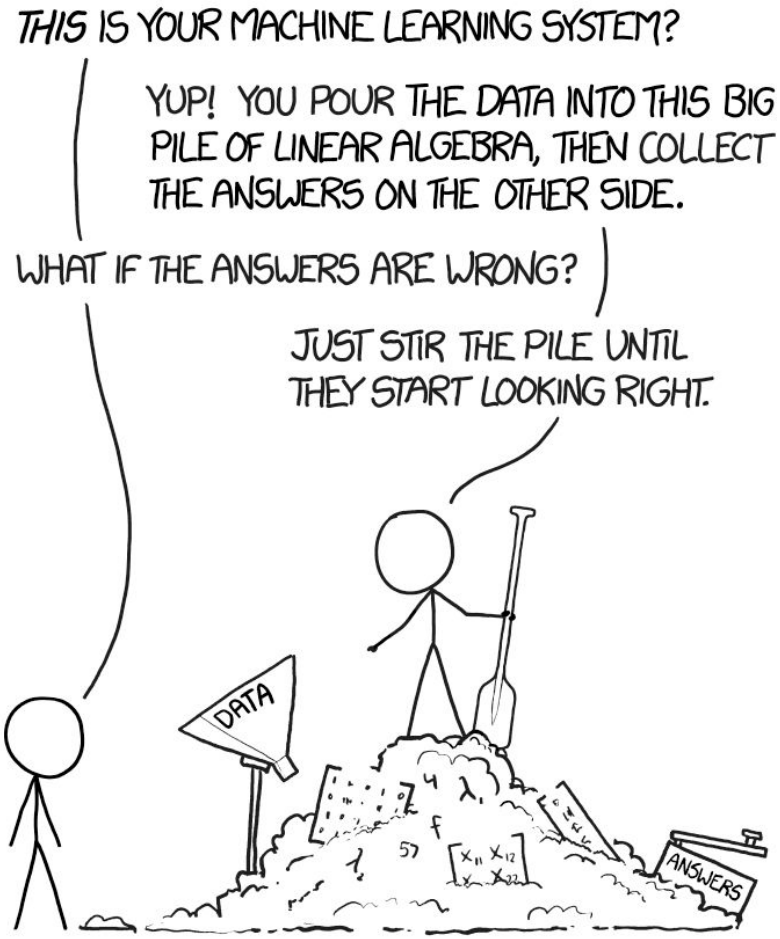
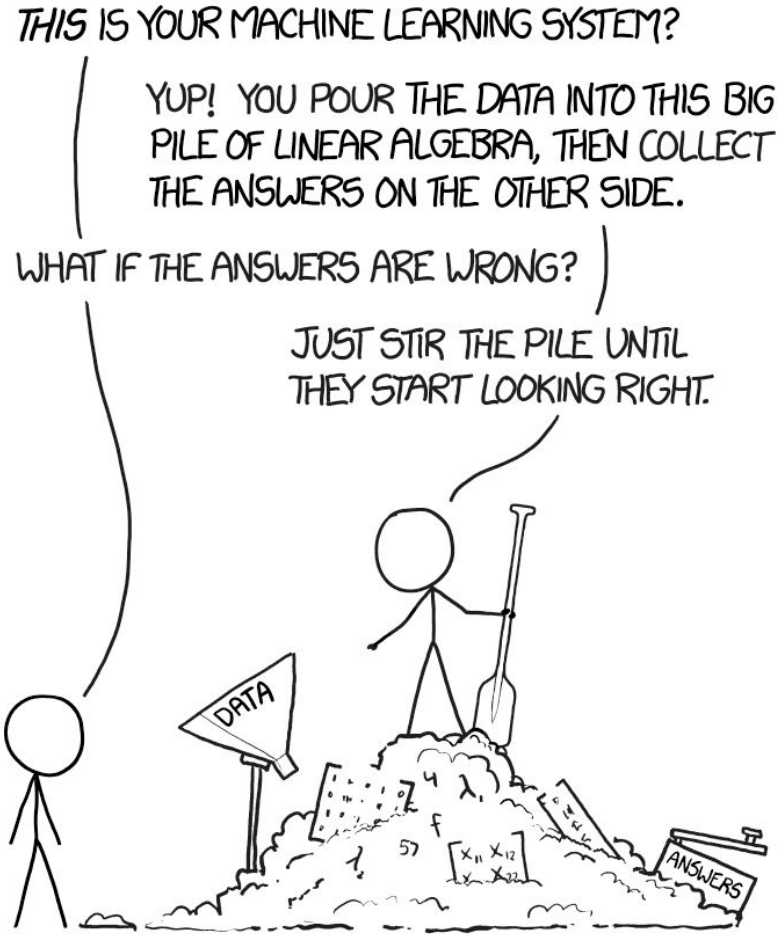
Symmetries and architectures

AstroAI Asian Network Summer School, Taipei 2026

Vera Maiboroda, IPMU UTokyo


THIS IS YOUR MACHINE LEARNING SYSTEM?
YUP! YOU POUR THE DATA INTO THIS BIG PILE OF LINEAR ALGEBRA, THEN COLLECT THE ANSWERS ON THE OTHER SIDE.
WHAT IF THE ANSWERS ARE WRONG?
JUST STIR THE PILE UNTIL THEY START LOOKING RIGHT.
DATA
ANSWERS

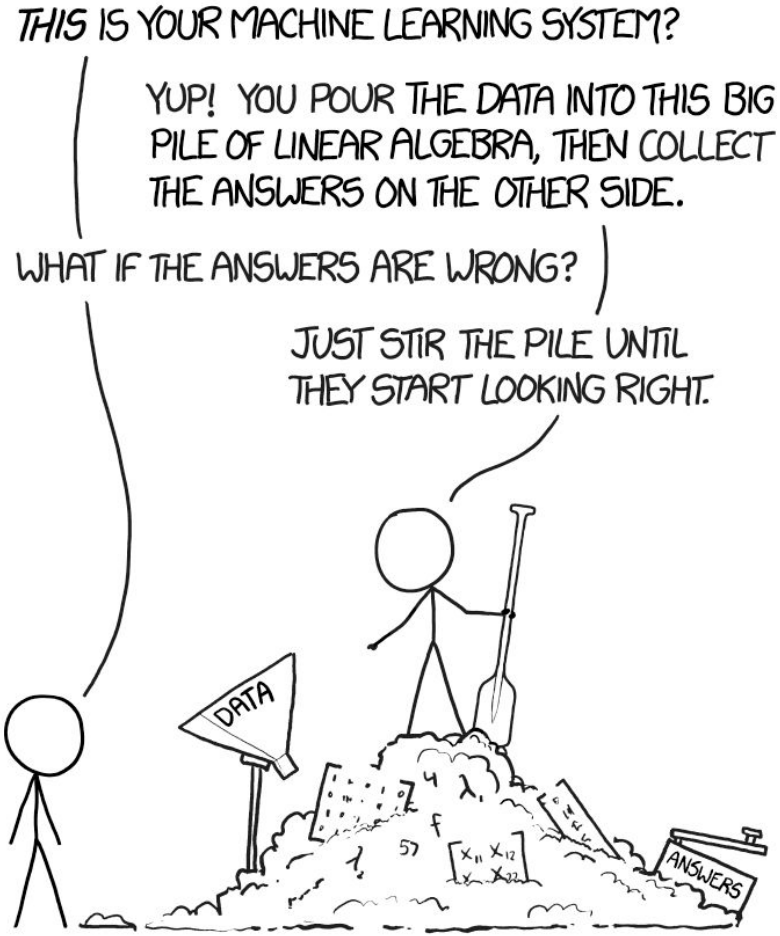


Bottom-up (ML)

large amount of (labeled) training data to learn from

✅ *High flexibility/adaptability*

❌ *Unconstrained model can lead to unphysical/unreliable outcomes*



Top-down (physics)

strong modeling assumptions from physical laws

✅ *Principled and interpretable models*

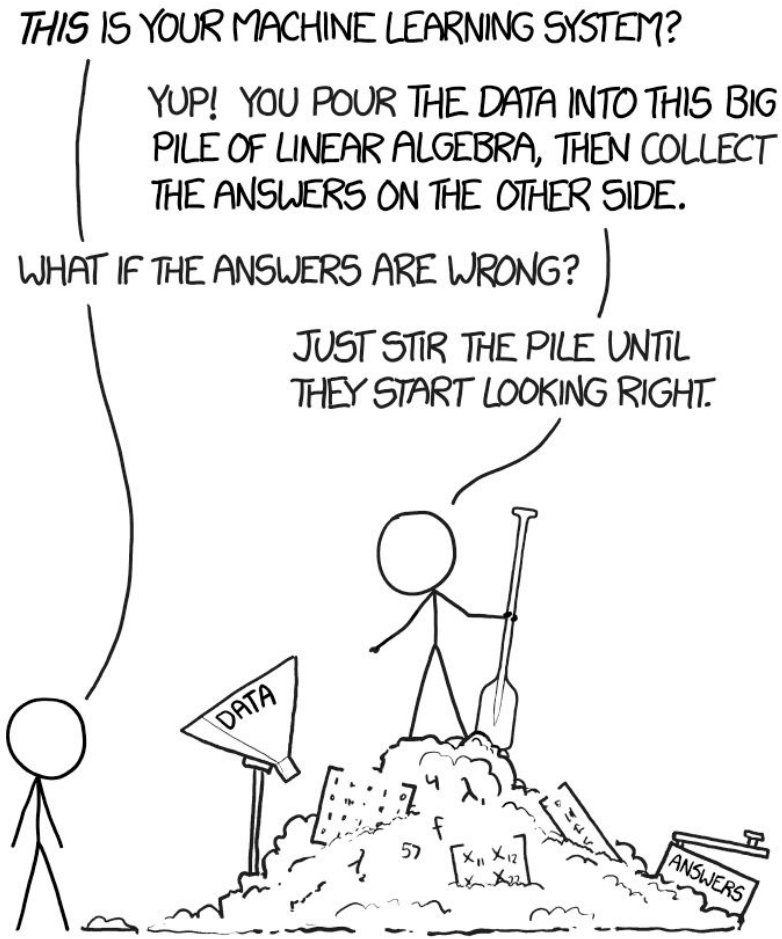
❌ *Strong modeling assumptions may not fit real data well. Complex physical systems are hard to model (intractable theory)*

Bottom-up (ML)

large amount of (labeled) training data to learn from

✅ *High flexibility/adaptability*

❌ *Unconstrained model can lead to unphysical/unreliable outcomes*



Top-down (physics)

strong modeling assumptions from physical laws

✅ *Principled and interpretable models*

❌ *Strong modeling assumptions may not fit real data well. Complex physical systems are hard to model (intractable theory)*

Physics-guided ML

Incorporating physics assumptions in expressive models

Bottom-up (ML)

large amount of (labeled) training data to learn from

✅ *High flexibility/adaptability*

❌ *Unconstrained model can lead to unphysical/unreliable outcomes*

Physics-guided ML
Incorporating physics assumptions in expressive models

… inductive bias

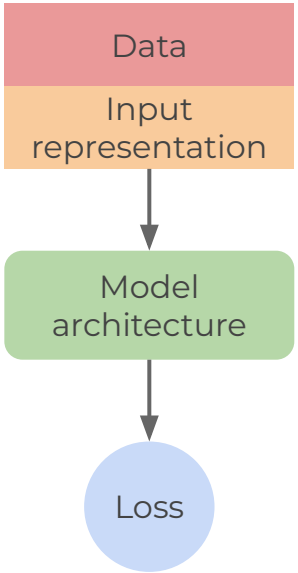
> An inductive bias allows a learning algorithm to prioritize one solution (or interpretation) over another, independent of the observed data. Inductive biases can express assumptions about either the data-generating process or the space of solutions.

Relational inductive biases, deep learning, and graph networks

- A priori hypotheses on properties of the **model solution** and the optimal **data representation**
- **Assumptions** about the learning objective and expected solution can guide the design of **effective** ML algorithms
- Assumptions can be translated in **induced bias** on the algorithm

Physics-guided ML
Incorporating physics assumptions in expressive models

… inductive bias



Assumptions → induced bias:

- Implicit bias (data augmentation):
  hand-craft training examples or their abundance to help the model learn specific features of the problem
- Representation bias (data preparation):
  choose which information to feed into the algorithm and in which form (data structure and selection)
- Hardware constraint (architecture design):
  hard-code symmetry constraints (invariance or equivariance) in the model architecture
- Software constraint (loss function design):
  soft-impose conservation laws adding penalty terms in the loss function






Observation:
In Switzerland I came across across a cow with spots and a cowbell

Possible generalization hypothesis:

There are cows in Switzerland

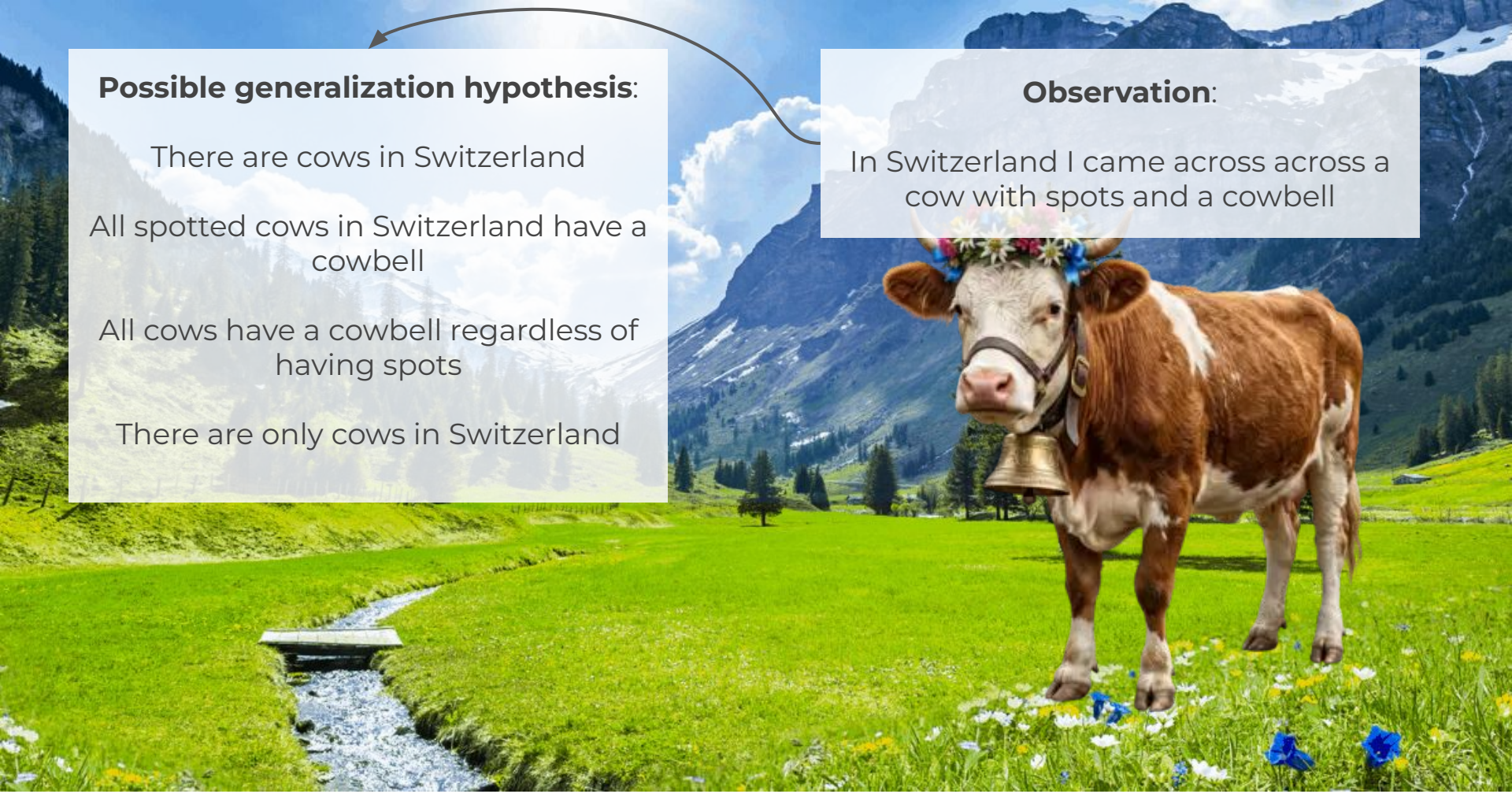
All spotted cows in Switzerland have a cowbell

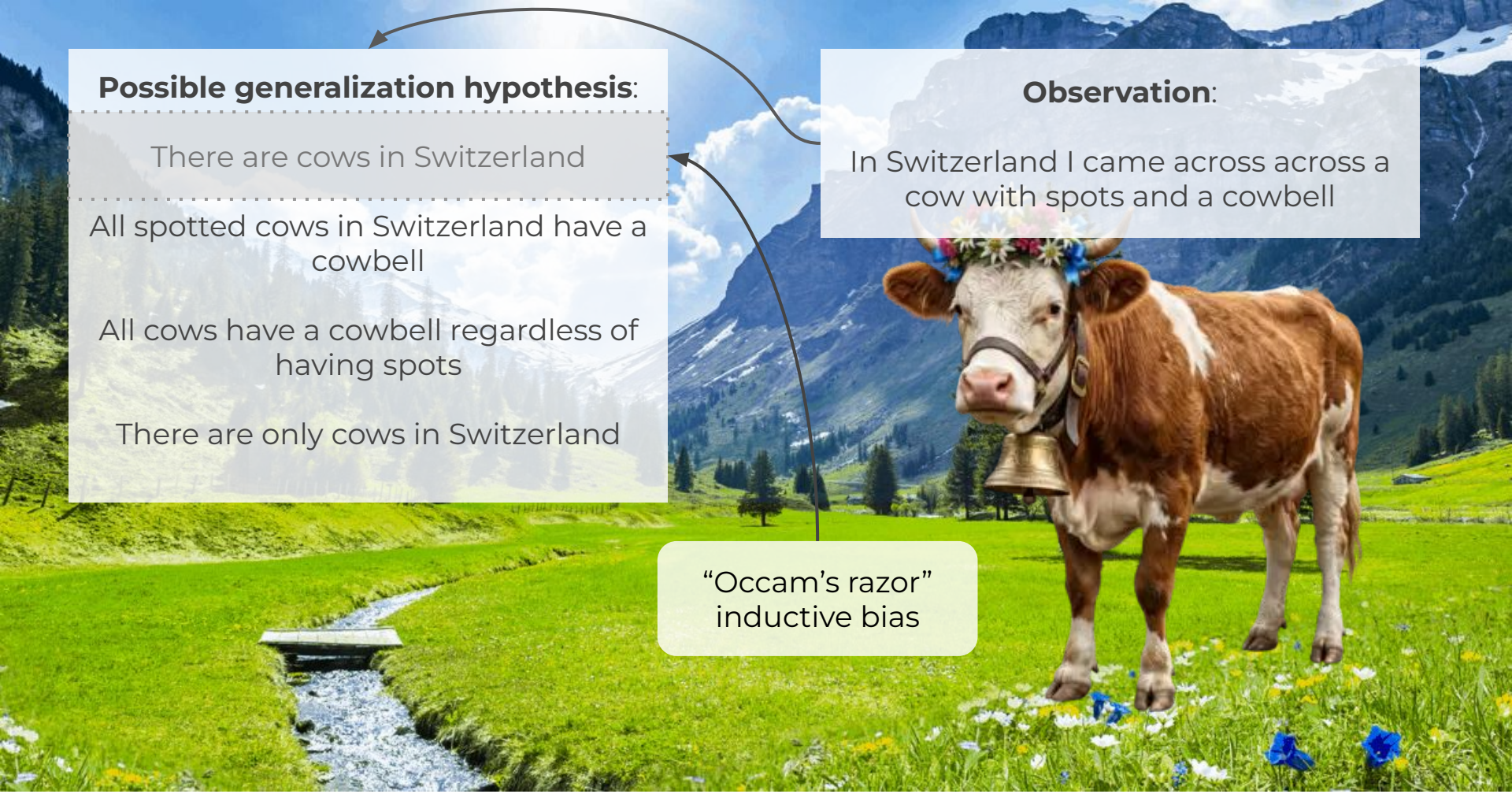
All cows have a cowbell regardless of having spots

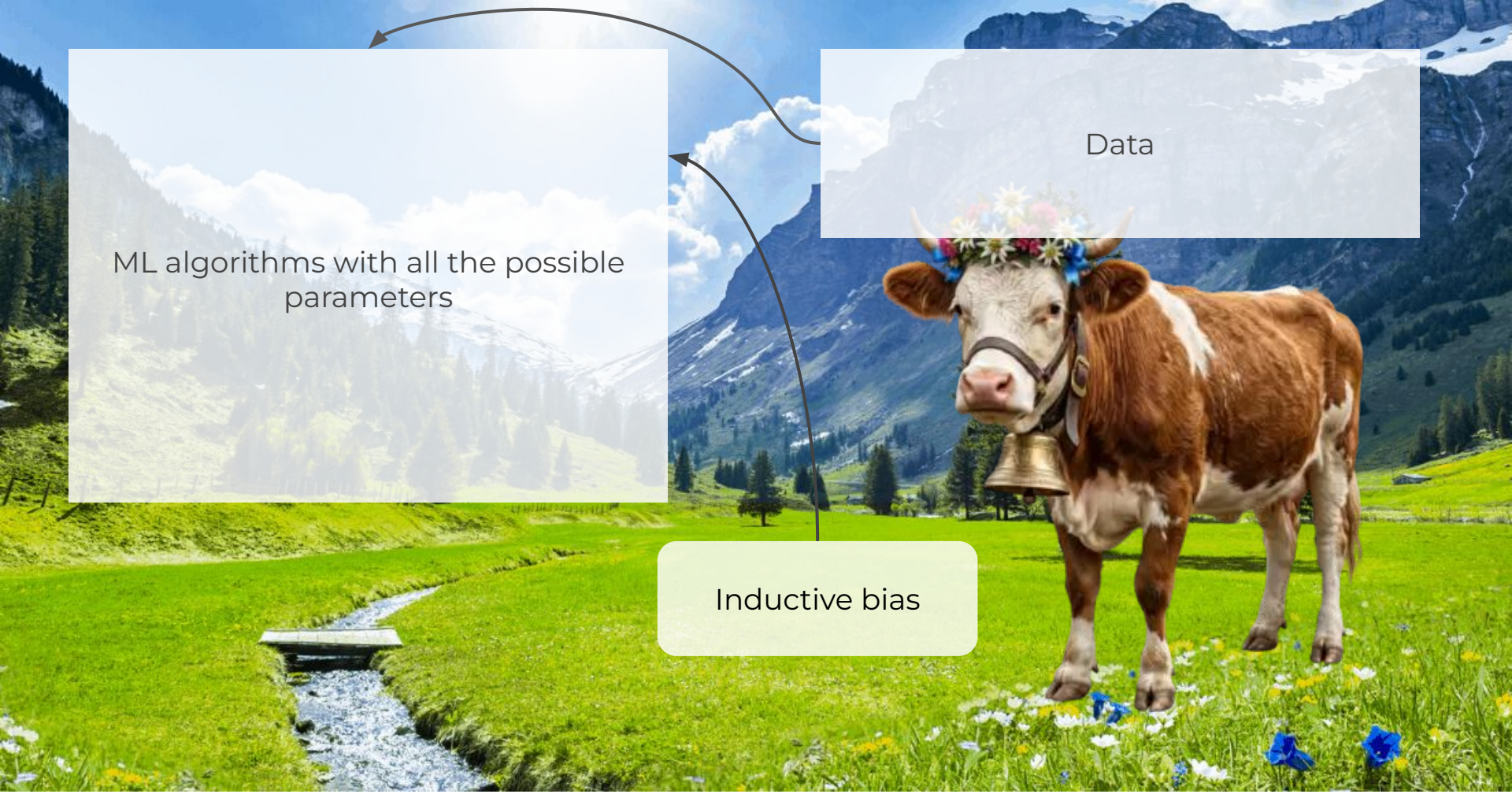
There are only cows in Switzerland

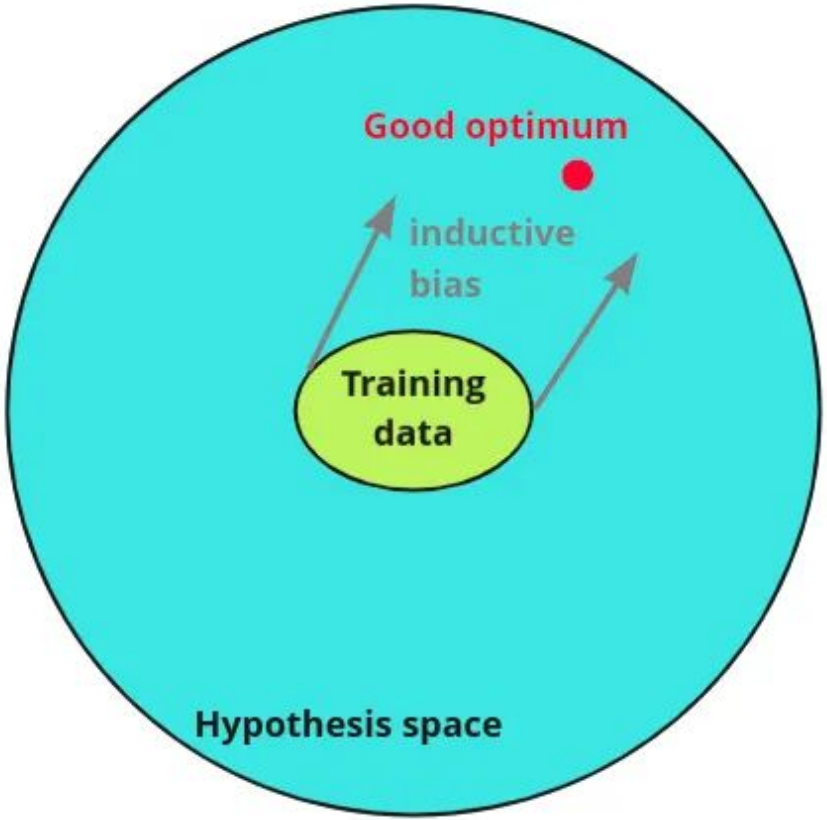
Observation:

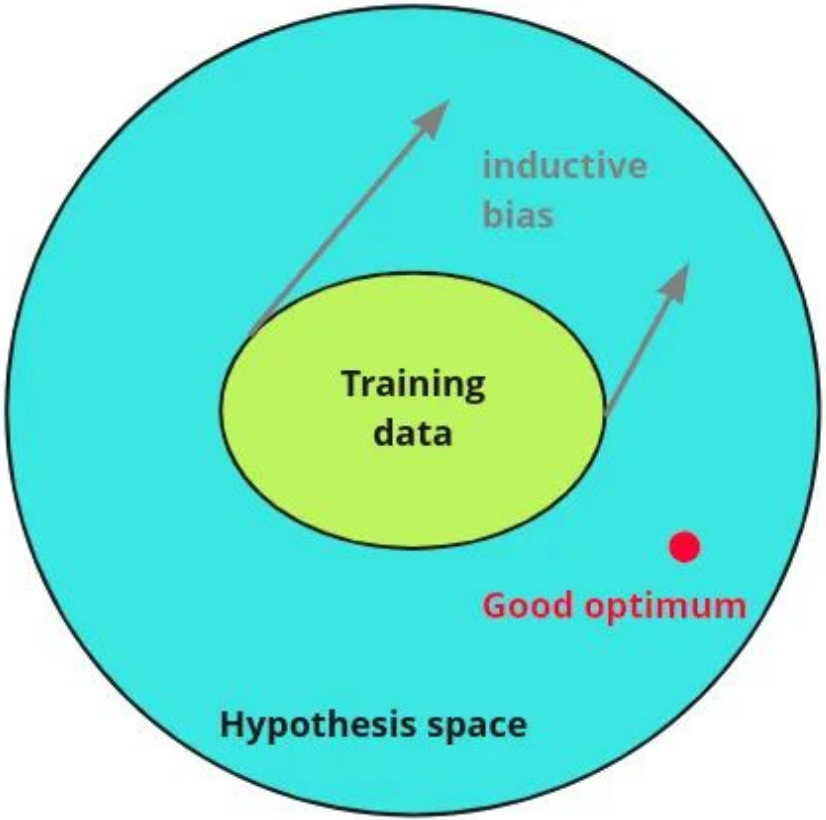
In Switzerland I came across across a cow with spots and a cowbell

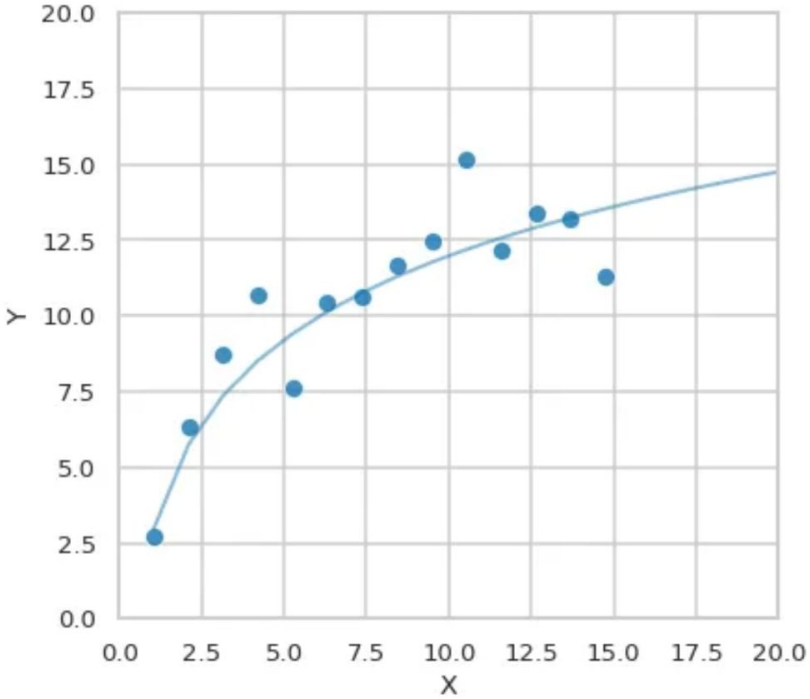


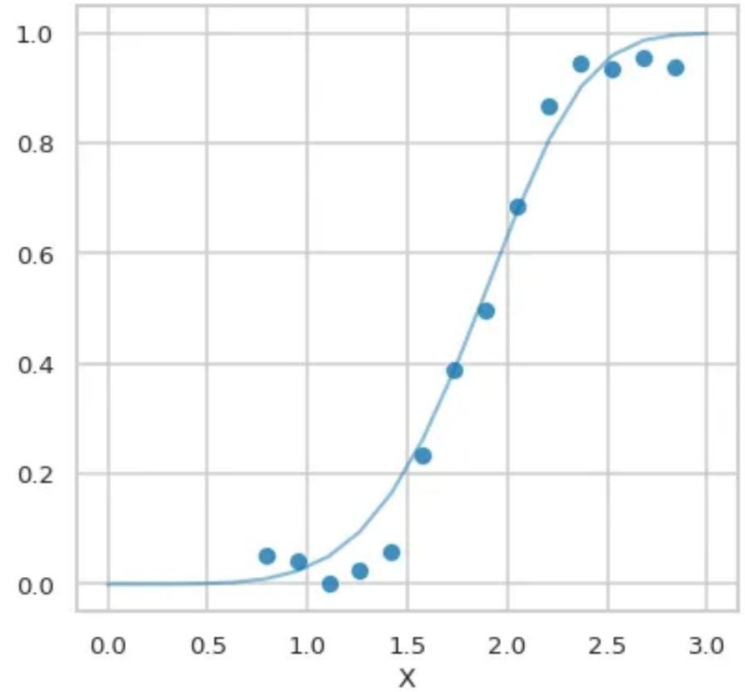

Possible generalization hypothesis:
There are cows in Switzerland
All spotted cows in Switzerland have a cowbell
All cows have a cowbell regardless of having spots
There are only cows in Switzerland
Observation:
In Switzerland I came across across a cow with spots and a cowbell
“Occam’s razor” inductive bias


Data
ML algorithms with all the possible parameters
Inductive bias

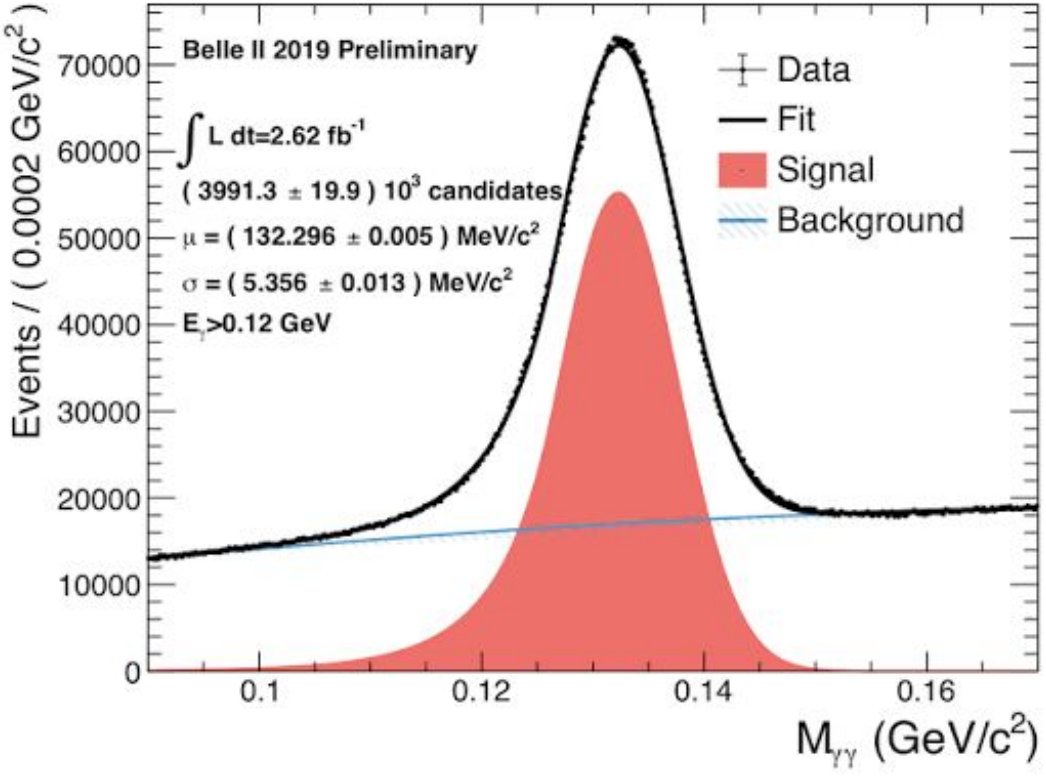

Good optimum
inductive bias
Training data
Hypothesis space

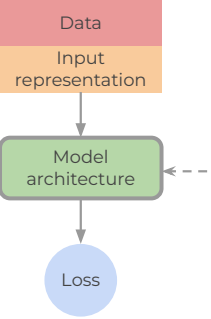

inductive bias
Training data
Good optimum
Hypothesis space



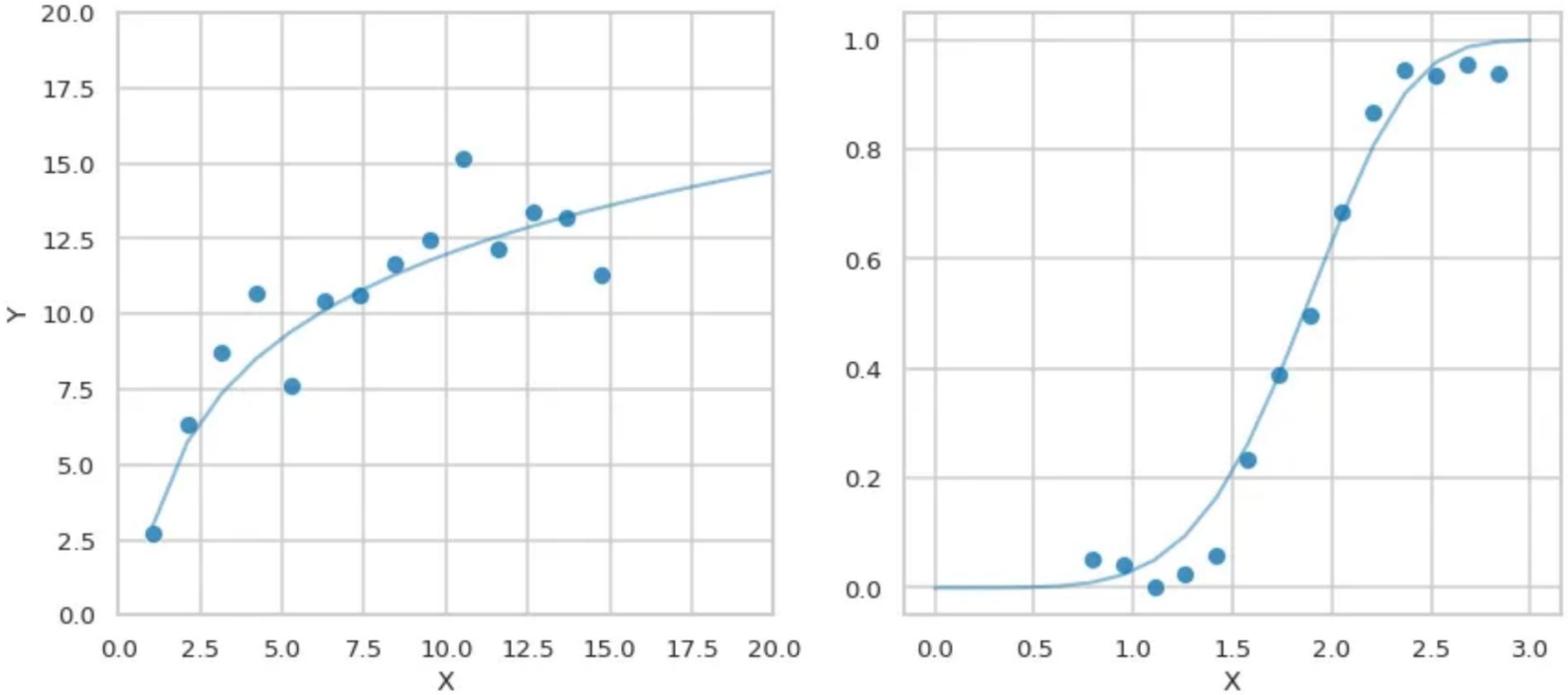


Models constrained to be the form of specific equations f(x)=k*log(m*x) (left) or f(x)=1-exp(-(x/k)^b) (right).


Belle II 2019 Preliminary
∫ L dt=2.62 fb-1
(3991.3 ± 19.9) 10^3 candidates
μ = (132.296 ± 0.005) MeV/c^2
σ = (5.356 ± 0.013) MeV/c^2
Eγ>0.12 GeV
Data
Fit
Signal
Background
Events / (0.0002 GeV/c^2)
Mγγ (GeV/c^2)

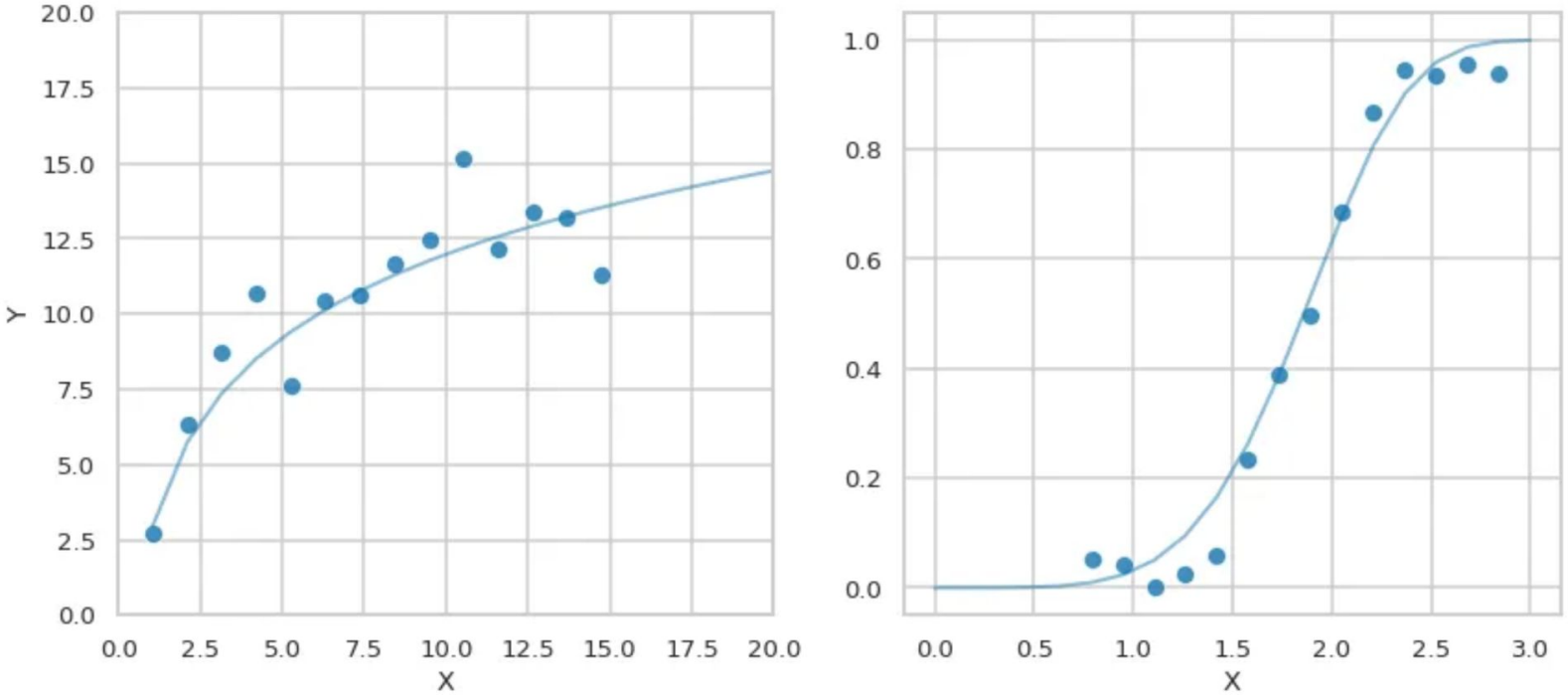


Architecture design



Models constrained to be the form of specific equations f(x)=k*log(m*x) (left) or f(x)=1-exp(-(x/k)^b) (right).

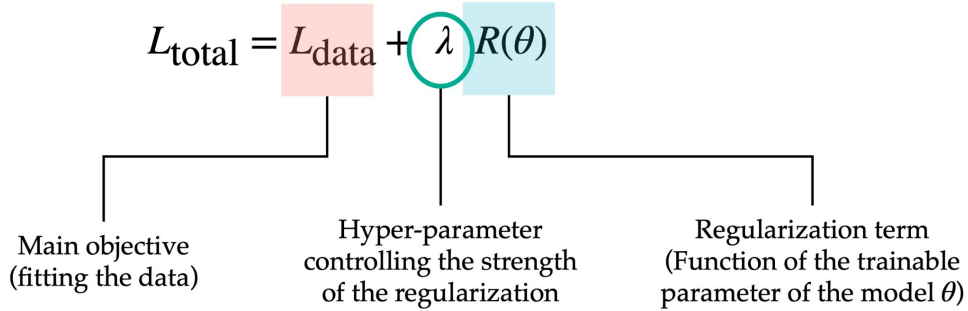
Loss function design

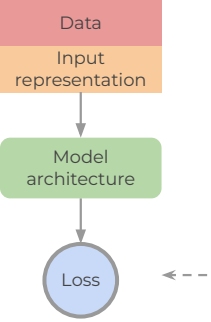


Models constrained to be the form of specific equations f(x)=k*log(m*x) (left) or f(x)=1-exp(-(x/k)^b) (right).

Loss function design

Penalty constraints in the loss function bias the model solution towards a preferred family of models.





Loss function design

Penalty constraints in the loss function bias the model solution towards a preferred family of models.

$$L_{\text{total}} = L_{\text{data}} + \lambda R(\theta)$$

- L_{data}: Main objective (fitting the data)
- λ: Hyper-parameter controlling the strength of the regularization
- $R(\theta)$: Regularization term (Function of the trainable parameter of the model θ)

Common penalties:

- **L2 regularization (weight decay):** $R(\theta) = ||\theta||_2^2$ (promotes *smoothness*)
- **L1 regularization:** $R(\theta) = ||\theta||_1$ (promotes *sparsity*)
- **Entropy regularization:** $R(p) = \sum_i p_i \log(p_i)$ (encourages *uncertainty* and *exploration* in a probabilistic output)

Loss function design

Penalty constraints in the loss function bias the model solution towards a preferred family of models.

$$L_{\text{total}} = L_{\text{data}} + \lambda R(\theta)$$

- L_{data}: Main objective (fitting the data)
- λ: Hyper-parameter controlling the strength of the regularization
- $R(\theta)$: Regularization term (Function of the trainable parameter of the model θ)

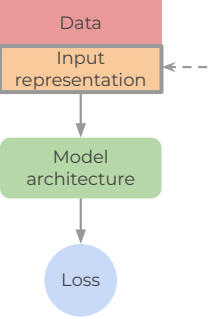
Common penalties:

- **Physics-based constraints** encourage to satisfy conservation laws
  - **Physics Informed NN (PINN)**: is a neural network trained not just to fit data, but also to satisfy a known physical law, typically expressed as a partial differential equation (PDE):

$$R_{\text{phys}}(\theta) = \frac{1}{M}\sum_{j=1}^{M} \left| \mathcal{F}[f_\theta](x_j, t_j) \right|^2$$

Partial differential operator: $\mathcal{F}[f] = \partial_t f + u\partial_x f$

Must satisfy: $\mathcal{F}[f(x,t)] = 0$, for $x \in \Omega$, $t \in [0,T]$



Representation bias

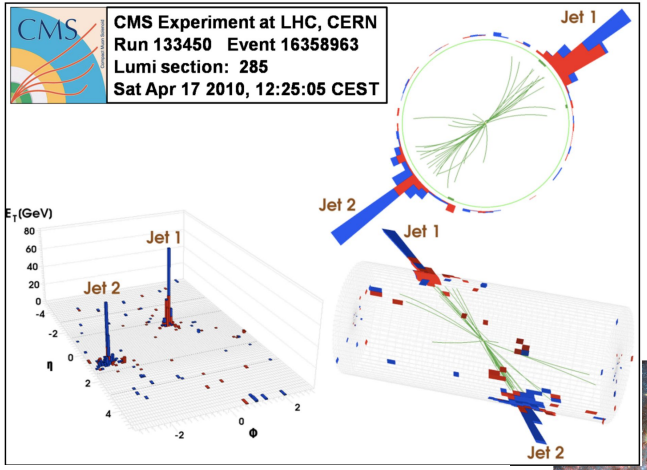
Identifying relevant structures in data and preserve them in their representation is the first step for successfully solve a machine learning task.

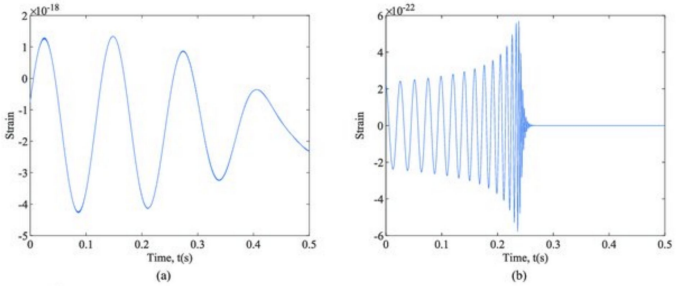
Representation bias

Identifying relevant structures in data and preserve them in their representation is the first step for successfully solve a machine learning task.

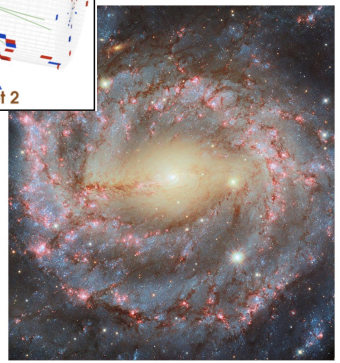
Physics data are typically highly *structured* (*t*ime series, images, point clouds, …)

Different mathematical representations of the data capture different *relations* between pieces of information encoded in the data.

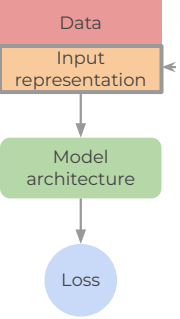




10.1109/ACCESS.2021.3139850



ESA/Hubble & NASA, A. Riess, D. Thilker, D. De Martin (ESA/Hubble), M. Zamani (ESA/Hubble)



Representation bias

Graph-structured data

Generally, we can describe the *relations* between properties encoded in data using graphs.
Graphs are mathematical structures defined[1] as a 3-tuple

$$G = (\mathbf{u}, V, E)$$

Where:

- $\mathbf{u}$ are the **global attributes** (e.g. feature of the system encoded in the graph)
- $V = \{\mathbf{v}_i\}_{i=1}^{N_v}$ is the set of **nodes** (of cardinality N_v) and each $\mathbf{v}_i$ represents a **node attributes**
- $E = \{(\mathbf{e}_k, s_k, r_k)\}_{k=1}^{N_e}$ is the set of **edges** (of cardinality N_e) and each $\mathbf{e}_k$ represents an **edge attributes**, s_k the index of the sender and r_k the index of the receiver.

Physics data can generally be encoded in graphs.
Depending of the specific properties of the data, special types of graph can be used.

Representation bias

Grid-organized data

In many physics applications collected data are organized in a *regular grid* in (Euclidean) space.
These data can be described using *graphs*.

Images:
Images are a special case of graph where nodes are organized in a *2D lattice* in Euclidean space.
Each pixel has a *fixed set of neighbors*. Spacial relations between neighbors define the edges of the graph.

Examples of images in experimental physics data are: astronomical images, 1-layer detector hits

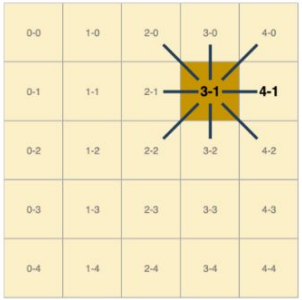
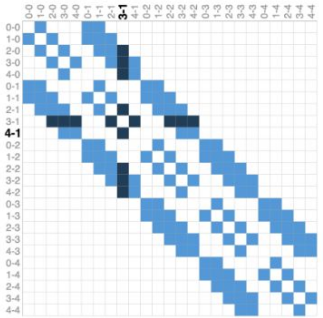
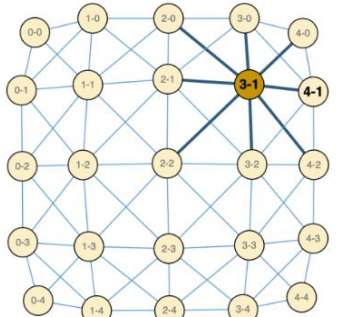


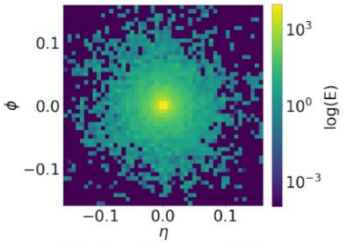
Image Pixels


Adjacency Matrix

https://distill.pub/2021/gnn-intro/



Graph



Photon shower on EM calorimeter (Geant simulation)

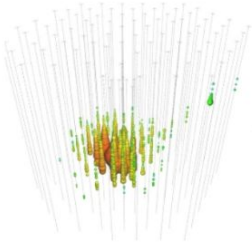

ESA/Hubble & NASA, A. Riess, D. Thilker, D. De Martin (ESA/Hubble).

Representation bias

Grid-organized data

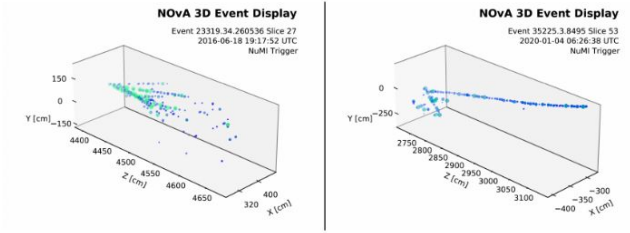
In many physics applications collected data are organized in a *regular grid* in (Euclidean) space.
These data can be described using *graphs*.

3D-lattice data:
In experimental particle physics detectors often cover an extended volume in space to capture a particle trajectory or decay. The detector geometry is constant giving rise to 3D grid.



A reconstructed detection in the **IceCube** sensor array of a suspected **neutrino** with an energy of 252.7 trillion electron-volts. The strings of detectors descend from the surface of the ice to more than two kilometers below. Credit: Jakob van Santen doi.org/10.1038/nature.2013.13026

Depictions of particle traces at the far detector of the NOva experiment. https://www.physics.wisc.edu/2025/02/26/nova-study-sets-tighter-limit-on-sterile-neutrinos/



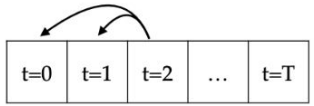
Representation bias

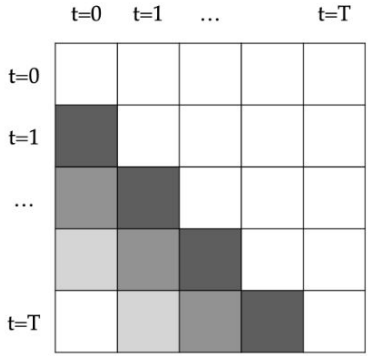
Grid-organized data

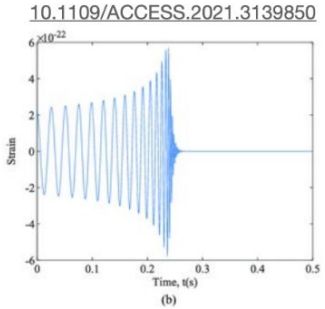
In many physics applications collected data are organized in a *regular grid* in (Euclidean) space.
These data can be described using graphs.

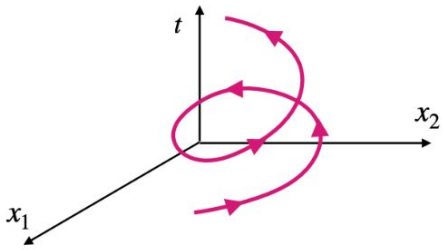
Time series:
Constant rate sampling of sensory data also result in 1 or higher dimensional grids.
Due to causality, each time stamp depends only the previous steps.
Dependence may decays the distance in times increases.

Example of time series in fundamental physics: gravitational waves









Representation bias

Grid-organized data

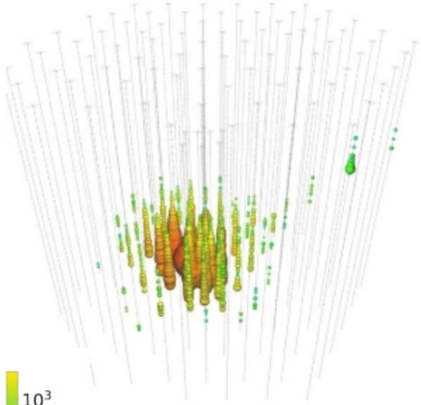
In many applications in **fundamental physics** nodes containing relevant information in regular grids are *sparse*.

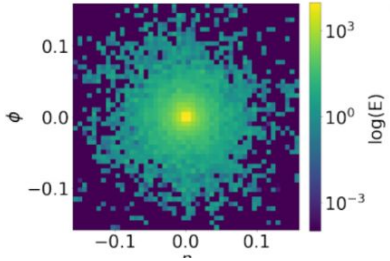
In these cases, regular graphs can be simplified by *dropping irrelevant nodes*, leading to a more *efficient* representation.

https://www.flickr.com/photos/nasahubble/54533174375/



doi.org/10.1038/nature.2013.13026





Representation bias

Spacial-organized data

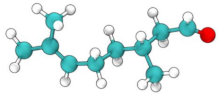
In many applications in **fundamental** physics nodes containing relevant information in regular grids are *sparse*.

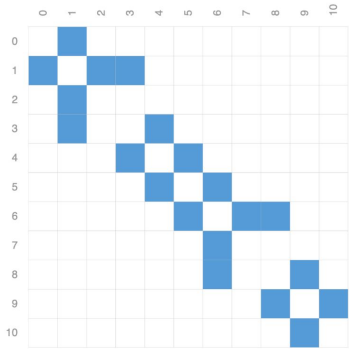
In these cases, regular graphs can be simplified by *dropping irrelevant nodes*, leading to a more *efficient* representation.

Molecules:

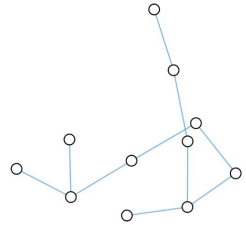
Covalent bonds between atoms in molecules define a stable 3*D geometry* for the molecule in Euclidean space.

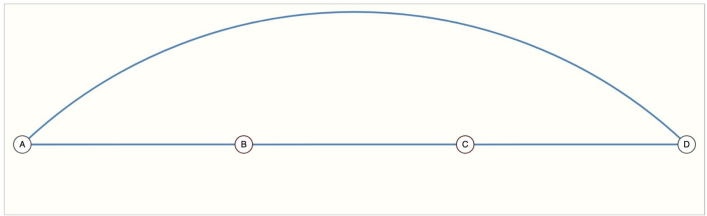
Different pairs of atoms and bonds result in different distances between atoms.

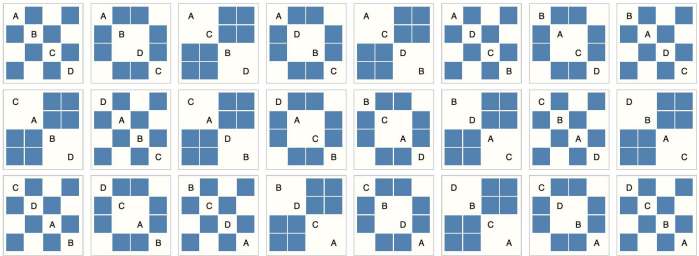




Adjacency matrix







Graphs are **permutation invariant**, no difference on how we number nodes

Data
Input representation
Model architecture
Loss

Representation bias

Feature engineering

Particle Jets:
In high-energy collisions (e.g. at the LHC), energetic quarks or gluons produced in a hard scattering process cannot exist freely due to **color confinement**. Instead, they undergo a process known as a **parton shower**, followed by **hadronization**, resulting in a collimated spray of particles called a **jet**.
The resulting hadrons are detected in calorimeters or tracking systems.

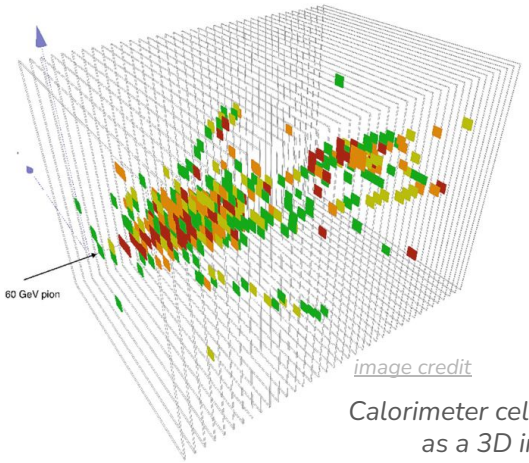
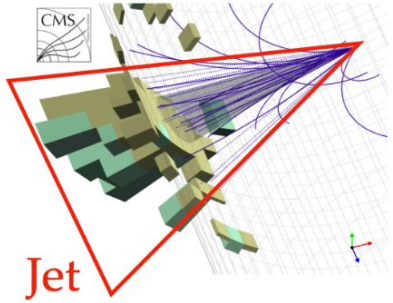


image credit

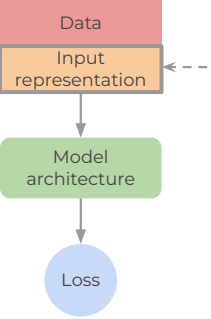
Calorimeter cells response as a 3D image

From *sensor-level* representation we reconstruct the *object-level* information

Collider data



- Different representations can be optimal for different tasks
- *Feature engineering* exploiting domain knowledge can help to simplify some tasks and reach better solution with less data



Representation bias

Conservation laws and symmetries

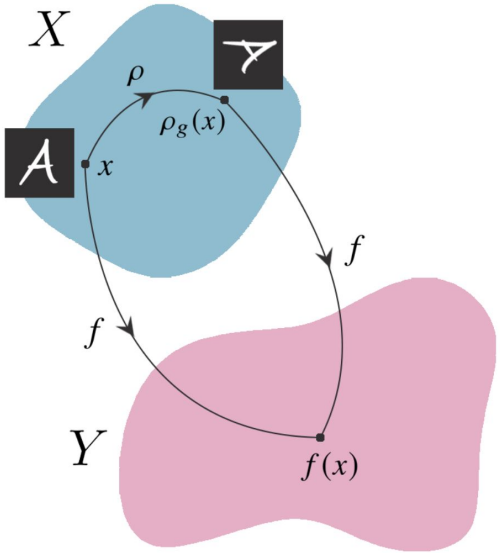
- Physical systems obey fundamental *conservation laws* (e.g., energy, momentum, charge).
- According to **Noether's theorem**, every conservation law corresponds to a continuous *symmetry* of the system (e.g., time invariance → energy conservation).
- Symmetries define *constraints* on how physical quantities can transform under operations like translation, rotation, or reflection.
- Recognizing the relevant symmetry groups can help reduce the search space for learning.
- Enforcing expected physical invariance or equivariance can help increase **generalization** and **reliability** of the ML solution.

Representation bias

Conservation laws and symmetries

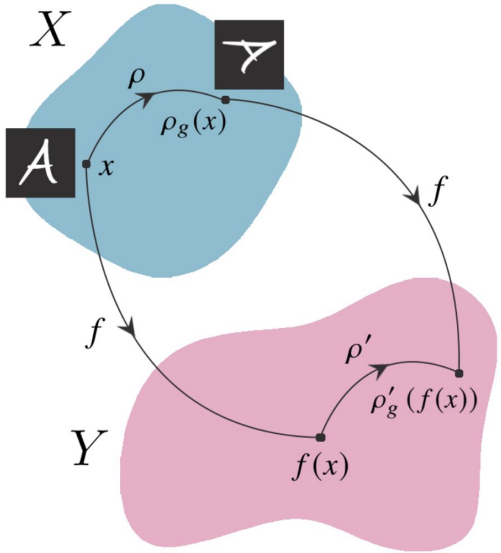
Invariance

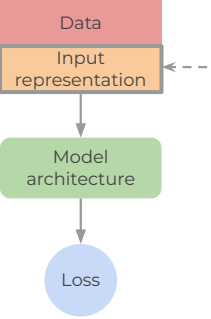
$$f(\rho_g(x)) = f(x)$$



Equivariance

$$f(\rho_g(x)) = \rho'_g(f(x))$$





Representation bias

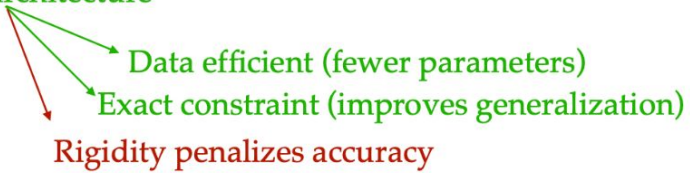
Conservation laws and symmetries

If your data and task are known to be symmetric under a group G, then:

- Your model should respect those symmetries
- The output should be **invariant** or **equivariant** under group actions

There are multiple ways to embed symmetries in a model:

- Learn them from data
- Suggest them with penalties
- Encode them in the architecture



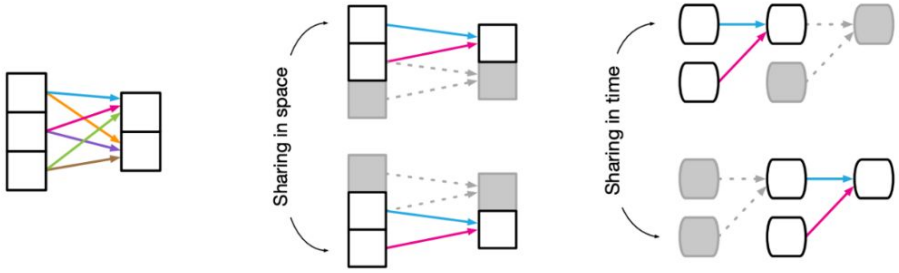
Architecture design

Data structures and learning objectives motivates architecture design

Architectures should be designed in such a way to guarantee that the relevant information flows from the input all the way to the output

- Invariant/equivariant
- Preserve the data structure/topology
- Prefer models that respect priors on the expected solution

Connectivity is a critical aspect of model design



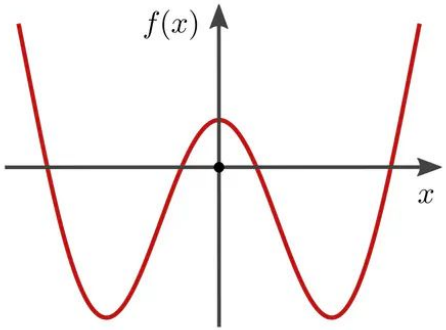
(a) Fully connected (b) Convolutional (c) Recurrent

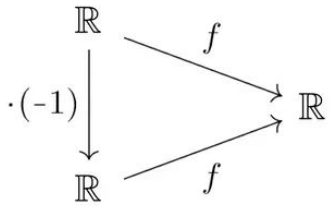
Not very informative! → Encode assumptions on relational structure in the data

Architecture design

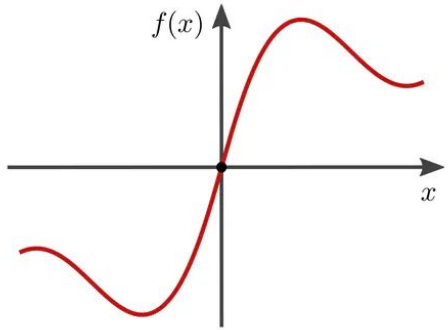
Equivariant model

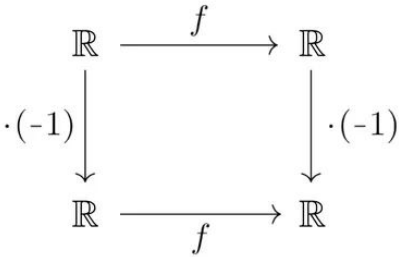
symmetric: $f(-x) = f(x)$





antisymmetric: $f(-x) = -f(x)$

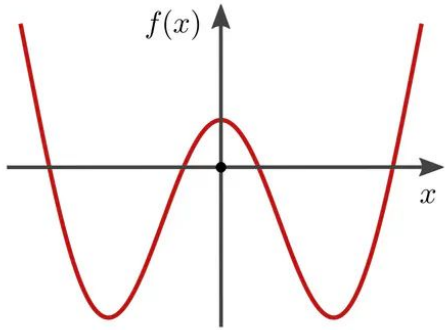




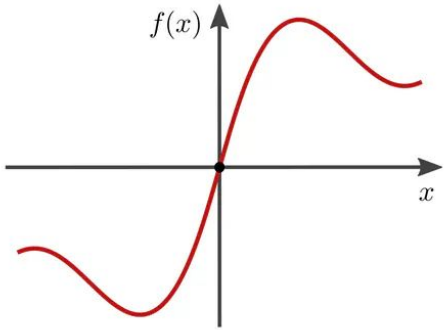
Architecture design

Equivariant model

symmetric: $f(-x) = f(x)$



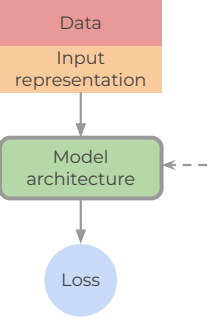
antisymmetric: $f(-x) = -f(x)$



$$y(x) = \sum_{n=0}^{N} w_n x^n \longrightarrow$$

$$y_{\text{symm}}(x) = \sum_{n \text{ even}}^{N} w_n x^n \qquad (\text{since } (-x)^n = x^n \text{ for even } n \in \mathbb{N})$$

$$y_{\text{anti}}(x) = \sum_{n \text{ odd}}^{N} w_n x^n \qquad (\text{since } (-x)^n = -(x^n) \text{ for odd } n \in \mathbb{N})$$



Architecture design

Equivariant model

$$\mathcal{F}_0 \xrightarrow{L_1} \mathcal{F}_1 \xrightarrow{L_2} \mathcal{F}_2 \xrightarrow{L_3} \cdots \xrightarrow{L_{N-1}} \mathcal{F}_{N-1} \xrightarrow{L_N} \mathcal{F}_N$$

Architecture design

Equivariant model

$$\mathcal{F}_0 \xrightarrow{L_1} \mathcal{F}_1 \xrightarrow{L_2} \mathcal{F}_2 \xrightarrow{L_3} \cdots \xrightarrow{L_{N-1}} \mathcal{F}_{N-1} \xrightarrow{L_N} \mathcal{F}_N$$

$$\begin{array}{ccccccccccc} \mathcal{F}_0 & \xrightarrow{L_1} & \mathcal{F}_1 & \xrightarrow{L_2} & \mathcal{F}_2 & \xrightarrow{L_3} & \cdots & \xrightarrow{L_{N-1}} & \mathcal{F}_{N-1} & \xrightarrow{L_N} & \mathcal{F}_N \\ \downarrow{\scriptstyle g \triangleright_0} & & & & & & & & & & \downarrow{\scriptstyle g \triangleright_N} \\ \mathcal{F}_0 & \xrightarrow[L_1]{} & \mathcal{F}_1 & \xrightarrow[L_2]{} & \mathcal{F}_2 & \xrightarrow[L_3]{} & \cdots & \xrightarrow[L_{N-1}]{} & \mathcal{F}_{N-1} & \xrightarrow[L_N]{} & \mathcal{F}_N \end{array}$$

Architecture design

Equivariant model

$$\mathcal{F}_0 \xrightarrow{L_1} \mathcal{F}_1 \xrightarrow{L_2} \mathcal{F}_2 \xrightarrow{L_3} \cdots \xrightarrow{L_{N-1}} \mathcal{F}_{N-1} \xrightarrow{L_N} \mathcal{F}_N$$

$$\begin{array}{ccccccccccc}
\mathcal{F}_0 & \xrightarrow{L_1} & \mathcal{F}_1 & \xrightarrow{L_2} & \mathcal{F}_2 & \xrightarrow{L_3} & \cdots & \xrightarrow{L_{N-1}} & \mathcal{F}_{N-1} & \xrightarrow{L_N} & \mathcal{F}_N \\
\downarrow{\scriptstyle g \triangleright_0} & & \downarrow{\scriptstyle g \triangleright_1} & & \downarrow{\scriptstyle g \triangleright_2} & & & & \downarrow{\scriptstyle g \triangleright_{N-1}} & & \downarrow{\scriptstyle g \triangleright_N} \\
\mathcal{F}_0 & \xrightarrow[L_1]{} & \mathcal{F}_1 & \xrightarrow[L_2]{} & \mathcal{F}_2 & \xrightarrow[L_3]{} & \cdots & \xrightarrow[L_{N-1}]{} & \mathcal{F}_{N-1} & \xrightarrow[L_N]{} & \mathcal{F}_N
\end{array}$$

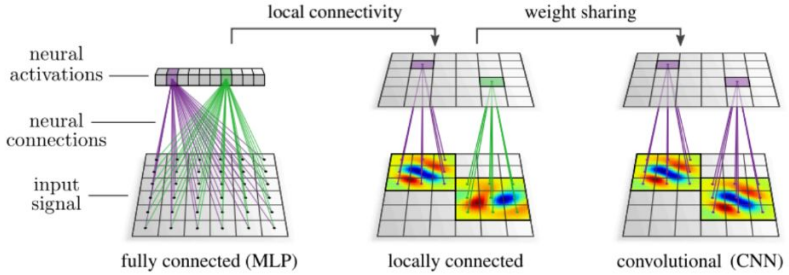
Architecture design

Convolutional NNs

Strong Inductive Bias: *spacial locality* and *space translation equivariance*

CNNs embed **prior knowledge** about image-like structures:

- *Spacial locality*: nearby pixels are more related than distant ones
  → Convolutional layers connect each neuron to a **local receptive field** of neighboring pixels.
- *Translation equivariance*: patterns recur across space
  → The same filter is applied across all spatial locations (weight sharing), encouraging learning of **position-independent features**

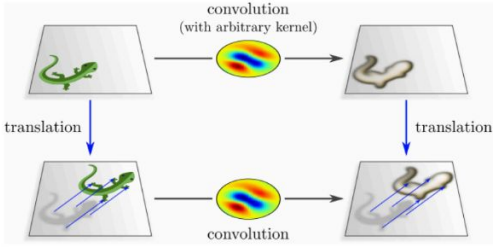


Basic CNN update

The core operation is a *convolution*:

$$y_{i,j}^{(k)} = \sigma\left(\sum_{c=1}^{C}\sum_{u,v} W_{u,v}^{(k,c)}\, x_{i+u,j+v}^{(c)} + b^{(k)}\right)$$

$x_{i,j}^{(c)}$: input value at position (i, j) in channel c
$W^{(k,c)}$: convolutional kernel weights for output channel k, input channel c
σ: non-linear activation function (e.g., ReLU)
$y_{i,j}^{(k)}$: output feature at position (i, j) in channel k



M. Weiler "Equivariant And Coordinate Independent CNNs"

Visualisations here and here

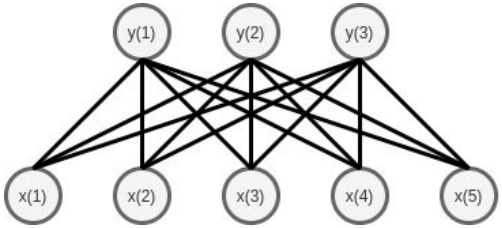
Architecture design

Convolutional NNs

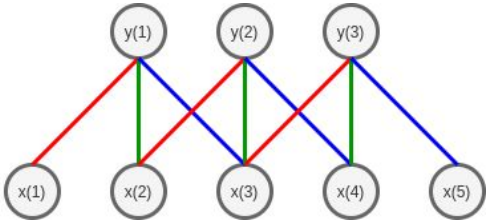
Parameter Efficiency
- **Weight sharing** reduces the number of parameters (CNNs are more efficient than DNNs for images).

Pooling and Invariance
- Pooling layers (e.g., max or average pooling) downsample the feature maps.
- Provide **translation invariance** to small shifts and reduce spatial resolution.
- Helps in **summarizing features** and reducing computational cost.

Hierarchical Feature Learning
- Initial layers learn simple patterns (edges, gradients).
- Deeper layers capture complex structures (shapes, object parts).
- Mimics the **hierarchical processing** in the human visual system.
- Enables **scalable architectures** like ResNet or EfficientNet.



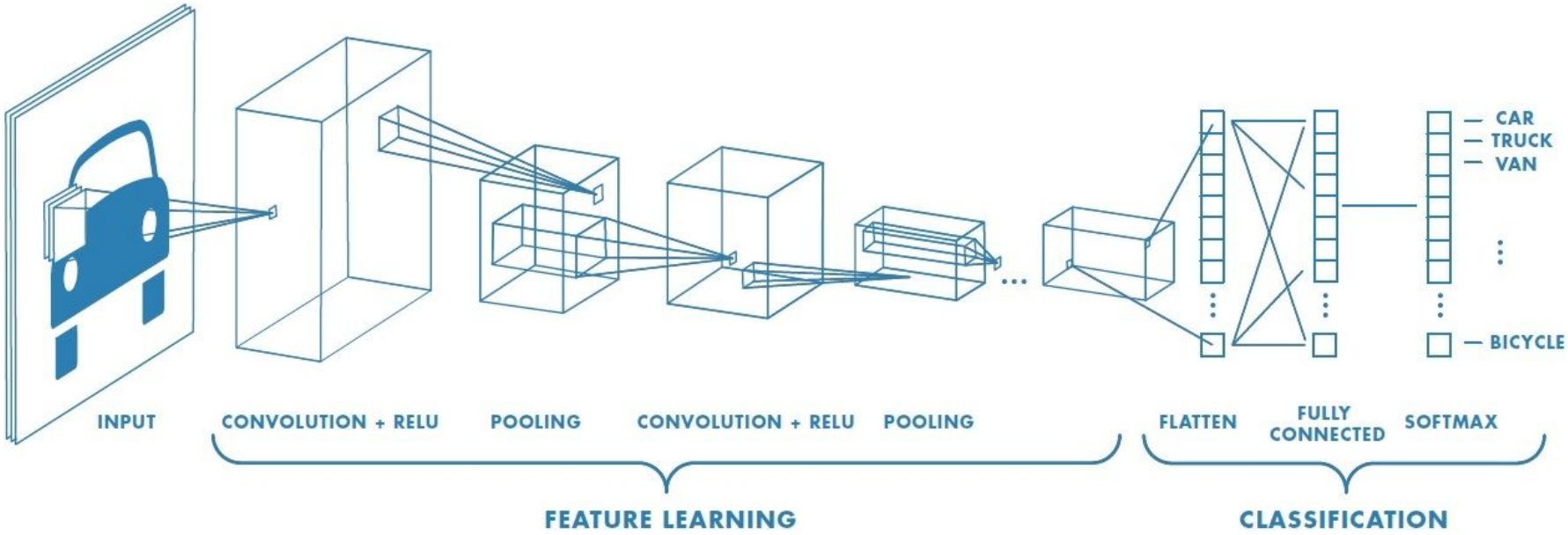
MLP



CNN: same weights - same color, much less to learn

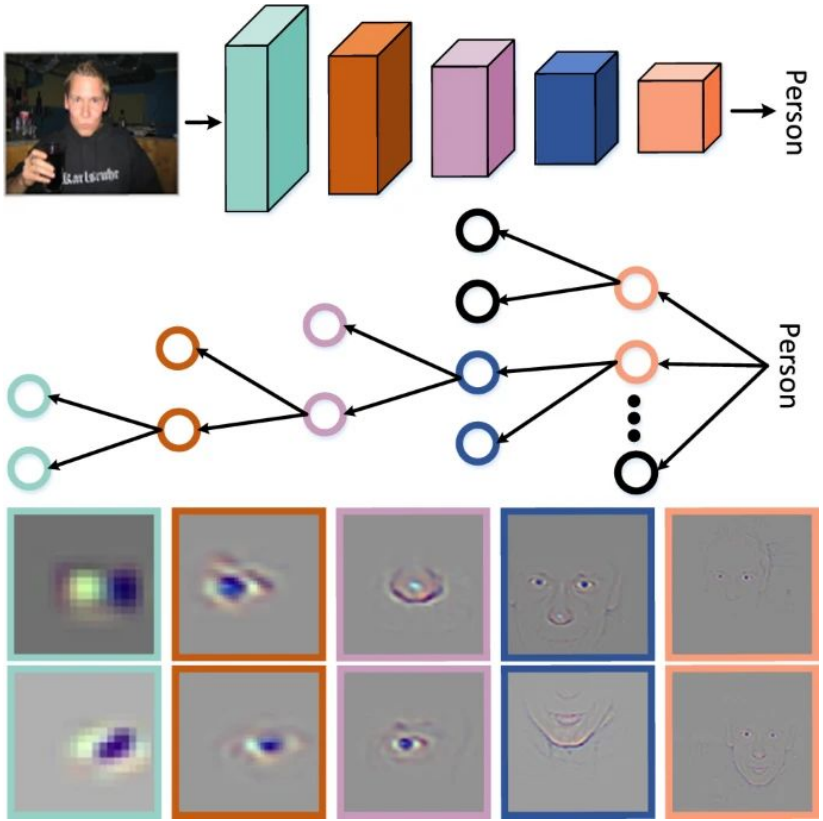
Architecture design

Convolutional NNs



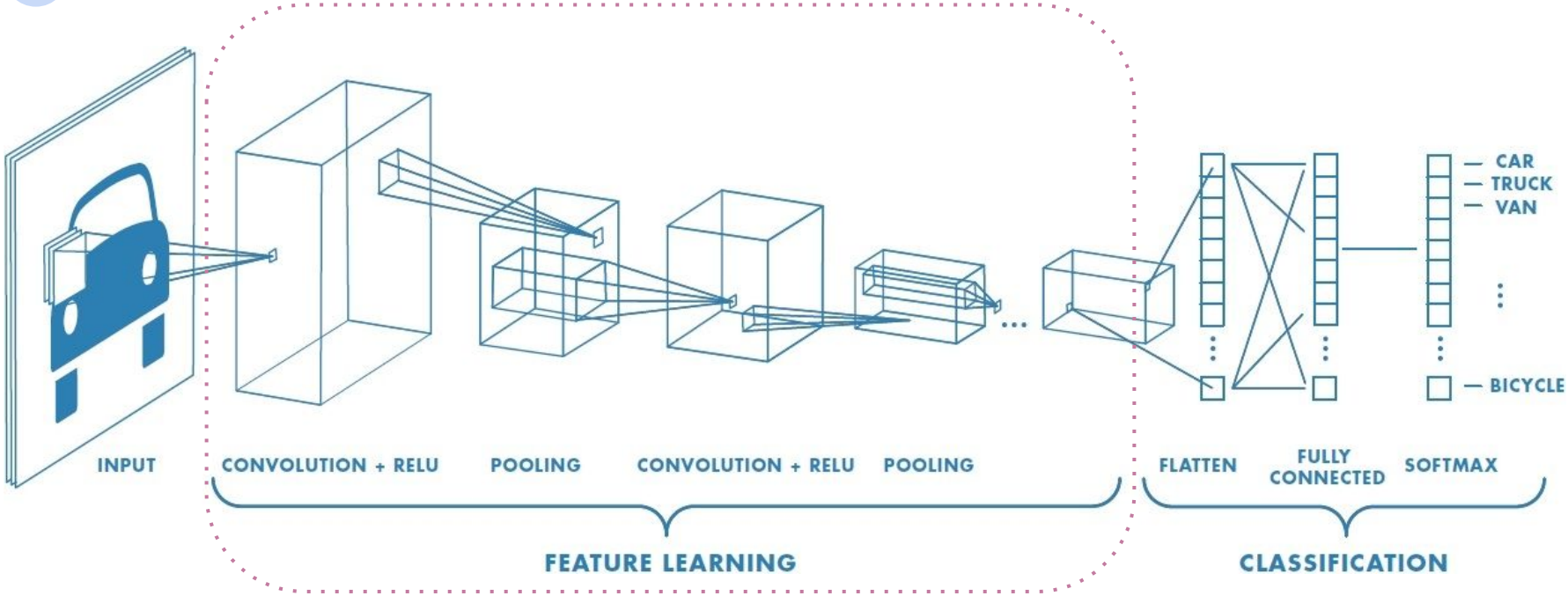
Architecture design

Convolutional NNs



Architecture design

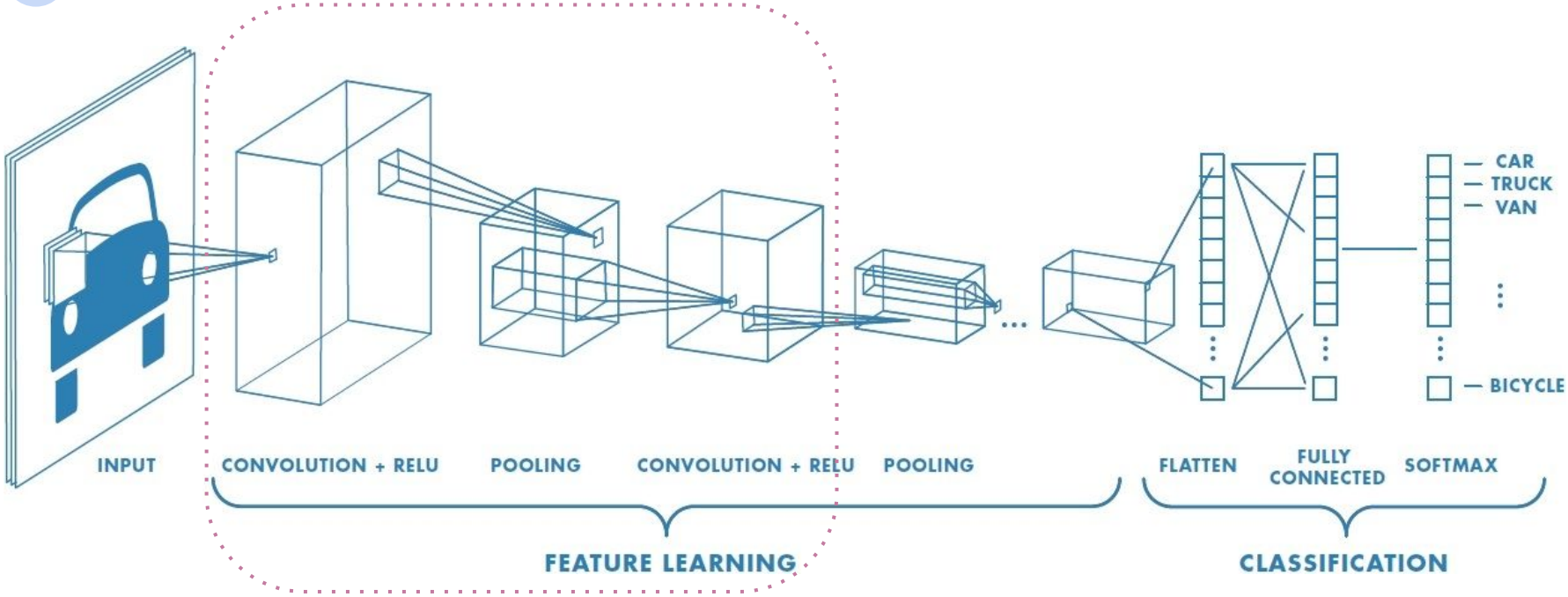
Convolutional NNs



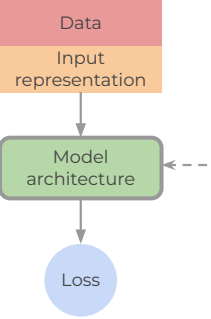
Reuse already trained feature extractor part

Architecture design

Convolutional NNs



Reuse already trained feature extractor part



Architecture design

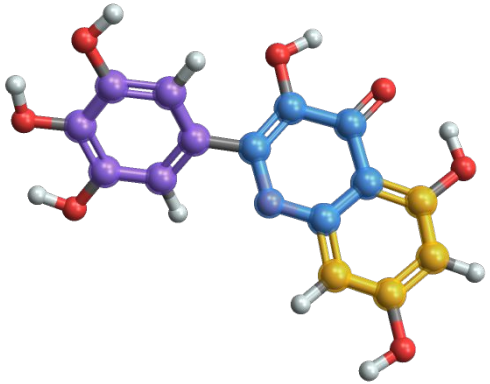
Graph NNs

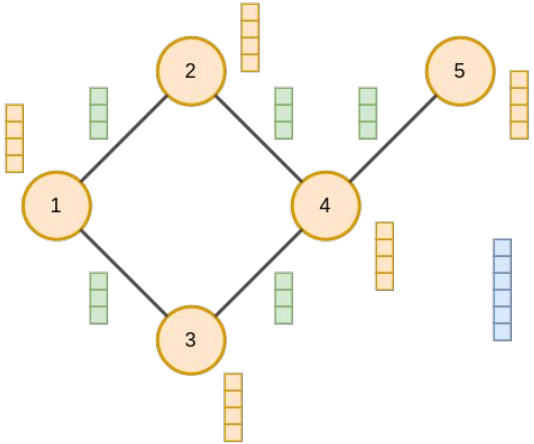
Strong inductive bias: *arbitrary relations* and *permutation invariance*

GNNs embed **prior knowledge** about any arbitrary relational data structure:

- *Arbitrary relations*: the connectivity map is hard coded in the graph.
  → GNNs determine how representation interact and are isolated, rather than those choices being determined by the fixed architecture
- *Permutation invariance*: properties of the system don't depend on the elements order in the representation
  → attributes aggregation methods are permutation invariant

How to describe a graph?





$\mathbf{h}_i$ *Nodes description*

$\mathbf{e}_j$ *Edges description*

$\mathbf{g}$ *Graph description*

How to describe a graph?

Nodes

```
[0 , 1 , 1 , 0 , 0 , 1 , 1 , 1 ]
```

Edges

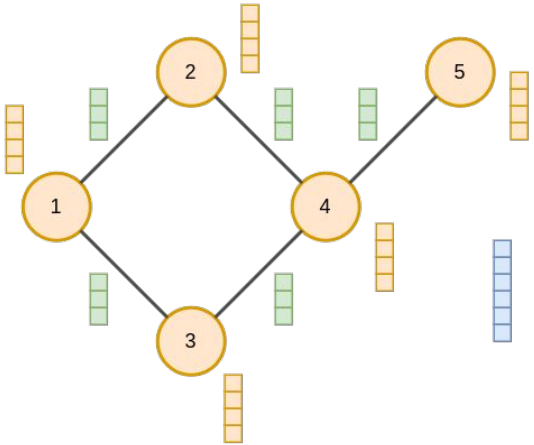
```
[2 , 1 , 1 , 1 , 2 , 1 , 1 ]
```

Adjacency List

```
[[1, 0] , [2, 0] , [4, 3] , [6, 2] ,
 [7, 3] , [7, 4] , [7, 5] ]
```

Global

```
0
```



$\mathbf{h}_i$ *Nodes description*

$\mathbf{e}_j$ *Edges description*

$\mathbf{g}$ *Graph description*

Architecture design

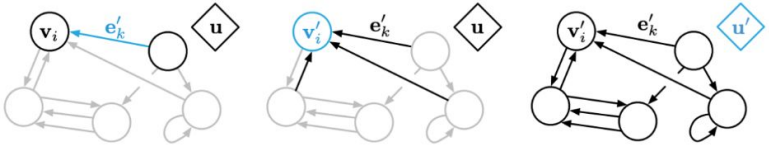
Graph NNs

Strong inductive bias: *arbitrary relations* and *permutation invariance*
GNNs embed **prior knowledge** about any arbitrary relational data structure:

- *Arbitrary relations*: the connectivity map is hard coded in the graph.
  → GNNs determine how representation interact and are isolated, rather than those choices being determined by the fixed architecture
- *Permutation invariance*: properties of the system don't depend on the elements order in the representation
  → attributes aggregation methods are permutation invariant

Basic GNN update:

Graph Neural Networks operate on graph-structured data by iteratively updating edges, nodes and global attributes.



(a) Edge update (b) Node update (c) Global update

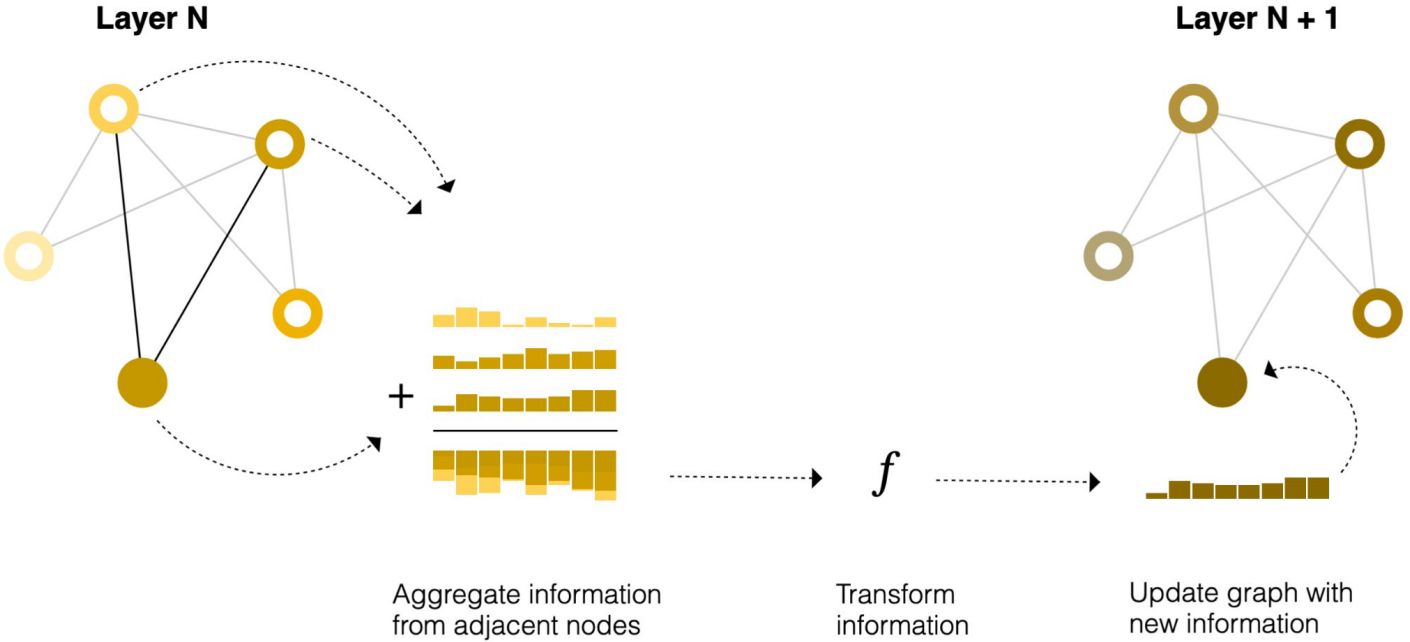
Algorithm 1 Steps of computation in a full GN block.

function GRAPHNETWORK(E, V, $\mathbf{u}$)
  for $k \in \{1 \dots N^e\}$ **do**
    $\mathbf{e}'_k \leftarrow \phi^e(\mathbf{e}_k, \mathbf{v}_{r_k}, \mathbf{v}_{s_k}, \mathbf{u})$ ▷ 1. Compute updated edge attributes
  end for
  for $i \in \{1 \dots N^n\}$ **do**
    let $E'_i = \{(\mathbf{e}'_k, r_k, s_k)\}_{r_k=i,\, k=1:N^e}$
    $\bar{\mathbf{e}}'_i \leftarrow \rho^{e\to v}(E'_i)$ ▷ 2. Aggregate edge attributes per node
    $\mathbf{v}'_i \leftarrow \phi^v(\bar{\mathbf{e}}'_i, \mathbf{v}_i, \mathbf{u})$ ▷ 3. Compute updated node attributes
  end for
  let $V' = \{\mathbf{v}'\}_{i=1:N^v}$
  let $E' = \{(\mathbf{e}'_k, r_k, s_k)\}_{k=1:N^e}$
  $\bar{\mathbf{e}}' \leftarrow \rho^{e\to u}(E')$ ▷ 4. Aggregate edge attributes globally
  $\bar{\mathbf{v}}' \leftarrow \rho^{v\to u}(V')$ ▷ 5. Aggregate node attributes globally
  $\mathbf{u}' \leftarrow \phi^u(\bar{\mathbf{e}}', \bar{\mathbf{v}}', \mathbf{u})$ ▷ 6. Compute updated global attribute
  return $(E', V', \mathbf{u}')$
end function

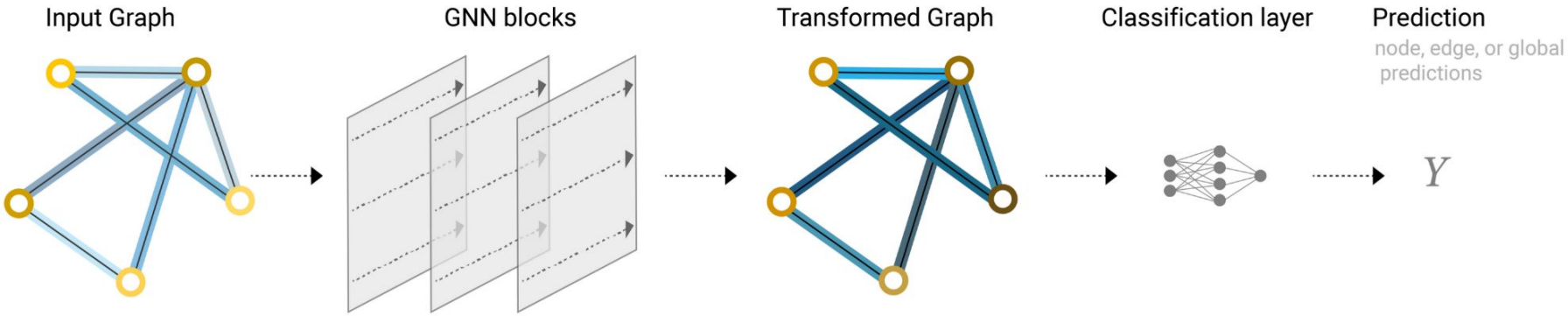
Relational inductive biases, deep learning, and graph networks

Two main operations: message passing

- For each node in the graph,
  gather all the neighboring node embeddings (or messages)
- Aggregate all messages via an aggregate function (like sum).
- All pooled messages are passed through an update function,
  usually a learned neural network.



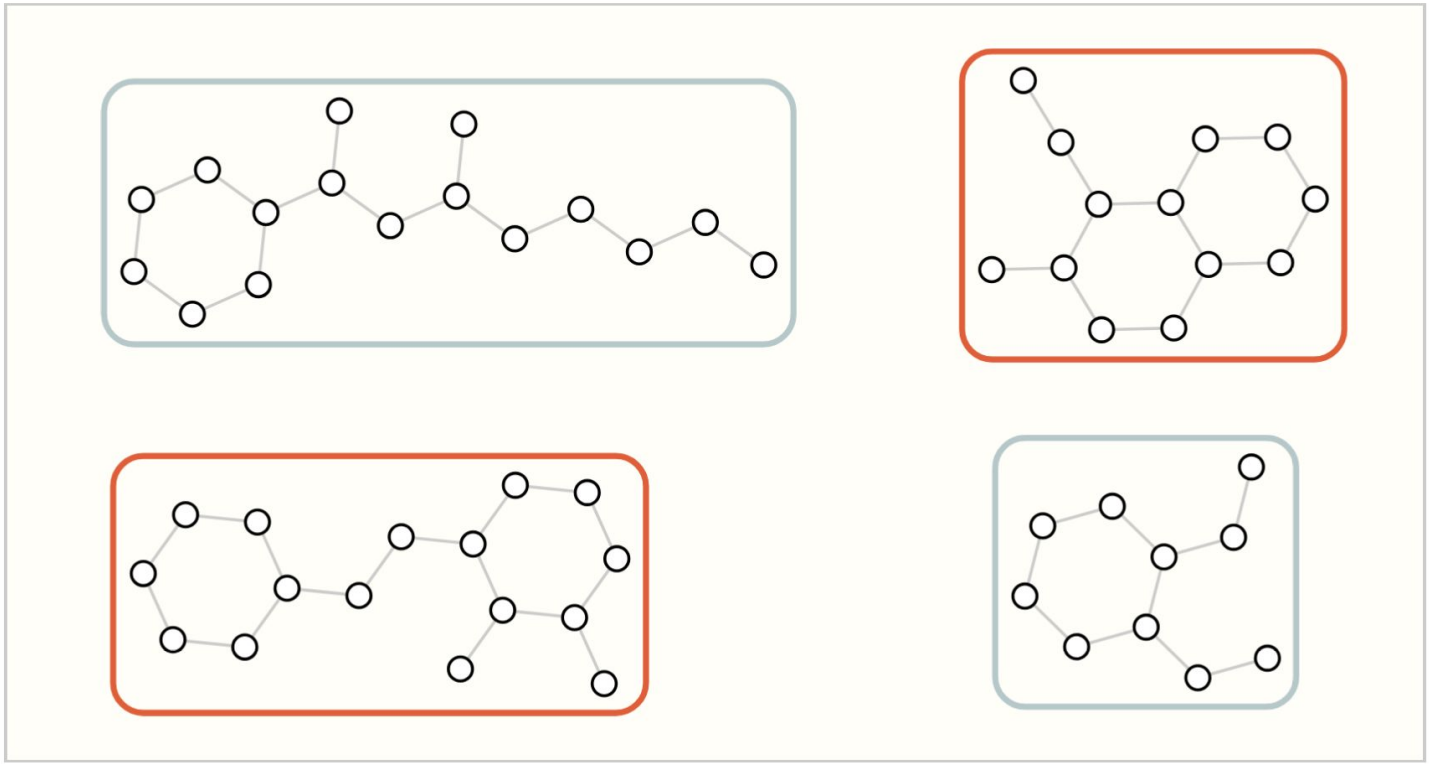
Pipeline



An end-to-end prediction task with a GNN model.

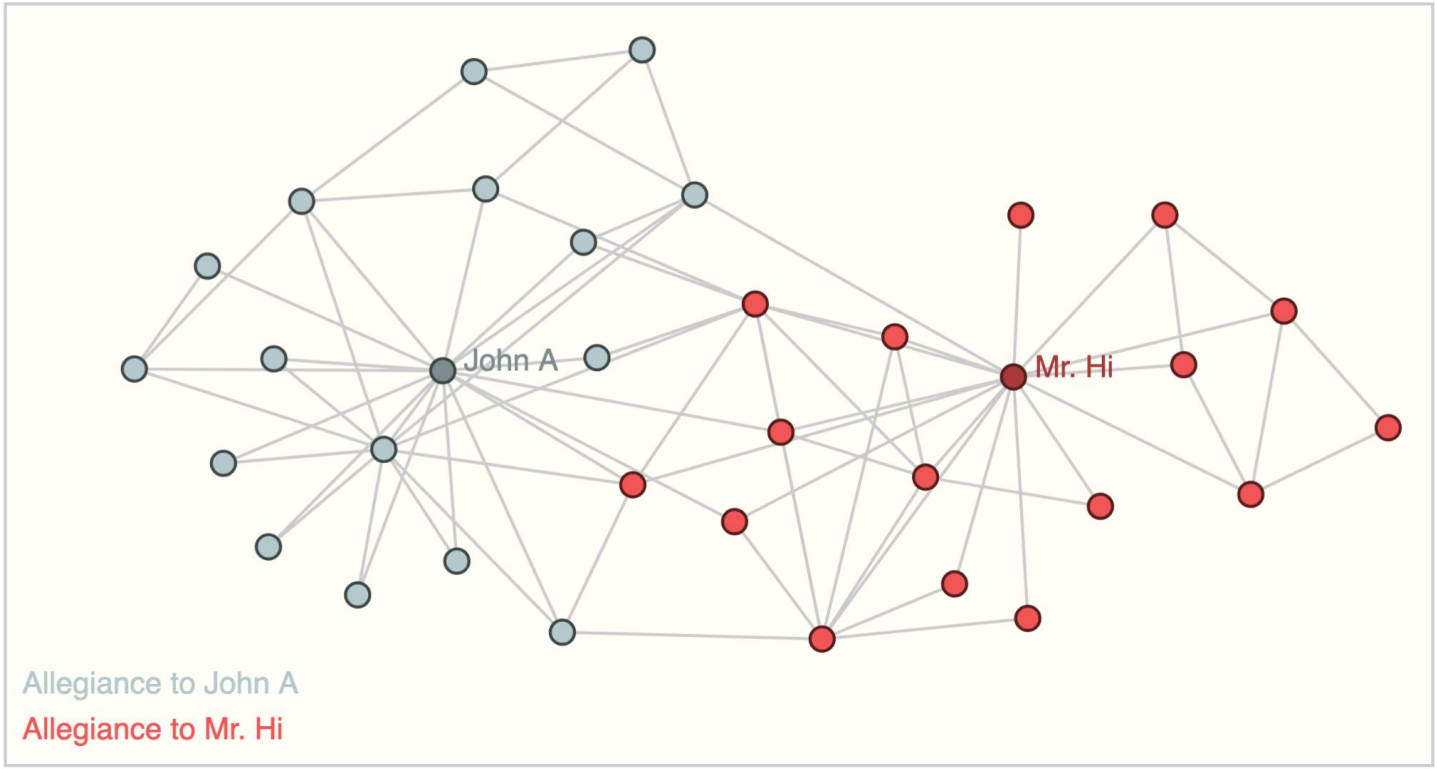
What can we do with graphs? Graph level

image credit for GNN



Output: labels for each graph, (e.g., "does the graph contain two rings?")

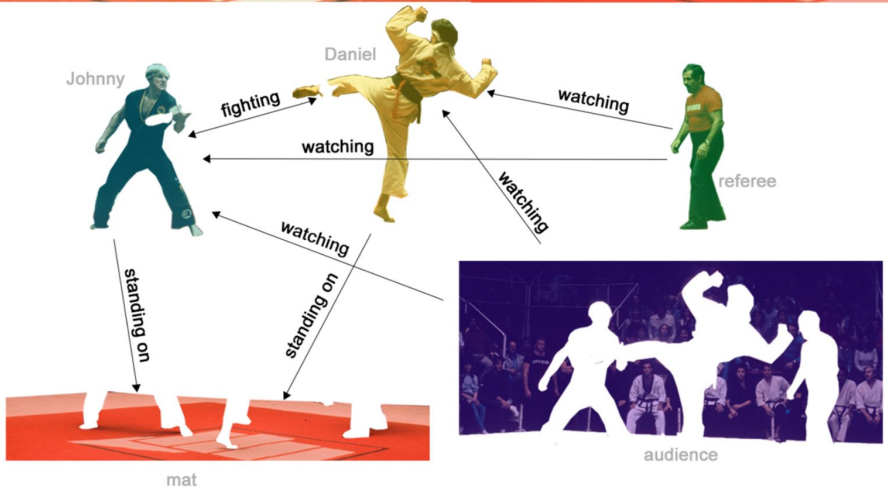
What can we do with graphs? Node level

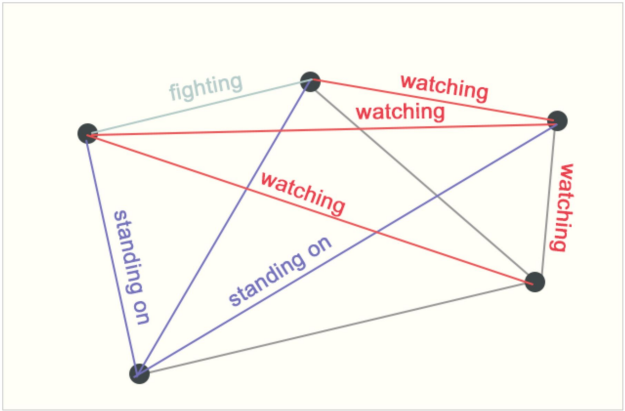


Output: graph node labels

What can we do with graphs? Edge level



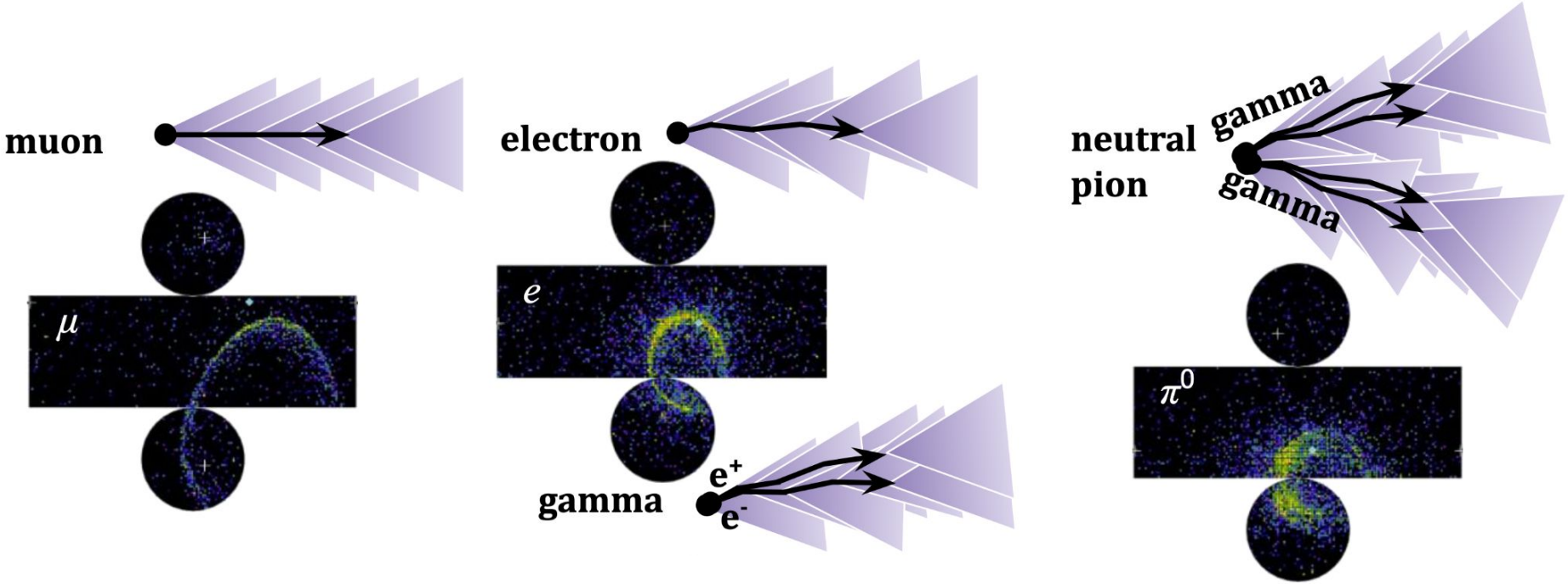




Output: labels for edges

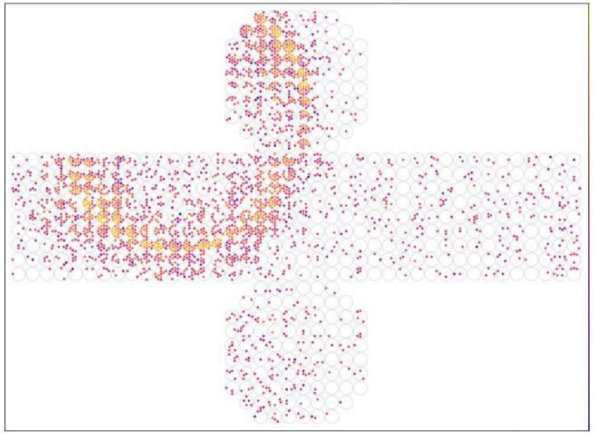
Example: GNN for particle reconstruction

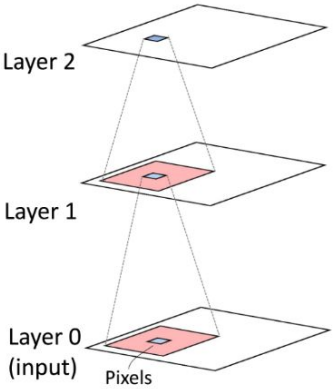
example from HK



Example: GNN for particle reconstruction

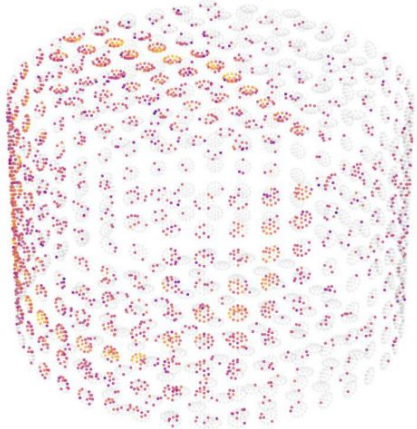
Networks on 2D image-like data

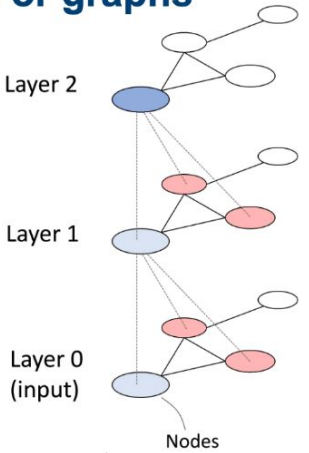




- Fast and well understood CNN-based networks
- Leverage extensive progress in computer vision

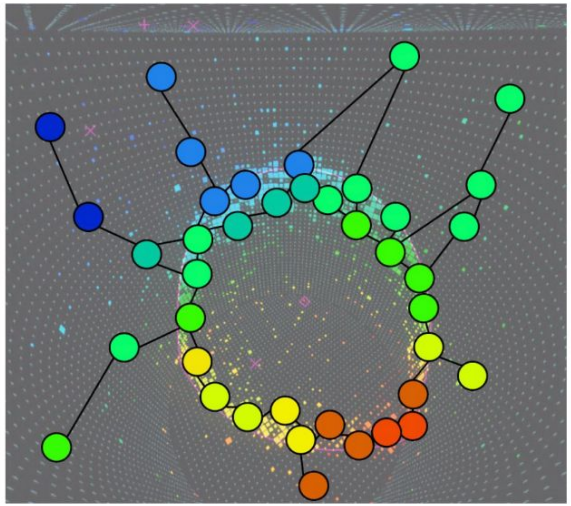
Networks on 3D point-clouds or graphs

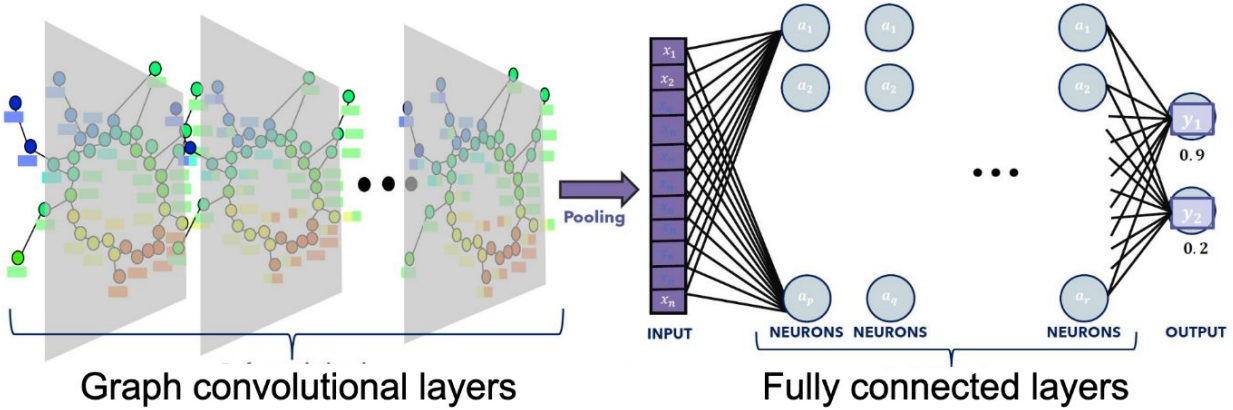


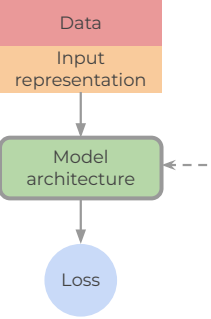


- Retain physical 3D detector geometry
- More complex and challenging to explore

Example: GNN for particle reconstruction







Architecture design

Transformers

Relatively bias-free:

- No assumptions of locality (unlike CNNs)
- No sequential order assumption (unlike RNNs)
- ***Permutation equivariance*** in self-attention allows flexible reasoning over arbitrary sets or graphs.

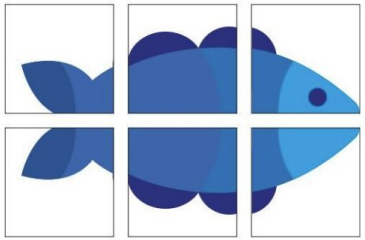
- Input: list of elements with d features: $x = (x_1, \ldots, x_n) \in \mathbb{R}^{n \times d}$
- ***Positional encoding*** can be reintroduced but is not hard coded.

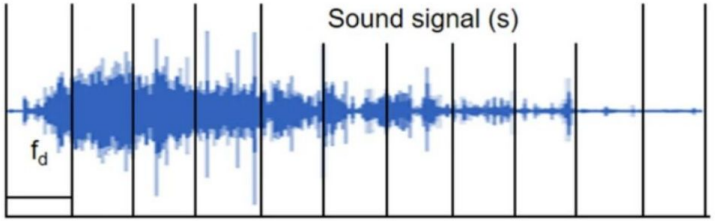
- The self-attention block allows to model arbitrary *pair-wise relations* between any two input elements

Transformers are working with composites (objects made up of elements)

- Each of these parts (represented numerically for computers) is called **token**

Text: The elements of a sentence are words

Images:  Patches

Sound: 

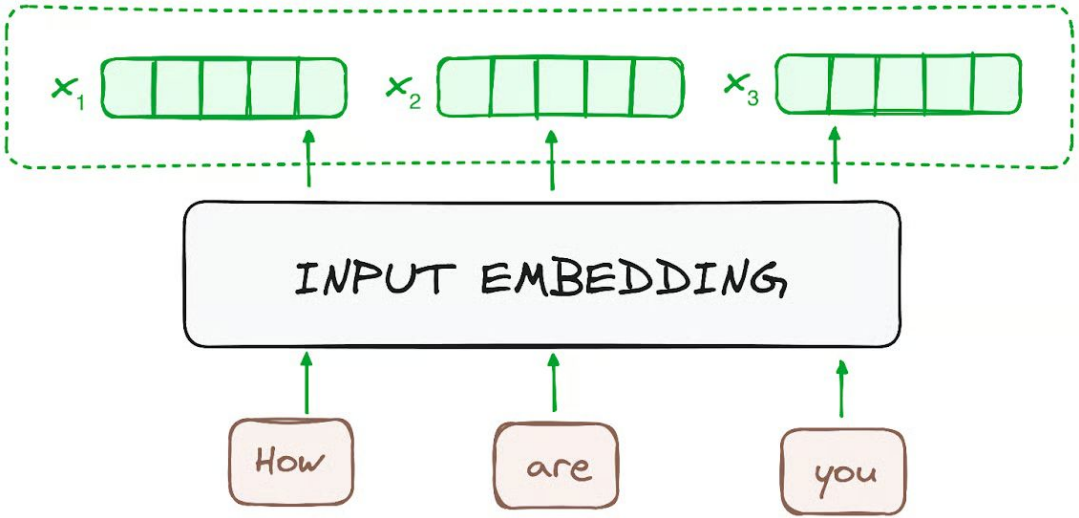
Frames

How to learn the relation between embeddings?

- Each of these parts (represented numerically for computers) is called **token**
- Obtain a representation of the composite. It does not have to be “good” or semantic. An initial representation.

How to learn the relation between embeddings?

- Each of these parts (represented numerically for computers) is called **token**
- Obtain a representation of the composite. It does not have to be “good” or semantic. An initial representation.



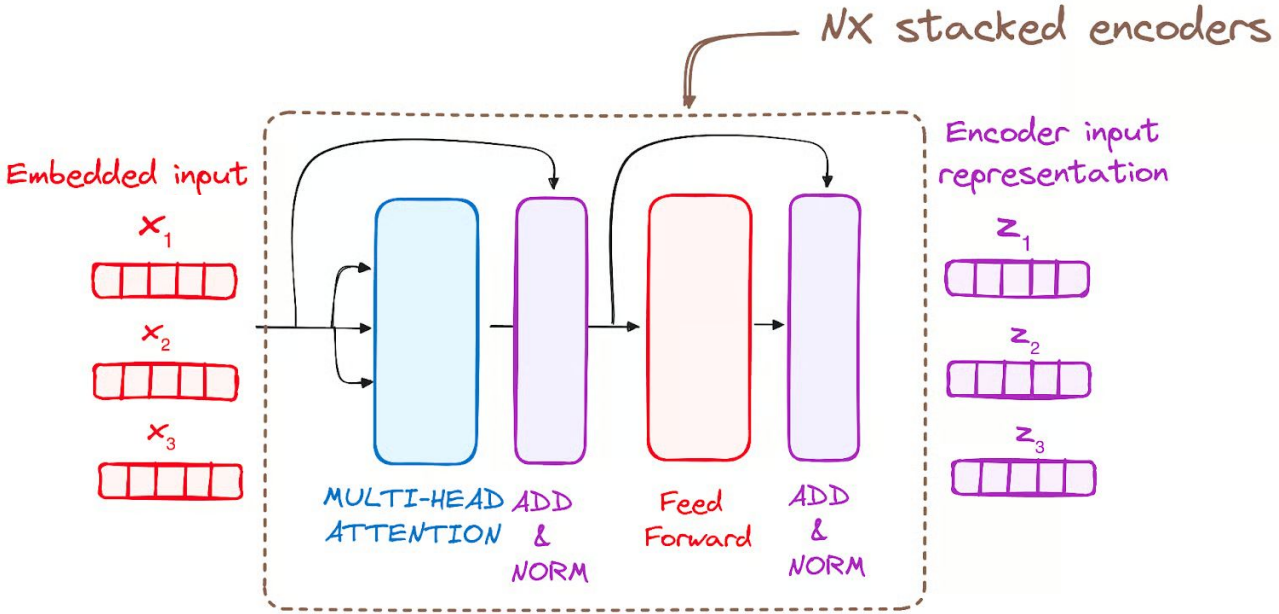
Each token is embedded into a vector of a fixed size.

Can use pretrained layers

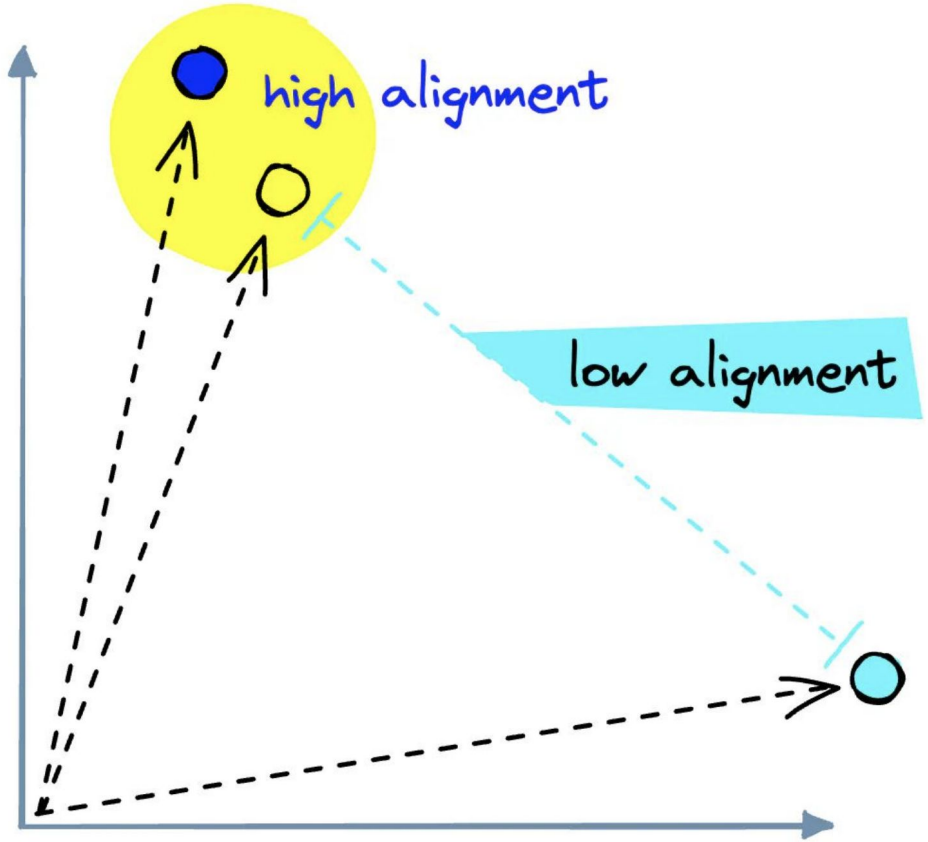
image credit for transformer architecture

How to learn the relation between embeddings?

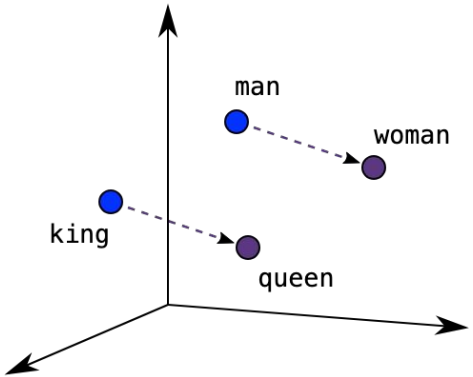
- Each of these parts (represented numerically for computers) is called **token**
- Obtain a representation of the composite. It does not have to be “good” or semantic. An initial representation.
- Feed the representation through several transformer layers to obtain “context-aware” embeddings.



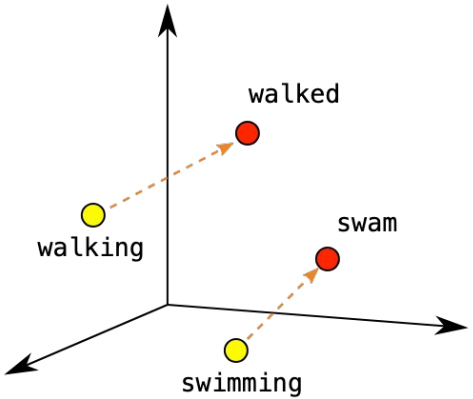
Key idea: learn a “good” representation of each composite = embedding



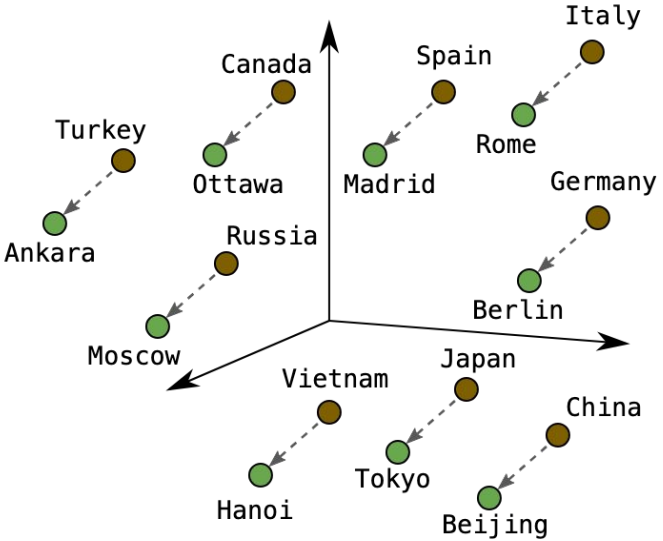
Words embedding



Male-Female

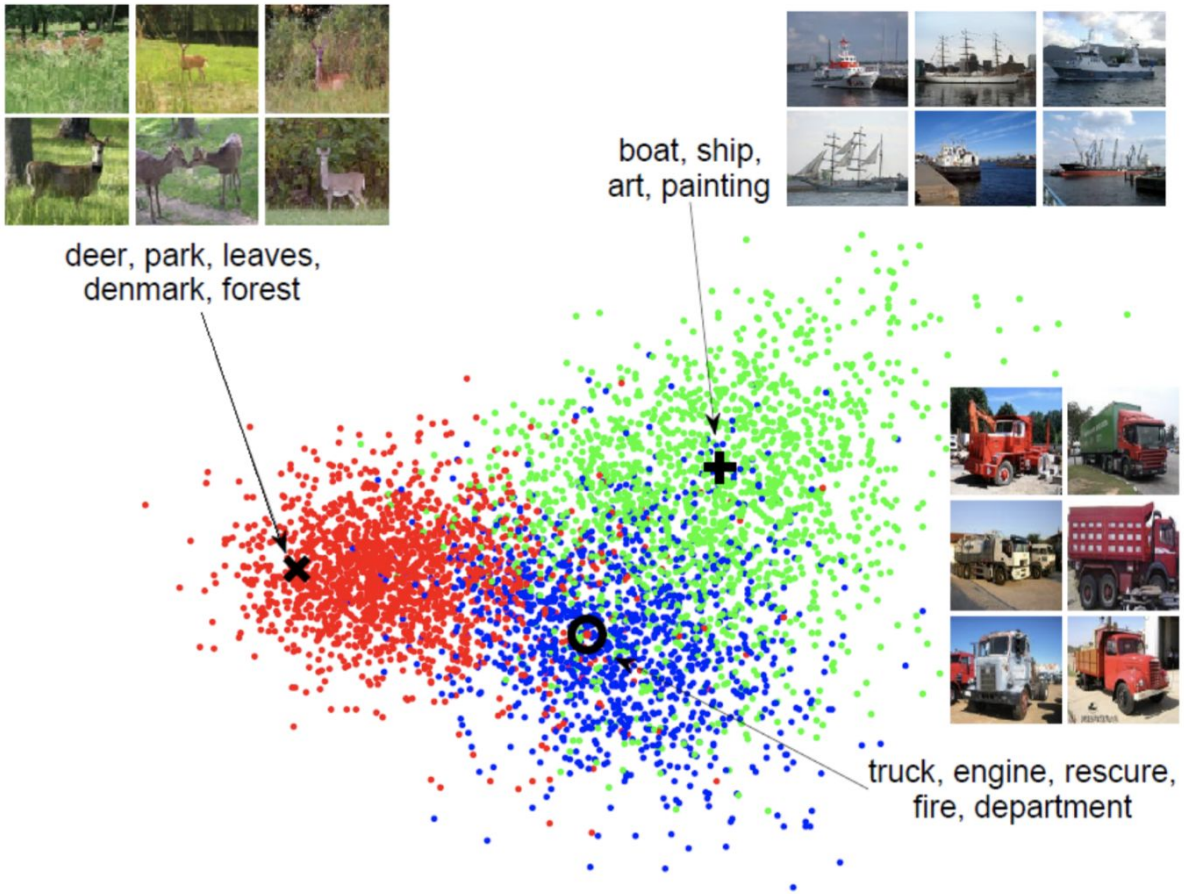


Verb Tense



Country-Capital

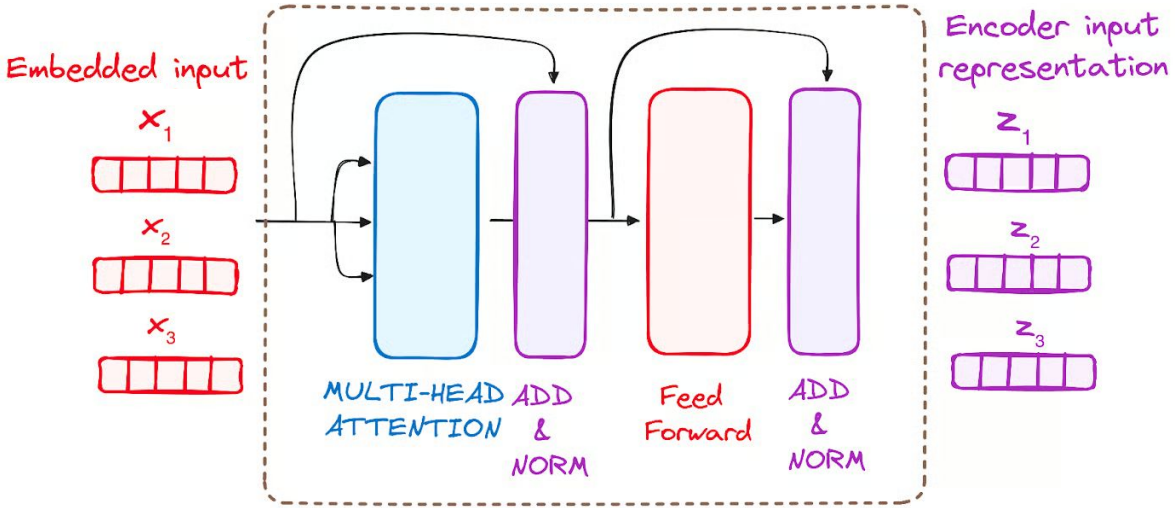
Images embedding



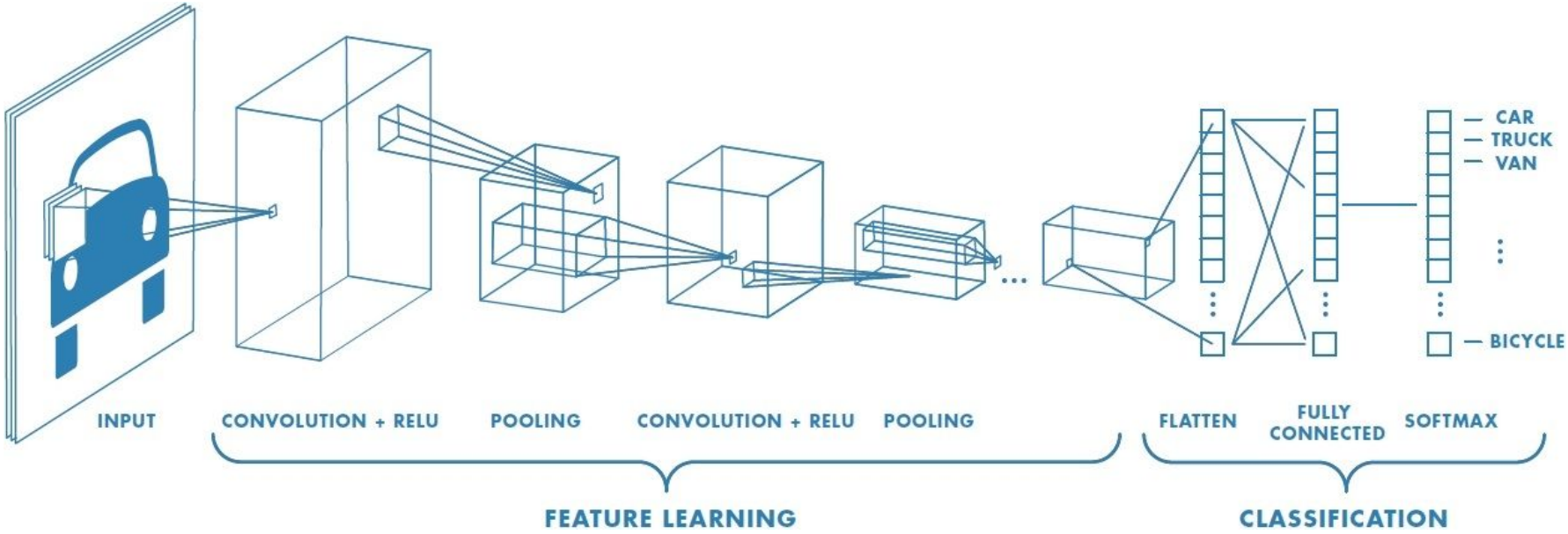
Context-aware embeddings

The humanoid robot did not cross the road, as it was very dangerous.

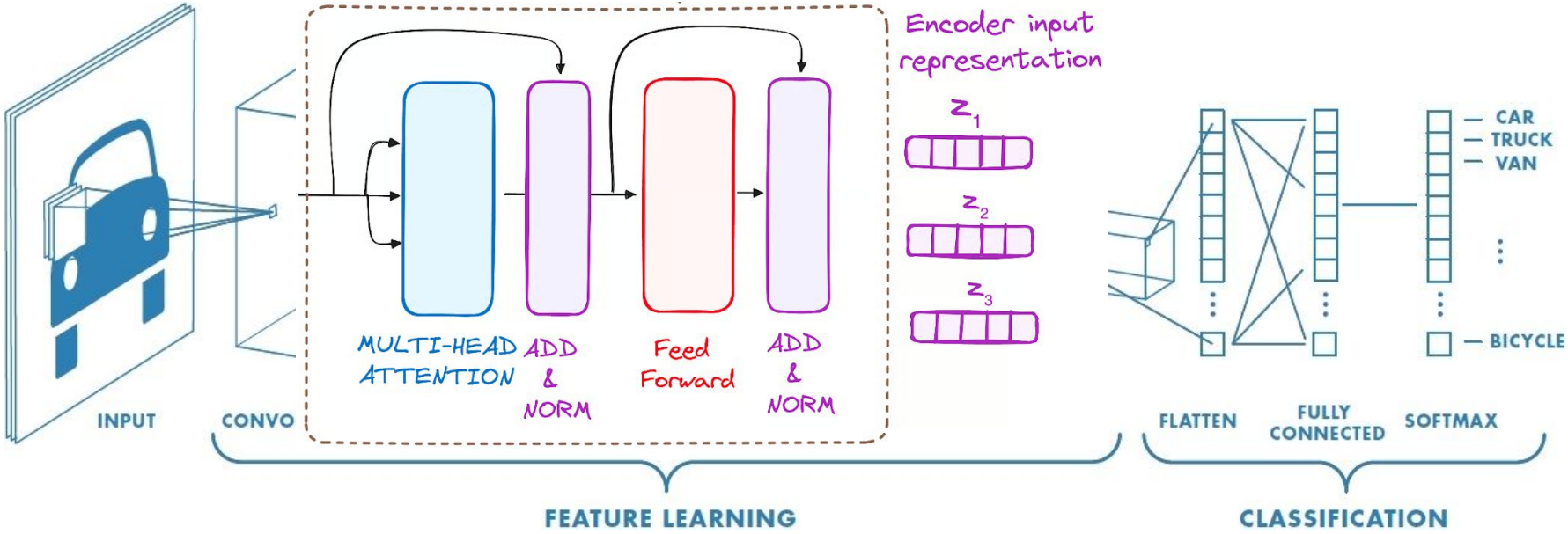
The humanoid robot did not cross the road because it was tired.



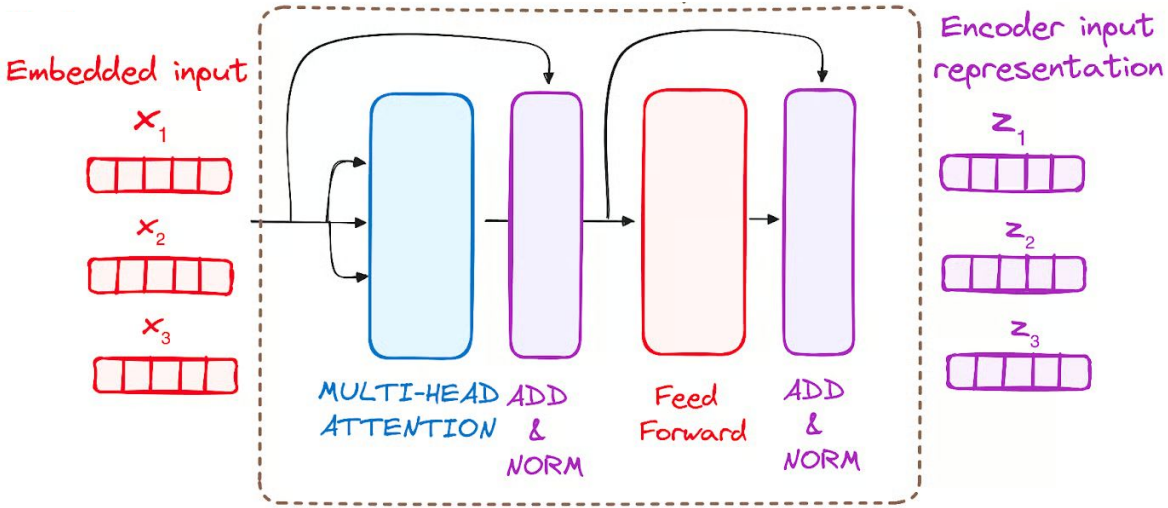
Something we've already seen



Something we've already seen



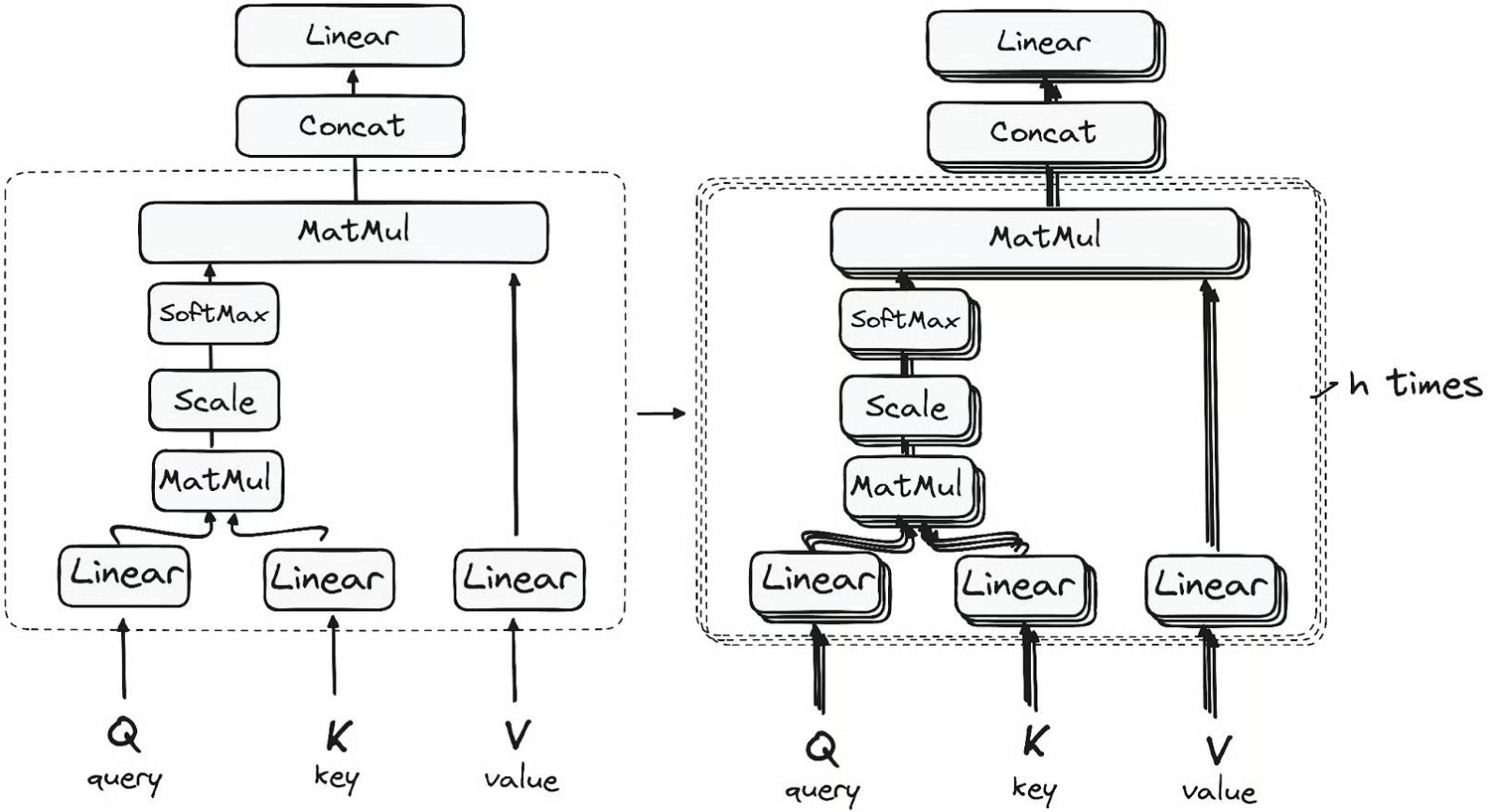
Attention is all we need



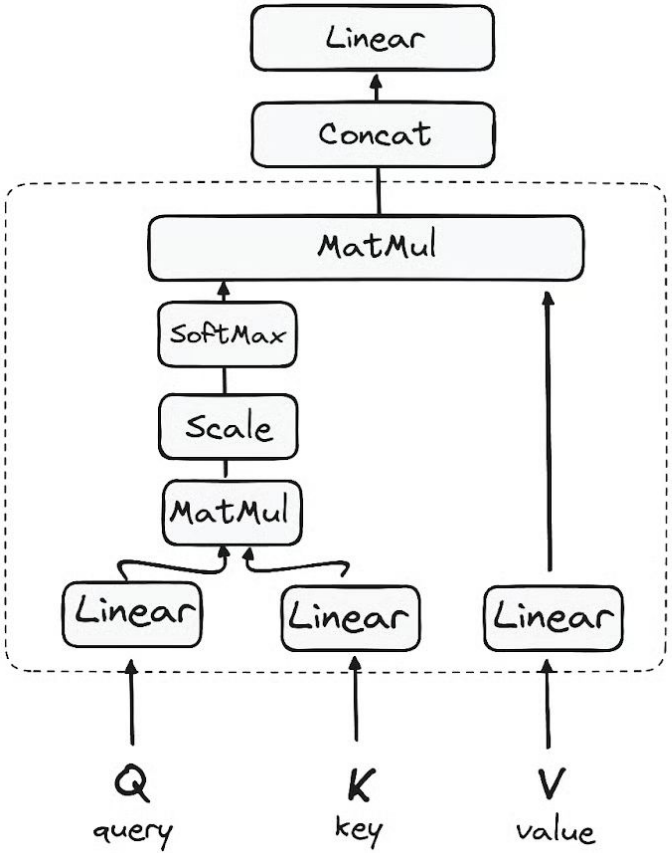
Transformer layers allows for the exchange of information between tokens.

- Each token sends a message to every other token.
- Each token computes how much does it want to take into account the messages of each other token. This is the idea of **attention**.
- Each token moves away from its embedding in the direction signaled by the message of each other token weighted by the attention that it is paying to that token.

Multi-head attention: attention mechanism performed in parallel



From input embeddings to query, key and value



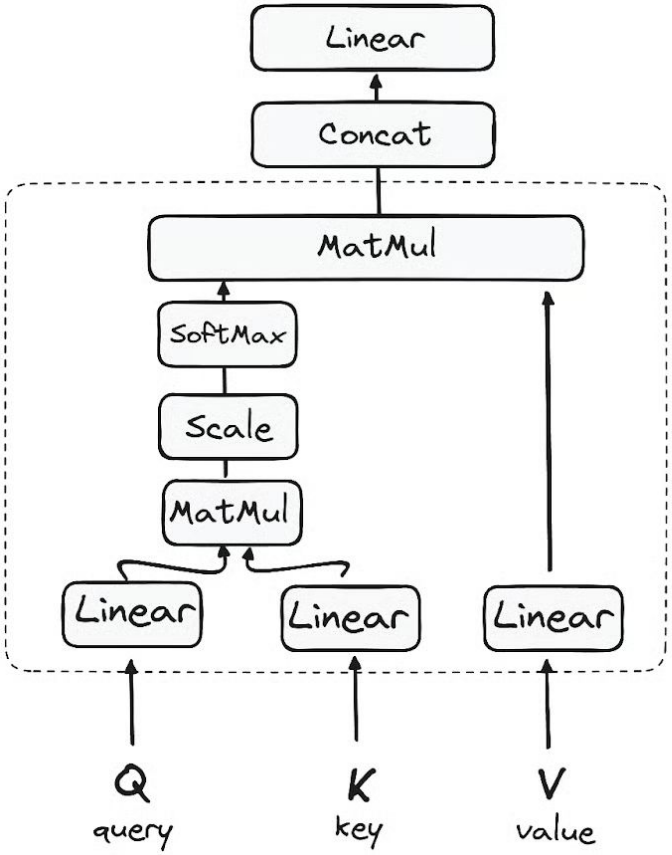
Processing different part of input embedding set while looking at a token:

- **Query**: what am I looking for? / what I want to pay attention to?
- **Key**: what do I contain? / what I offer for others to attend to?
- **Value**: what information I'll pass along if I'm selected?

Query, Key and **Value** are just matrices!

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^{\mathbf{Q}}; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^{\mathbf{K}}; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^{\mathbf{V}}$$

How does self-attention work

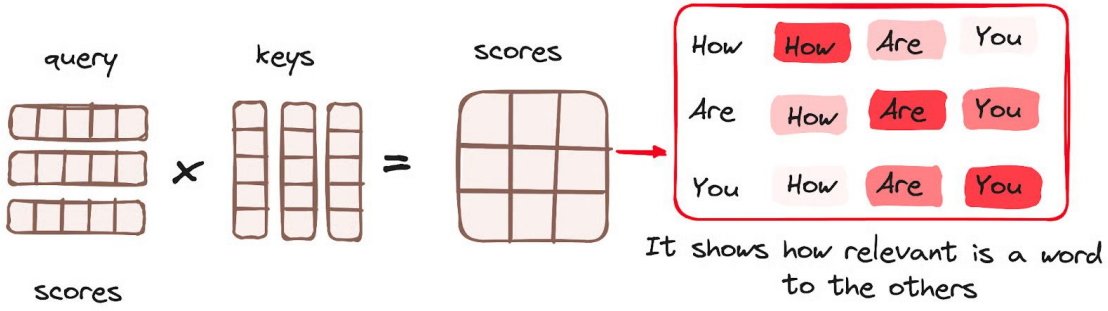


Processing different part of input embedding set while looking at a token:

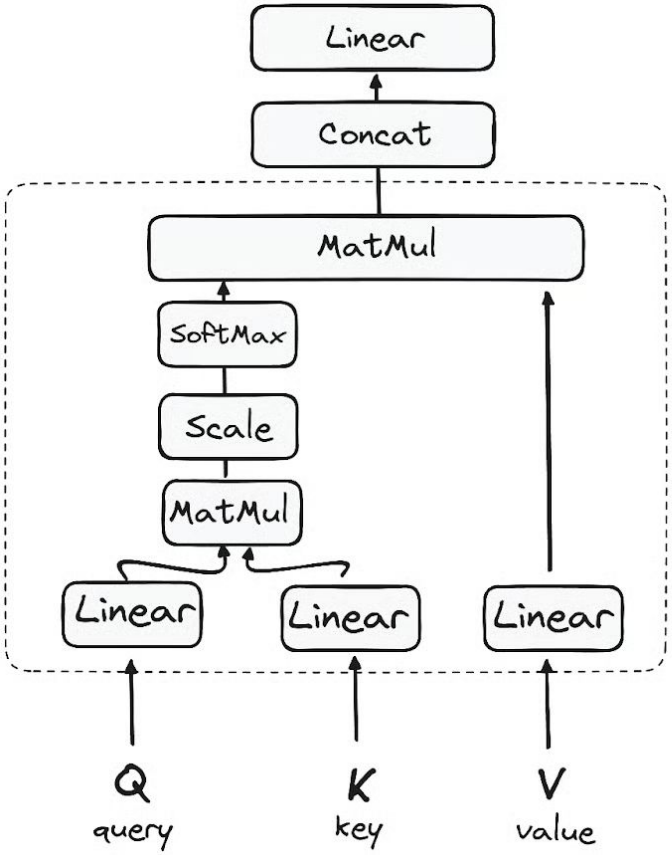
- **Query**: what am I looking for? / what I want to pay attention to?
- **Key**: what do I contain? / what I offer for others to attend to?
- **Value**: what information I'll pass along if I'm selected?

Query, Key and **Value** are just matrices!

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^{\mathbf{Q}}; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^{\mathbf{K}}; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^{\mathbf{V}}$$



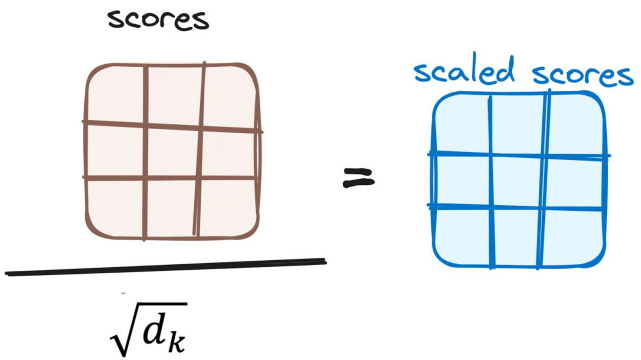
How does self-attention work



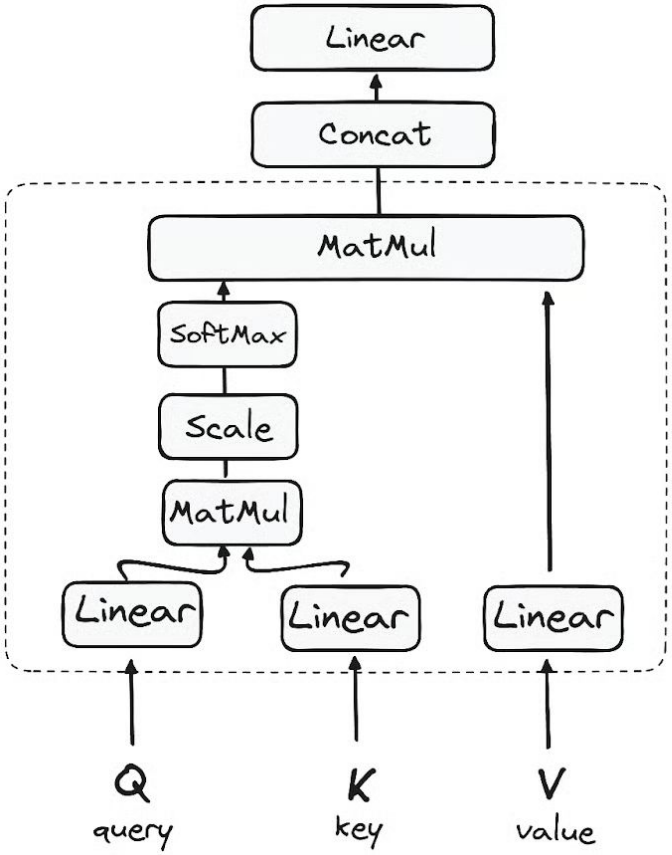
Processing different part of input embedding set while looking at a token:

- **Query**: what am I looking for? / what I want to pay attention to?
- **Key**: what do I contain? / what I offer for others to attend to?
- **Value**: what information I'll pass along if I'm selected?

Query, Key and **Value** are just matrices!



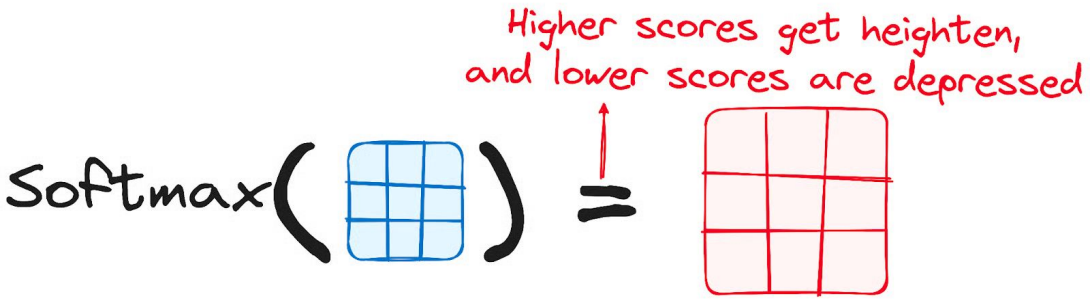
How does self-attention work



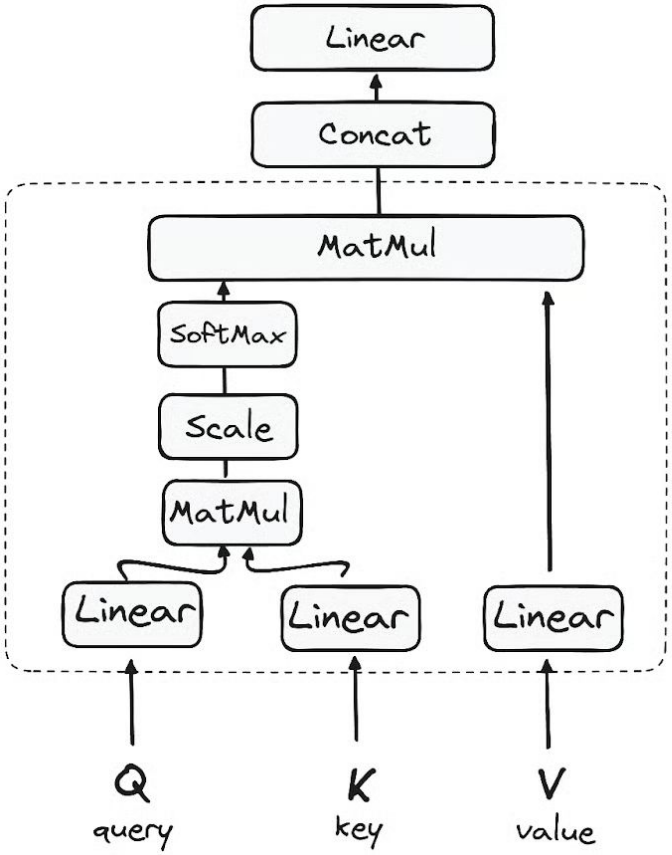
Processing different part of input embedding set while looking at a token:

- **Query**: what am I looking for? / what I want to pay attention to?
- **Key**: what do I contain? / what I offer for others to attend to?
- **Value**: what information I'll pass along if I'm selected?

Query, Key and **Value** are just matrices!



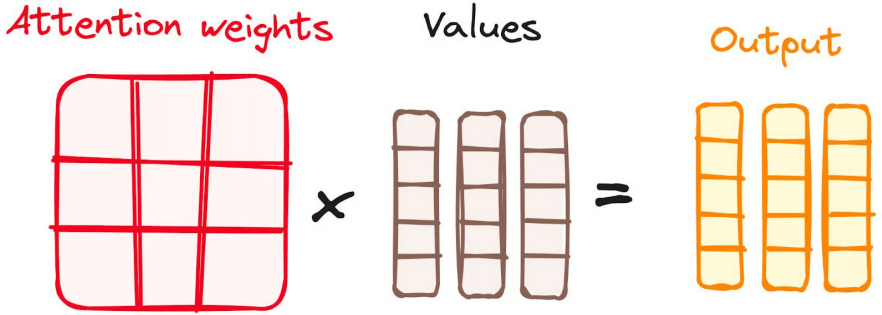
How does self-attention work



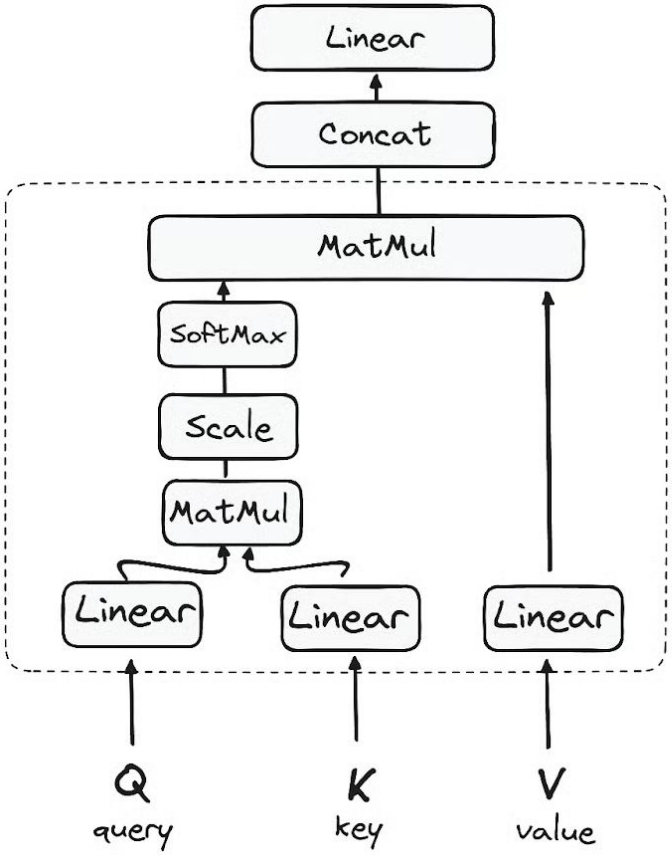
Processing different part of input embedding set while looking at a token:

- **Query**: what am I looking for? / what I want to pay attention to?
- **Key**: what do I contain? / what I offer for others to attend to?
- **Value**: what information I'll pass along if I'm selected?

Query, Key and **Value** are just matrices!



How does self-attention work



Processing different part of input embedding set while looking at a token:

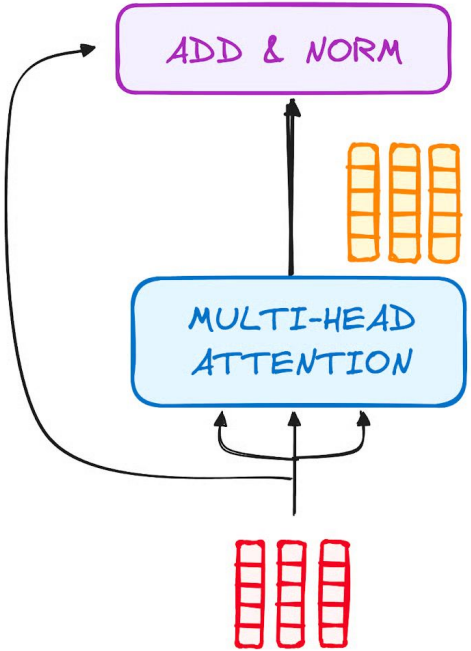
- **Query**: what am I looking for? / what I want to pay attention to?
- **Key**: what do I contain? / what I offer for others to attend to?
- **Value**: what information I'll pass along if I'm selected?

Query, Key and **Value** are just matrices!

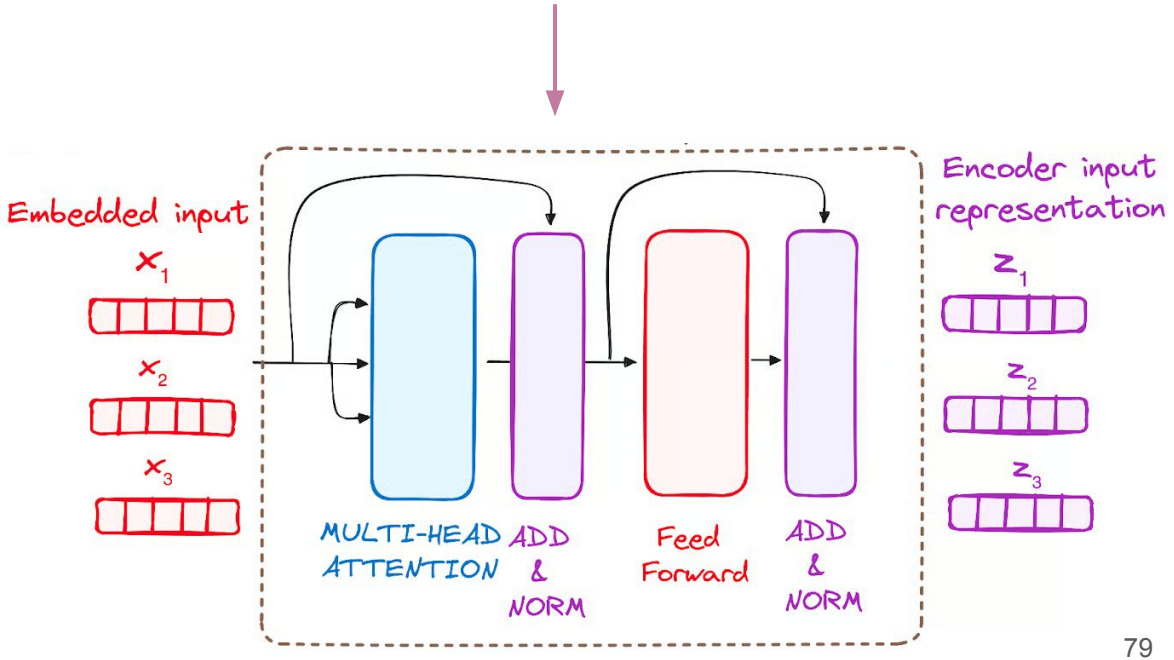
$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^{\mathbf{Q}}; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^{\mathbf{K}}; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^{\mathbf{V}}$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\text{T}}}{\sqrt{d_k}}\right)V$$

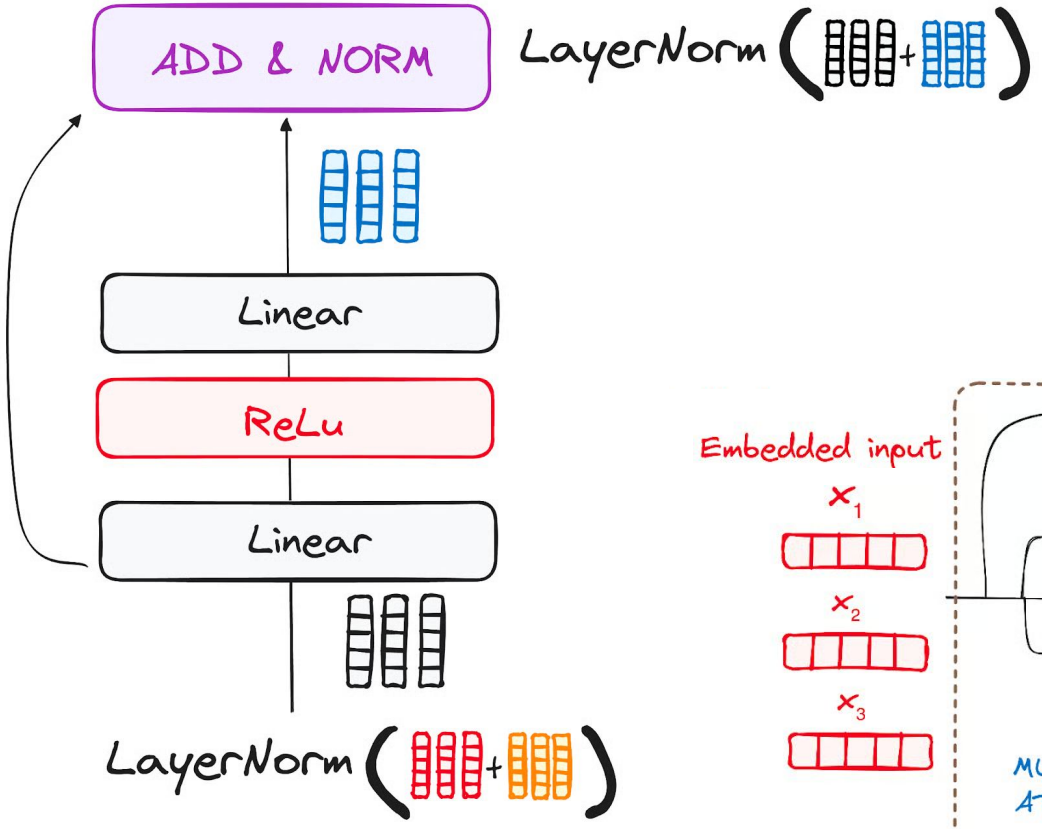
How does self-attention work

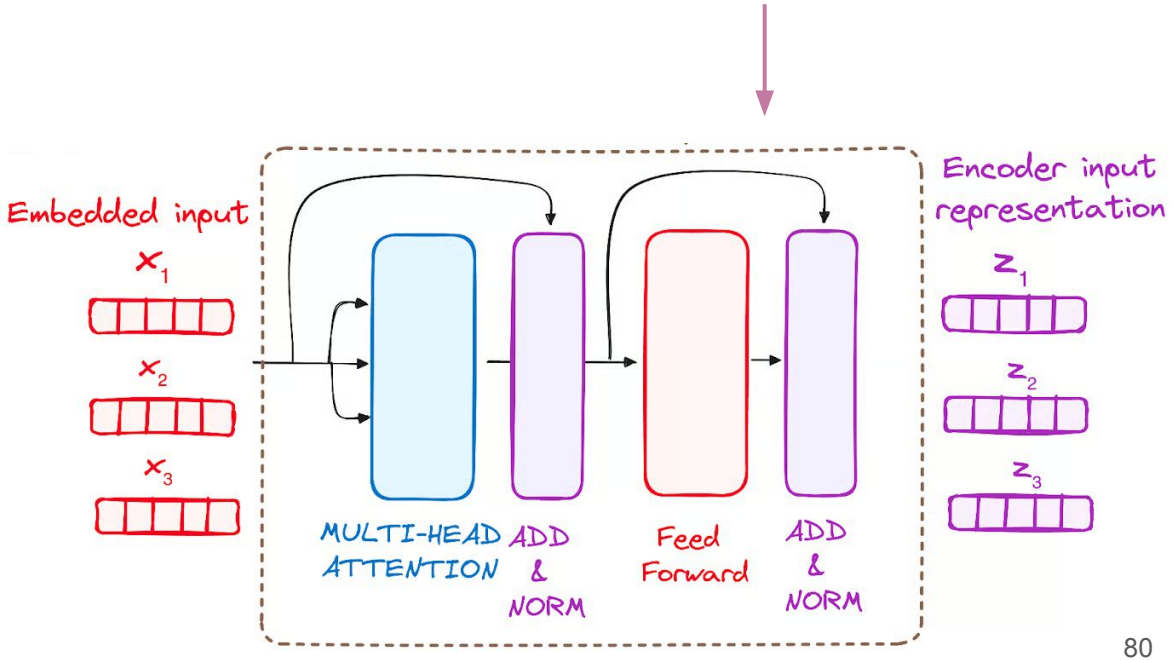




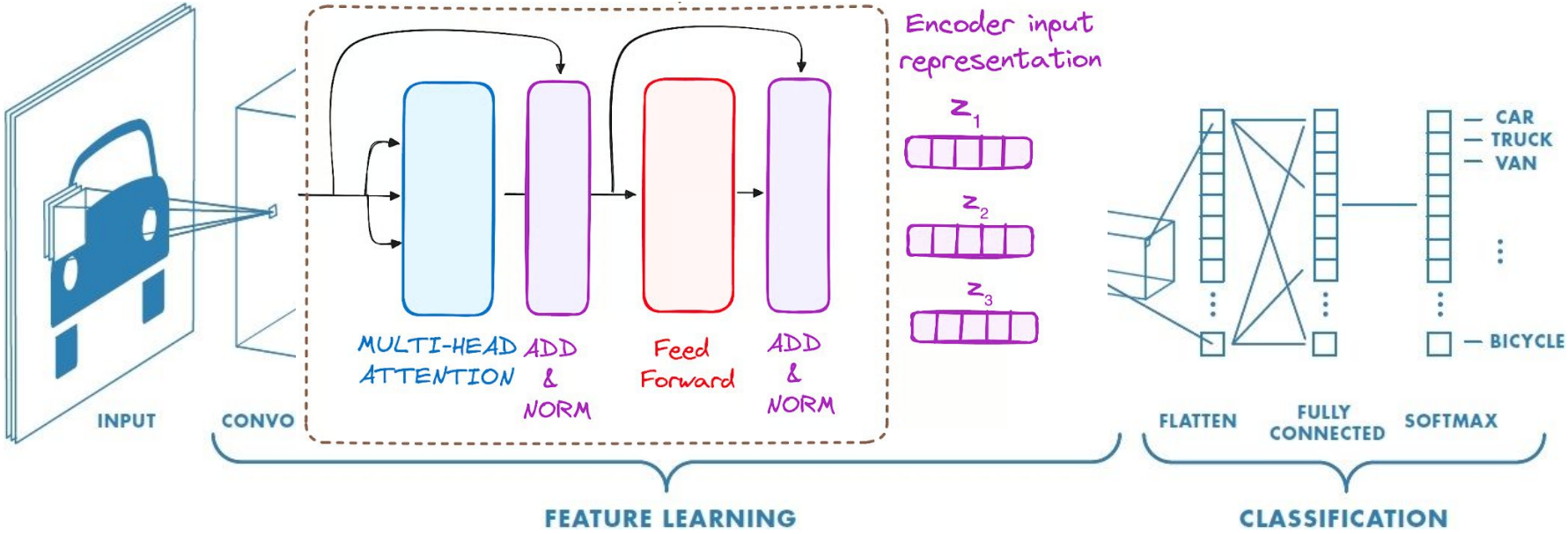


How does self-attention work

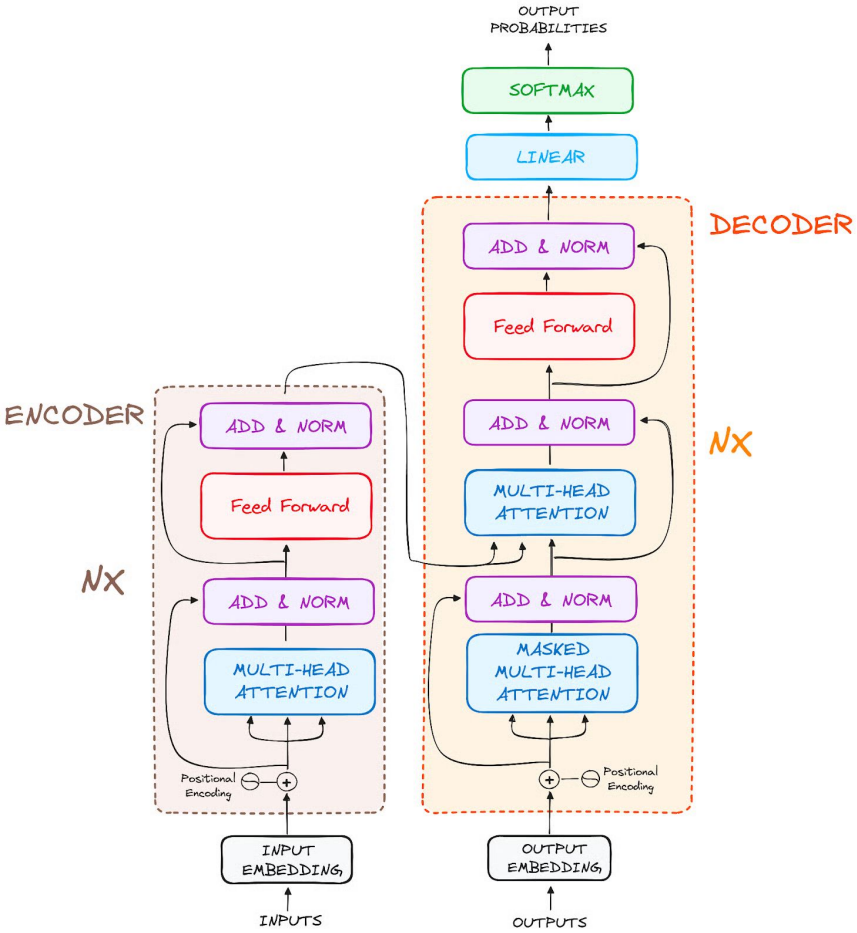




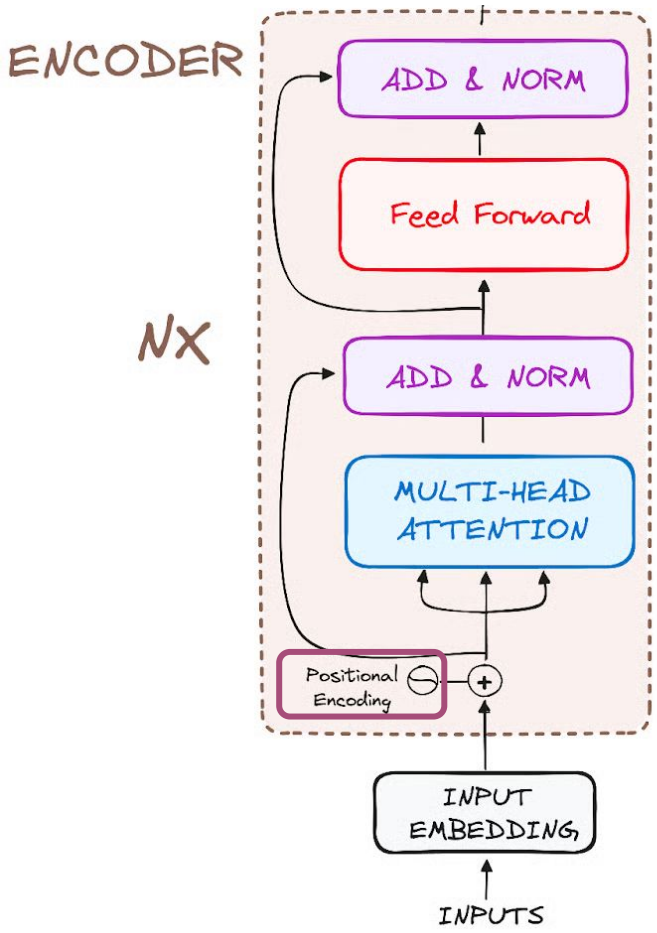
Transformer for feature extraction



Transformer for generative tasks

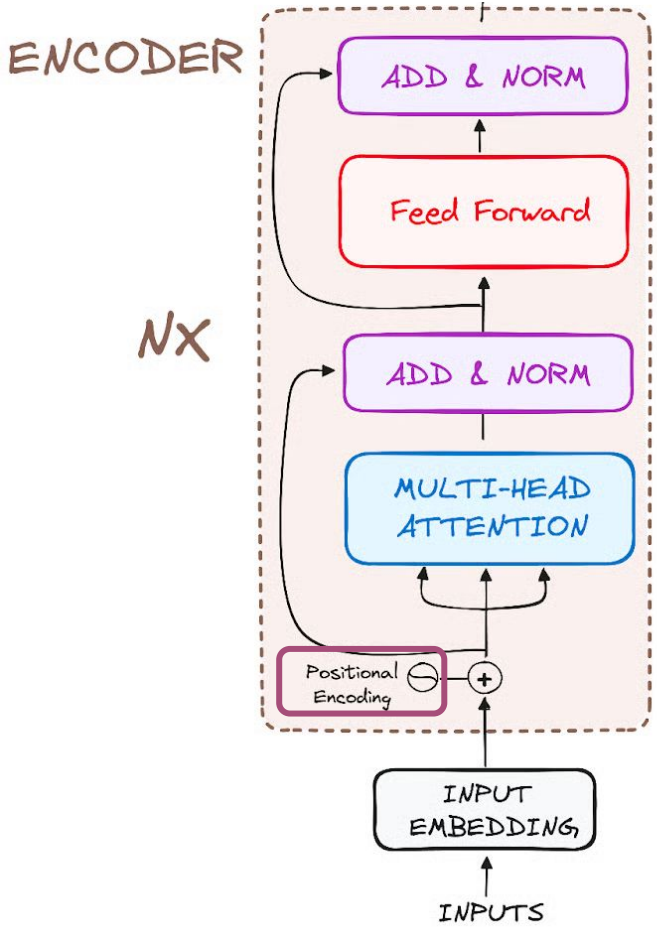


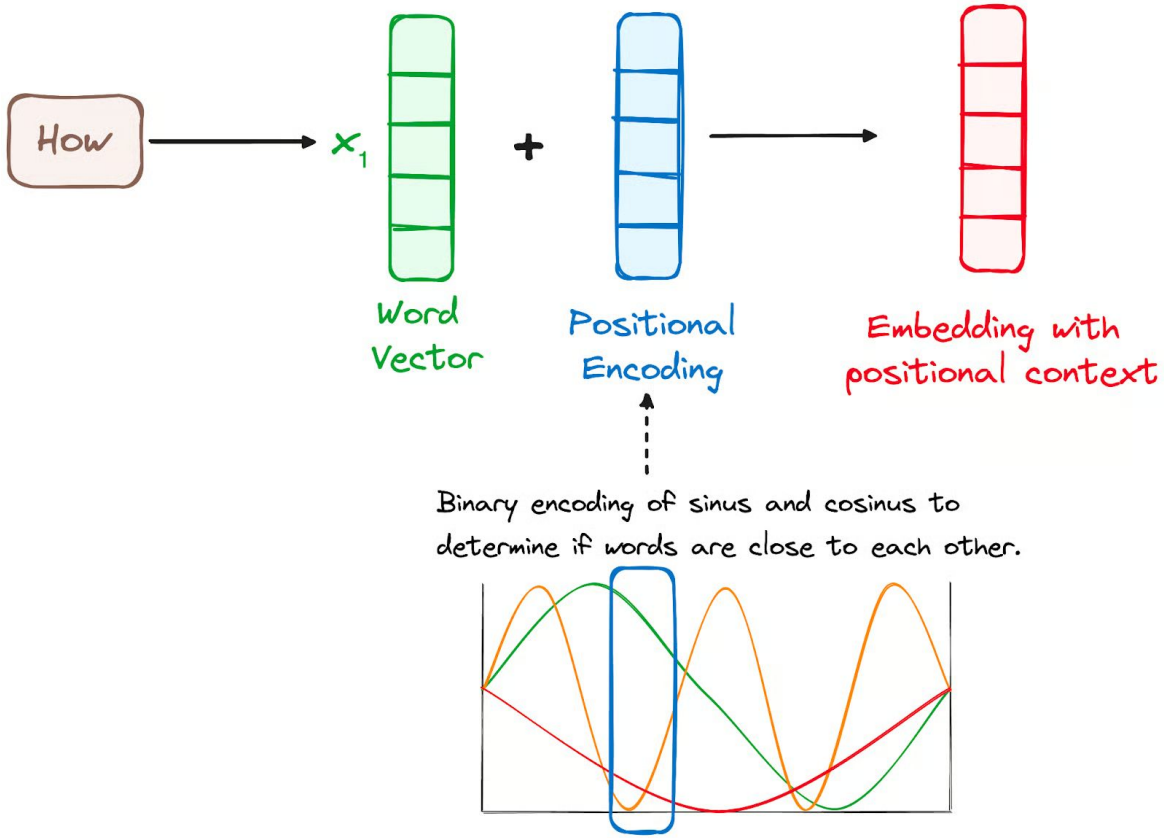
Self-attention is permutation equivariant



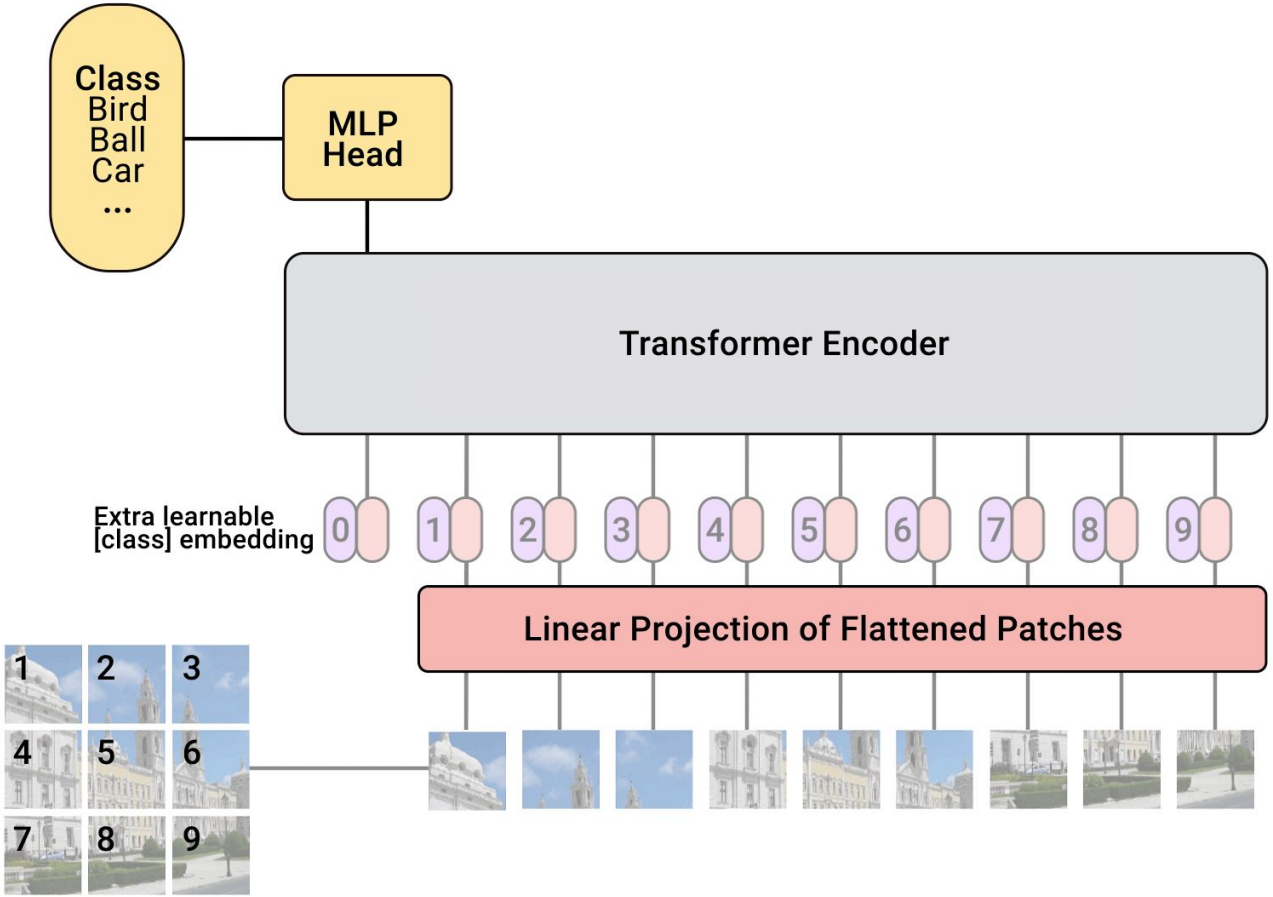
Each token embedding goes through the same MLP independently

Self-attention is permutation equivariant without positional encoding

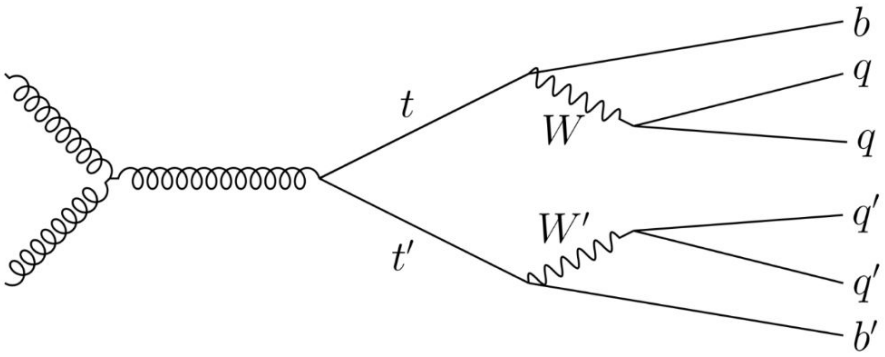




Transformer for image classification (ViT)



Transformer for jet matching (SPANet)



Set of Jets

$$\{j_1, j_2, j_3, j_4, j_5, j_6, j_7, j_8\}$$

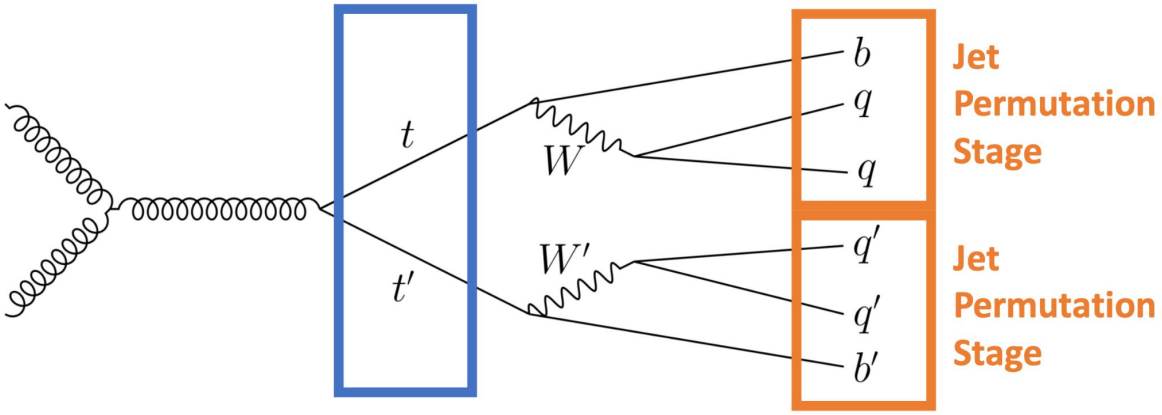
Match Jets to Parton Labels

$$\{b, q', \emptyset, q', b', \emptyset, q, q\}$$

Targets

Garbage Jets

Transformer for jet matching (SPANet)



$$q_1 q_2 b q_1' q_2' b' \leftrightarrow q_2 q_1 b q_1' q_2' b'$$
$$q_1 q_2 b q_1' q_2' b' \leftrightarrow q_1 q_2 b q_2' q_1' b'$$

We call these **jet symmetries.**
We will handle this with **attention**.

$$\underset{\mathcal{T}_1}{q_1} \underset{\mathcal{T}_2}{q_2} b \underset{\mathcal{T}_2}{q_1'} \underset{\mathcal{T}_1}{q_2'} b' \leftrightarrow \underset{\mathcal{T}_2}{q_1'} \underset{\mathcal{T}_1}{q_2'} b' \underset{\mathcal{T}_1}{q_1} \underset{\mathcal{T}_2}{q_2} b$$

We call this **particle symmetries.**
We will handle this with a **special loss function**.

Read more about transformers

- Transformer explainer (more options suggested here)
- Visual intro
- Intro on visual
- Coursera

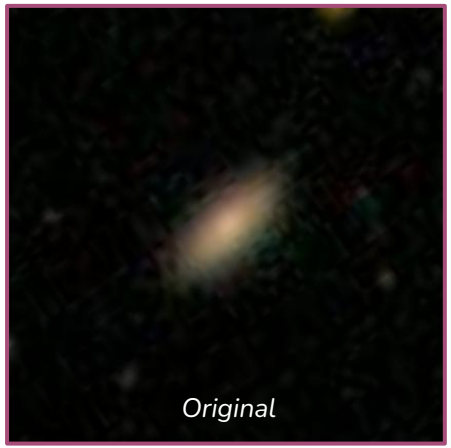
Data Augmentation

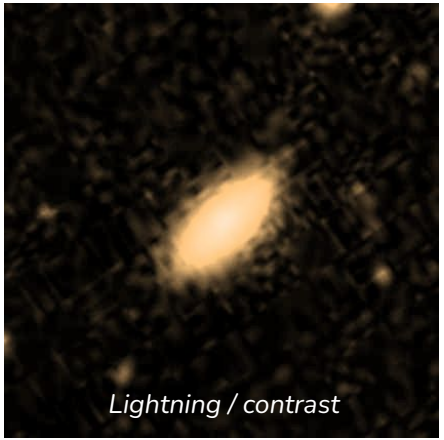
Hand-craft training examples can expose relevant corners of the solution space

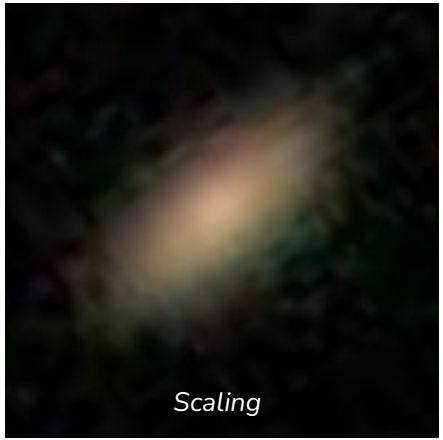
Data-Augmentation (DA) is a data-driven and informed regularization strategy that artificially increases the number of training samples.

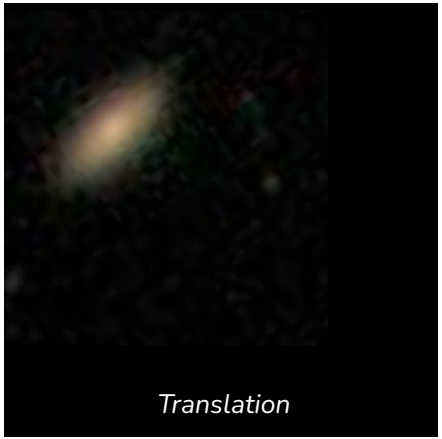
- DA is an *implicit* regularization, as it is not a function of a model's parameter but a function of the training samples.
- DA can be used to learn *invariance* or *equivariance* to specific transformations, even when those are not groups.
- DA requires more *domain knowledge* to be successful than weight decay (luckily we have plenty of it in scientific domains! :))

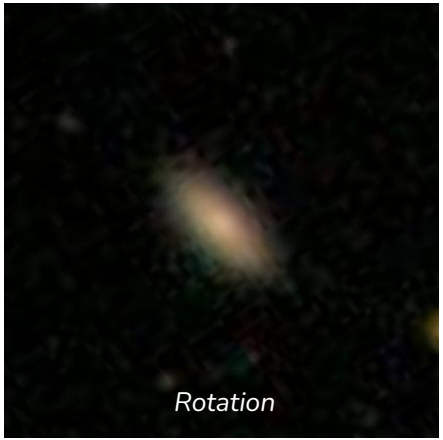
Made it for translation, but what about other invariants?









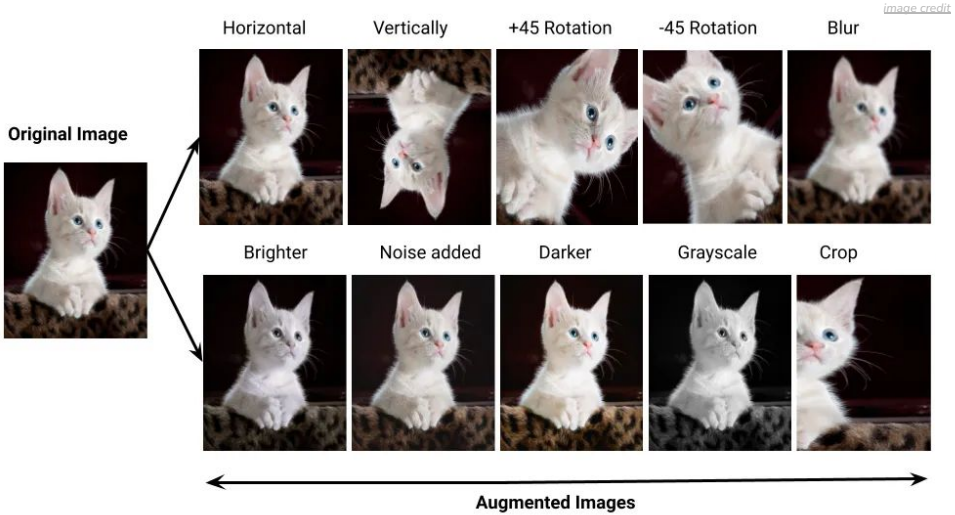


Data Augmentation

Hand-craft training examples can expose relevant corners of the solution space

Data-Augmentation (DA) is a data-driven and informed regularization strategy that artificially increases the number of training samples.

- DA is an *implicit* regularization, as it is not a function of a model's parameter but a function of the training samples.
- DA can be used to learn *invariance* or *equivariance* to specific transformations, even when those are not groups.
- DA requires more *domain knowledge* to be successful than weight decay (luckily we have plenty of it in scientific domains! :))



Data augmentation is largely employed in state-of-the-art **computer vision** models

For vision, the art of augmentations design is largely based on **heuristics**

Data Augmentation

Hand-craft training examples can expose relevant corners of the solution space

Data-Augmentation (DA) is a data-driven and informed regularization strategy that artificially increases the number of training samples.

- DA is an *implicit* regularization, as it is not a function of a model's parameter but a function of the training samples.
- DA can be used to learn *invariance* or *equivariance* to specific transformations, even when those are not groups.
- DA requires more *domain knowledge* to be successful than weight decay (luckily we have plenty of it in scientific domains! :))

In physics we can often define **controlled transformations** that are critical for successfully solve specific tasks.

Some examples:

high-level

low-level

- Physics laws: repeated simulations of a known stochastic physics process using different seeds provide different views of the same *physics concept* and help generalize
- Model uncertainties: simulated data varying the model parameters within uncertainties can be used to learn *domain shifts* or make a model robust against them.
- Experimental conditions: changing in the experimental setup that impact on the outcome of the observation of various physical phenomena can also be used to learn or mitigate unwanted *domain shifts*
- Symmetries: transformations that leave the solution invariant or modify it in an equivariant way are often independent of the specific physics phenomenon

Domain knowledge

Physics-informed ML

Summary

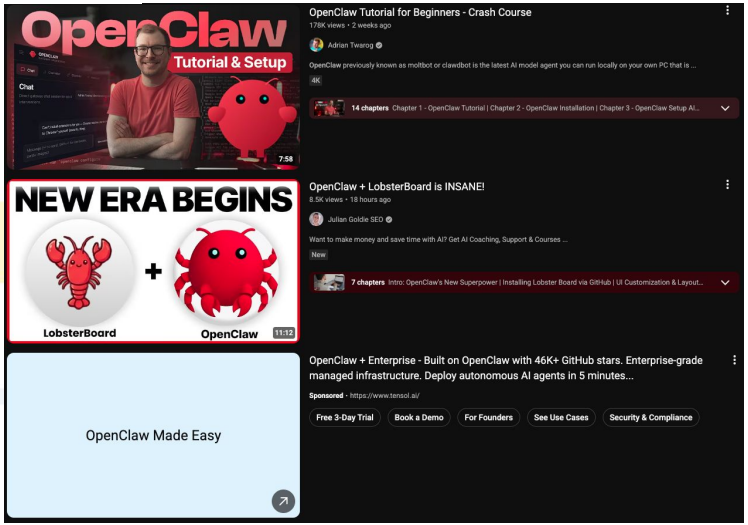
- Informing ML = the art of inducing bias
- First step: recognize the properties and structures of your data and your task
- Many different ways to guide ML:
  - *Data engineering* (representation, selection, augmentations)
  - *Architecture design* (shape connections and activation, embed symmetries)
  - *Shaping the loss landscape* (penalties, regularizations)

- Assumptions = Efficiency, interpretability, robustness
- Assumptions = Constraints

Agents on the OpenClaw example

OpenClaw:

openclaw Public | Watch 1358 | Fork 50.4k | Star 263k



Explore search trends

openclaw
Search term

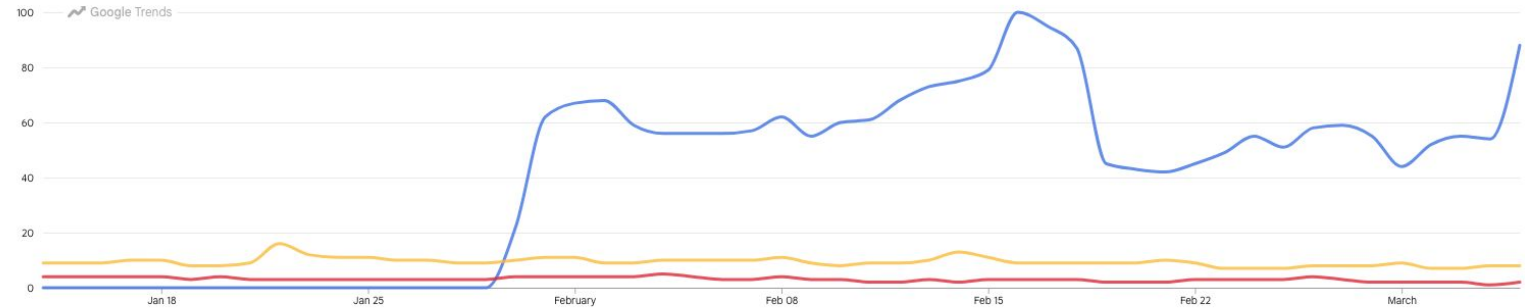
neutrino
Search term

supernova
Search term

Worldwide | Jan 15 - Mar 5 | Web Search

Interest over time

Worldwide · Jan 15 - Mar 5





What's OpenClaw? (based on medium post)

"*OpenClaw is an open-source framework for building autonomous AI agents that can*

- *run continuously (without supervising),*
- *acting on instructions,*
- *remember context, and*
- *take actions on external services*

— all on your own hardware."

What's OpenClaw? (based on medium post)

"*OpenClaw is an open-source framework for building autonomous AI agents that can*

- *run continuously (without supervising),*
- *acting on instructions,*
- *remember context, and*
- *take actions on external services*

— all on your own hardware."

```
Plan a weekend trip to Kyoto.
```

```
search flights
↓
check your calendar availability
↓
compare hotels
↓
generate itinerary
↓
send itinerary to you by email
```

```
Process today's emails.
```

```
read emails
↓
categorize (urgent / info / spam)
↓
draft replies
↓
schedule meetings mentioned in emails
```

```
Fix failing tests in this project.
```

```
run tests
↓
inspect error logs
↓
edit code
↓
run tests again
↓
repeat until tests pass
```

How does OpenClaw work? (based on medium post)



Installing OpenClaw
(your laptop, server, cloud)

How does OpenClaw work? (based on medium post)



Gateway

Communication channel
(Telegram, WhatsApp, Slack,etc)

```
Plan a weekend trip to Kyoto.
```

```
Process today's emails.
```

```
Fix failing tests in this project.
```

How does OpenClaw work? (based on medium post)



Gateway

Skills

Plug-in capabilities with triggers, permissions, and instructions (code)

ClawHub: community marketplace with ~10^3 skills

```python
def read_file(path: str) -> str:
    """Read a text file and return its contents."""

    with open(path, "r", encoding="utf-8") as f:
        content = f.read()

    return content
```

.py function

```yaml
name: read_file
description: Read the contents of a text file.
inputs:
  path:
    type: string
    description: Path to the file
outputs:
  content:
    type: string
    description: File contents
```

.yaml description

How does OpenClaw work? (based on medium post)



Gateway

Skills

Memory

Persistent context stored in local files

JSON

```
{
  "user_preferences": {
    "preferred_language": "English",
    "timezone": "UTC+9",
    "editor": "VSCode"
  },
  "known_projects": {
    "data_analysis_tool": {
      "repository": "/home/user/projects/data-analysis",
      "language": "python"
    }
  },
```

How does OpenClaw work? (based on medium post)



Model-Agnostic:
plug in any LLM
(Claude, GPT, etc)

Gateway

Skills

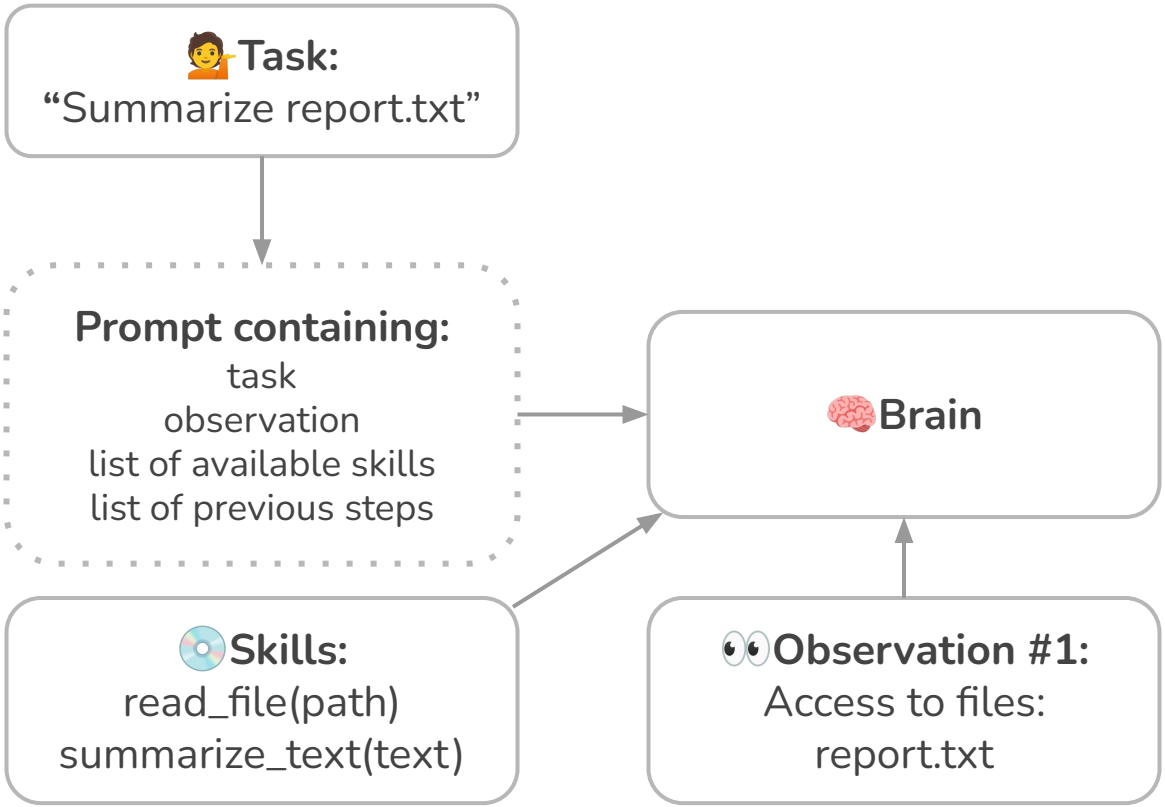
Memory

Brain

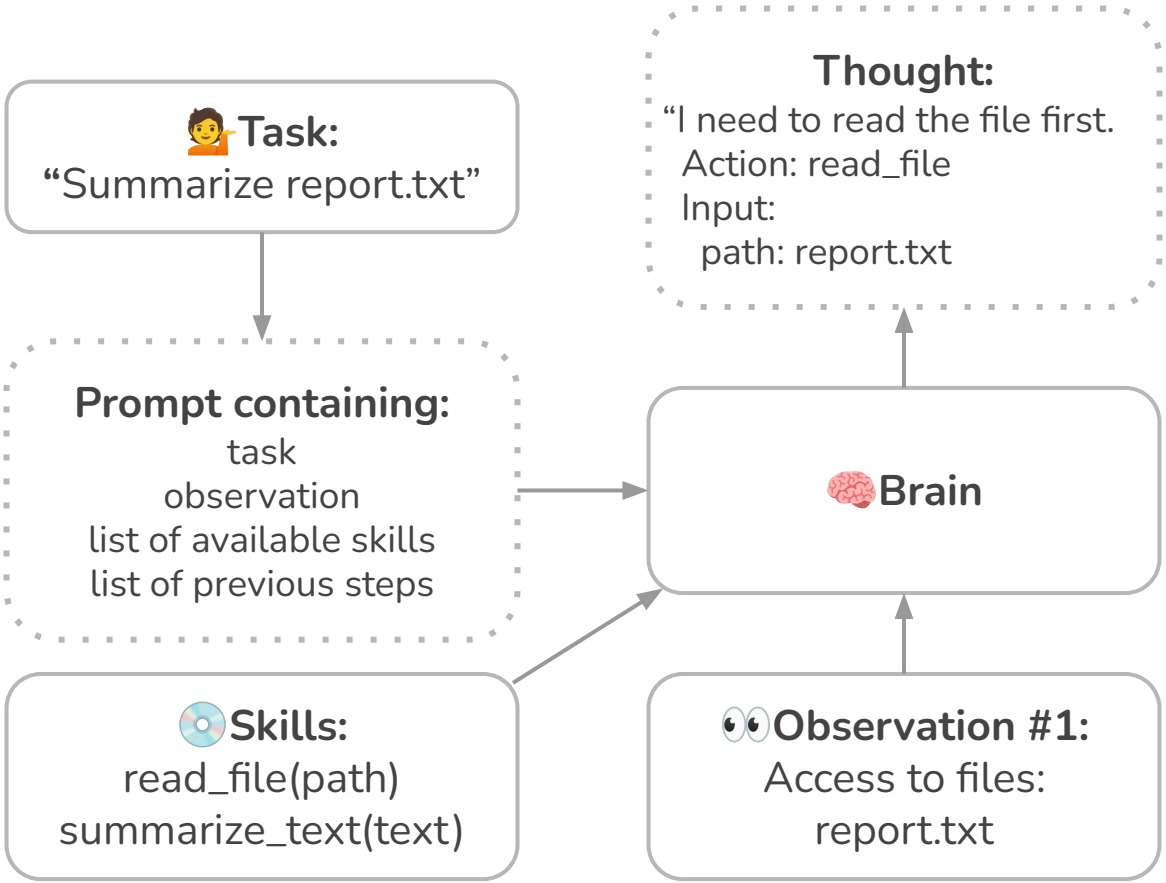
Reasoning and decision-making with LLM

```
Brain(task, observation, skills, memory) → action
```

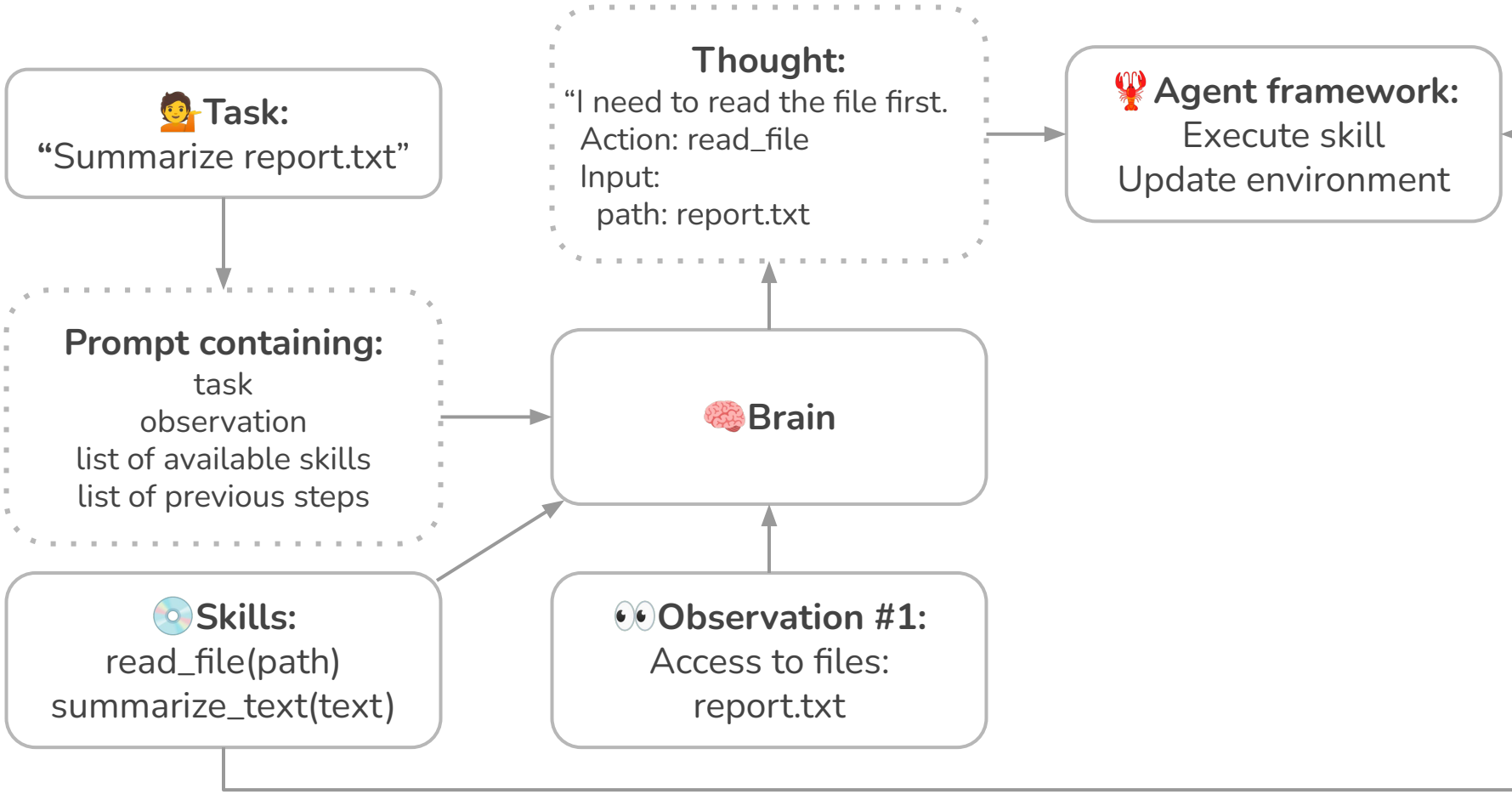
How does OpenClaw work? (based on medium post)



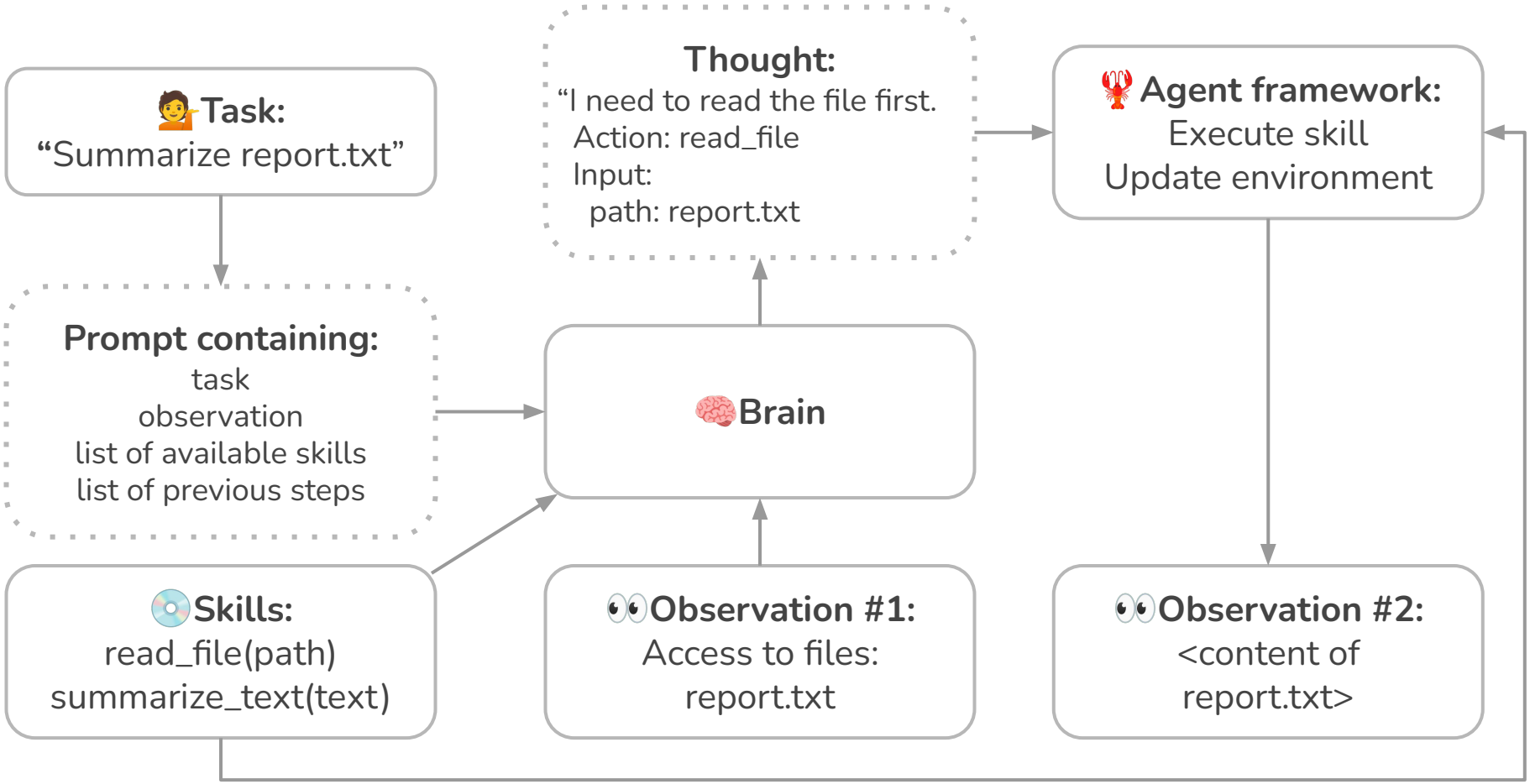
How does OpenClaw work? (based on medium post)



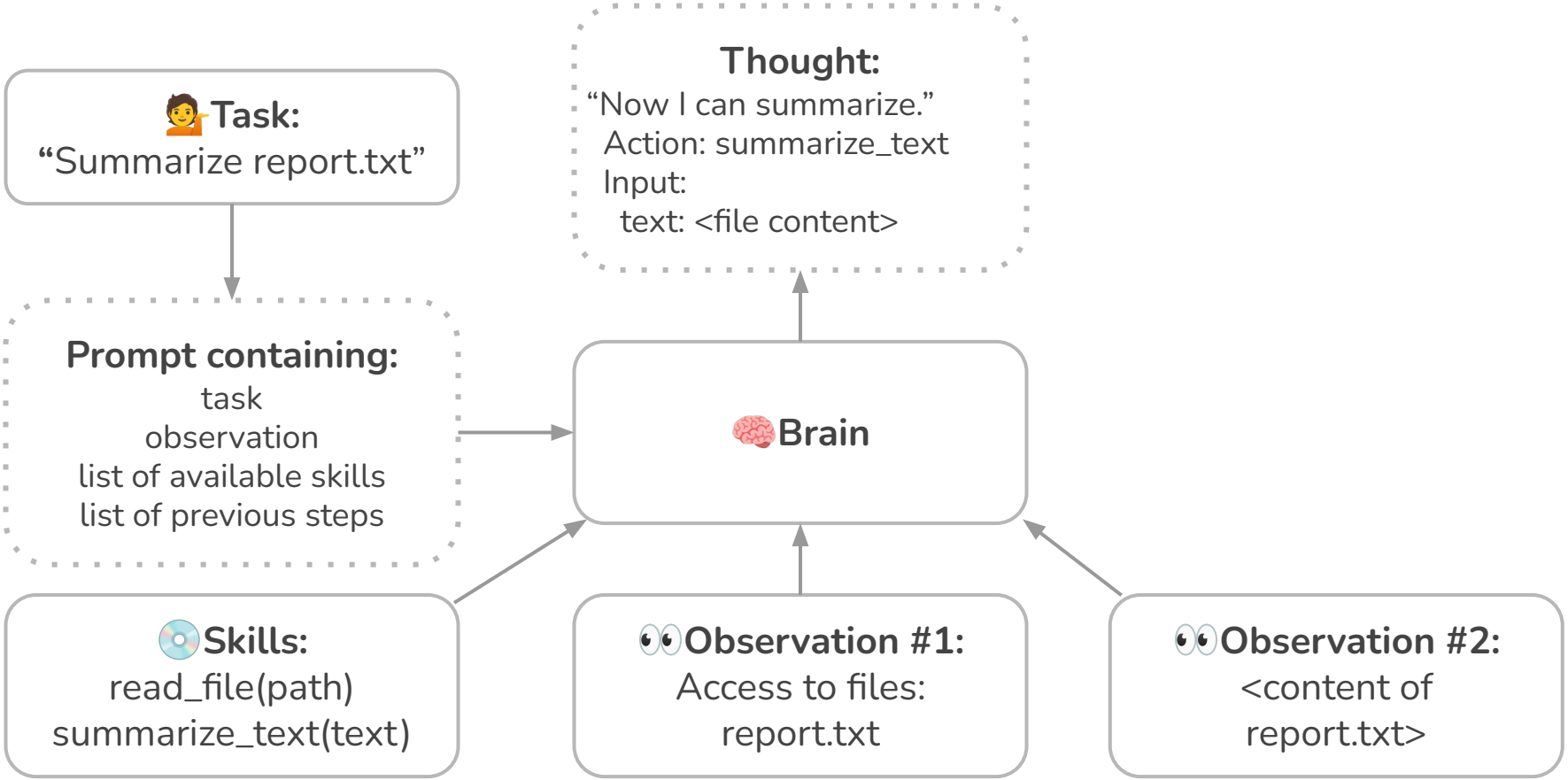
How does OpenClaw work? (based on medium post)



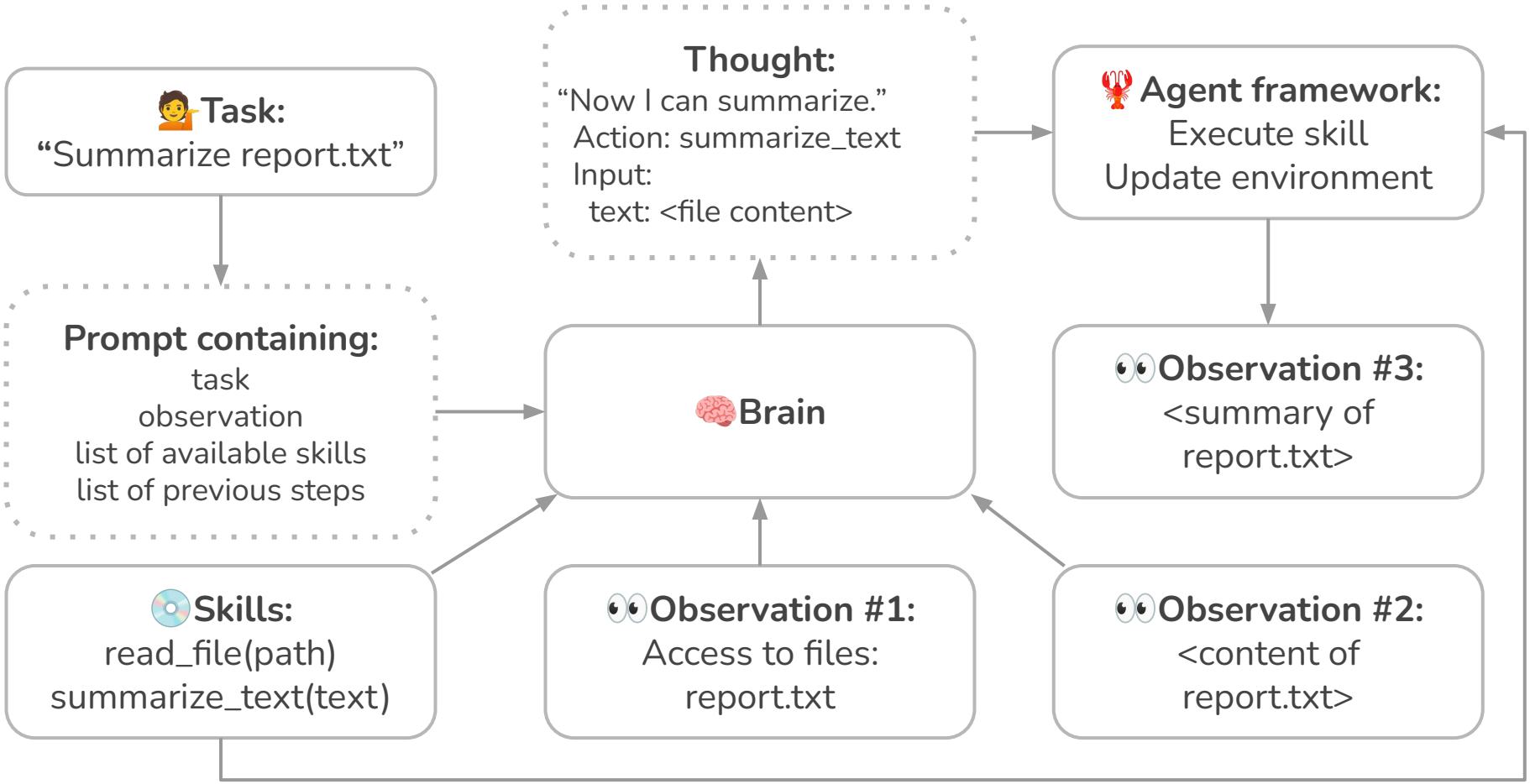
How does OpenClaw work? (based on medium post)



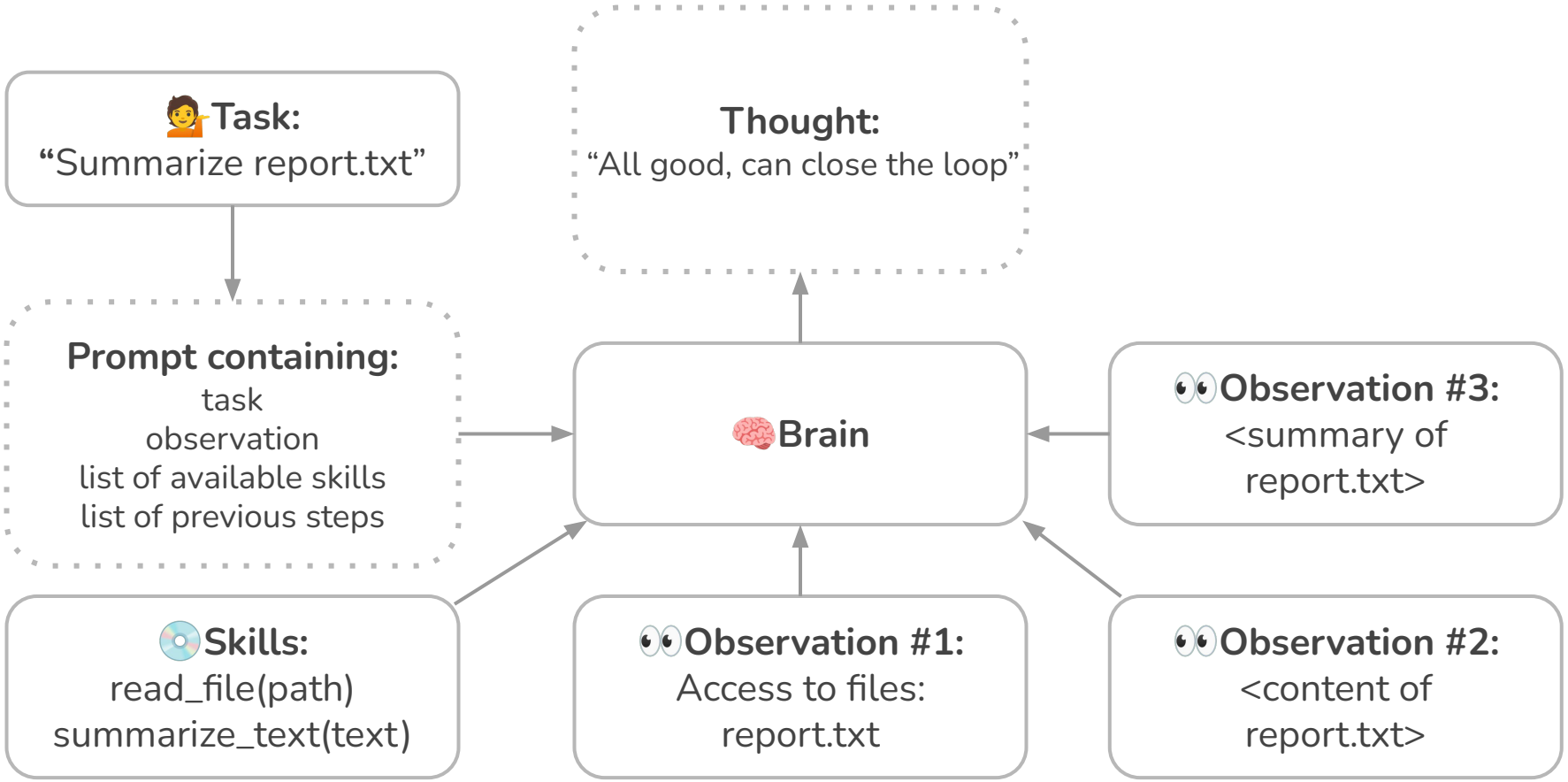
How does OpenClaw work? (based on medium post)



How does OpenClaw work? (based on medium post)



How does OpenClaw work? (based on medium post)



How does OpenClaw work? (based on medium post)



Gateway

Skills

Memory

Brain

Heartbeat

Signal telling the agent runtime:
“run the brain again immediately”

Personal take

- **OpenClaw makes AI agent system more available**
- **Privacy**: should be cautious setting up, especially when running on rented resources
- **Optimizable routine**:
  get a reminder to take umbrella on rainy days,
  automatically convert downloaded files e.g. from .jpeg to .png,
  remind of a saved cafe when see no plans for a weekend
- Wouldn't trust to handle my emails:)

